סיכום סיבוכיות וחישוביות - תשפ"ו 2026

28 ביולי 2026

גיא יער-און

הסיכום נכתב במהלך סמסטר ב' תשפ"ו (2026), ומבוסס על הרצאותיה של פרופסור טלי קאופמן ומבוסס על הקלטות הרצאות 2025 של פרופסור דודי מס, לכן יתכן שנפלו טעויות בסיכום, כך שהשימוש בו על אחריותכם בלבד.

תוכן עניינים

1 הרצאה 1 2
1.1 בעיות חיפוש 2
1.1.1 דוגמאות חשובות לבעיות חיפוש 3
1.2 בעיות הכרעה 3
1.3 פתרון בעיות חיפוש והכרעה 3
1.4 פתרון יעיל וסיבוכיות זמן 4
1.5 מחלקות של בעיות חיפוש 4
1.6 מחלקות של בעיות הכרעה 5
1.7 תרגול 1 6
2 הרצאה 2 7
2.1 הקשר בין בעיות חיפוש לבעיות הכרעה 7
2.2 רדוקציות 8
2.3 רדוקציית קוק 8
2.4 רדוקציות קארפ 9
2.5 תרגול 2 10
3 הרצאה 3 12
3.1 NP-שלמות 12
3.2 משפט קוק-לוין 13
4 הרצאה 4 15
4.1 בעיות NP שלמות שעוסקות בגרפים 15
4.2 תרגול 4 18
5 הרצאה 5 20
5.1 המשך בעיות בגרפים בNPC 20
5.2 בעיות מספרים בNPC 21
5.3 תרגול 5 22
6 הרצאה 6 23
6.1 רדוקציות עצמיות 23
6.2 המחלקה $CoNP$ 24
6.3 תרגול 6 25
7 הרצאה 7 26
7.1 ההירכיה הפולינומית 27
7.2 תחת ההנחה $P = NP$: $PH = P$ 28
7.3 תרגול 7 30
8 הרצאה 8 30
8.1 חישוב לא דטרמיניסטי 30
8.2 הגדרה שקולה להירכיה הפולינומית 31

8.3 חישוב לא יוניפורמי 32
8.3.1 מודל המעגלים 32
8.4 תרגול 8 32
9 הרצאה 9 33
9.1 מכונות טיורינג שמקבלות עצה 33
9.2 מכונת טיורינג עם עצה שקולה לסדרת מעגלים 33
9.3 $NP \subseteq P/Poly \implies PH = \Sigma_2$ 34
9.4 תרגול 35
10 הרצאה 10 36
10.1 סיבוכיות מקום 36
10.2 NL ־ שלמות 37
10.3 תרגול 38
11 הרצאה 11 40
11.1 משפט סביץ' 40
11.2 משפט אימרמן (1987) 41
11.3 תרגול 42
12 הרצאה 12 44
12.1 אלגוריתמים הסתברותיים 44
12.2 תרגול 47
13 סיכום 50
13.1 רדוקציות 50
13.2 בעיות NPC 50
13.3 שאלות פתוחות 50
13.4 סגירויות וטענות חשובות 50
13.5 הגדרות של מחלקות 51
13.6 בעיות NLC 51
13.7 הסתברות 51

1 הרצאה 1

בסיום הקורס "מודלים חישוביים" דנו בשאלה: "מה ניתן לחשב באמצעות מחשב?", וראינו שישנן בעיות כמו בעיית אימות תוכנה (ATM) שלא ניתנת לפתרון באמצעות מחשב, וראינו בעיות רבות נוספות שלא ניתנות לפתרון באמצעות מחשב. בקורס זה, נדון בשאלה: "מה אפשר לחשב באמצעות מחשב באופן יעיל?". כלומר, יתכן שהזמן שניתן לפתור בעיה מסויימת הוא אקספוננציאלי וזה לא באמת ישים לחישוב באמצעות מחשב (עבור $n \to \infty$). על יעילות אפשר להסתכל ממרחב היבטים ־ סיבוכיות זמן, מקום, כמה ביטים אקראיים ו/או אינטרקציה (כל אלו נקראים "משאבי חישוב").

מטרת הקורס: אילו בעיות ניתן לחשב באופן יעיל?
כעת, נרחיב על מושגים אלו: בעיה, חישוב, יעילות.

1.1 בעיות חיפוש

הגדרה 1. נגדיר בעיית חיפוש מהצורה הבאה: בהינתן בעיה, נדרש למצוא לה תשובה/פתרון.

לדוגמה: בהינתן גרף $G = (V, E)$ וזוג קודקודים $s, t \in V$, בעיית החיפוש המתאימה יכולה להיות: מצא את המסלול הקצר ביותר בין s ל־t".

הגדרה 2. נייצג בעיית חיפוש באמצעות אוסף של זוגות, כלומר: יחס $R \subseteq \{0,1\}^* \times \{0,1\}^*$ כאשר זוג $(x, y) \in R$ אם y הינו פתרון חוקי עבור בעיה x.

לדוגמה:

$$R_{ST-conn} = \{((G, s, t), P) | G \ Graph, \ P \ is \ a \ path \ from \ s \ to \ t \ in \ G\}$$

הערה 3. אנחנו מייצגים את בעיית החיפוש באמצעות $\{0,1\}^*$ כיוון שניתן לקודד בינארית כל קלט ופלט (בעיה ופתרון) לאפסים ואחדות.

1.1.1 דוגמאות חשובות לבעיות חיפוש

דוגמה 4. בעיית $3-COL$
קלט: נתון גרף $G=(V,E)$
פלט: פונקציית צביעה $\phi: V\to\{1,2,3\}$ ללא קשת מונוכרומטית (ללא קשת ששני קודקודיה צבועים באותה צבע), אם קיימת כזו.
פורמלית:

$$R_{3-COL}=\{(G=(V,E),\phi)|G \text{ is Graph},\phi: V\to\{1,2,3\}\}$$

דוגמה 5. בעיית SAT
קלט: נתונה נוסחה בוליאנית בצורת $3CNF$, נזכר כי צורת CNF היא כזו שביטויי "וגם" מפרידים בין פסוקיות שונות, כל חלק של סוגריים יקרא פסוקית ובתוך כל פסוקית ישנם משתנים בוליאניים (ליטרלים) מופרדים עם סימן $\vee$ בלבד ויכולים להופיע מפורשות או בצורת השלילה שלהם. $3CNF$ היא נוסחת CNF כנ"ל בה בכל פסוקית יש בדיוק 3 משתנים. למשל: $(X_1\vee X_2\vee\overline{X_3})\wedge(X_1\vee X_2\vee X_4)\wedge(X_7\vee\overline{X_2}\vee\overline{X_{90}})$.
פלט: פונקציית השמה למשתנים שנותנת השמה מספקת (כלומר, ערך הביטוי הבוליאני בצורת $3CNF$ יהיה $True$), אם קיימת כזו.

1.2 בעיות הכרעה

הגדרה 6. בעיית הכרעה: בהינתן קלט כלשהו, נדרש להכריע האם הוא מקיים תכונה כלשהי. התשובה היא בינארית: כלומר, כן או לא.

לדוגמה:
1. האם בהינתן $G=(V,E)$ גרף, G קשיר?
2. האם בהינתן $G=(V,E)$ וזוג קודקודים $s,t\in V$ קיים מסלול בין s ל-t?
3. יהי $x\in\mathbb{N}$. האם x הוא מס' ראשוני?

הגדרה 7. מידול בעיית הכרעה: נתאר בעיית הכרעה בעזרת קבוצה S באשר $x\in S$ אמ"מ x מקיים את התכונה שאנחנו מעוניינים בה π. ולכן, בהינתן קלט ההכרעה היא האם $x\in S$ (מקיים, כלומר התשובה היא כן) או שלא, ואז $x\notin S$.

לדוגמה:

$$S_{ST-conn}=\{(G,s,t)|G\ Graph\ ,\exists path\ from\ s\ to\ t\ in\ G\}$$

1.3 פתרון בעיות חיפוש והכרעה

על מנת לדון פורמלית בפתרון בעיות חיפוש והכרעה, נפתח בהגדרה רשמית של מושג האלגוריתם.

הגדרה 8. **אלגוריתם** הוא אוסף סופי של כללים חישוביים שמגדירים תהליך חישוב במודל "סביר".
המודל ה"סביר" בו נעסוק בקורס הוא מכונת טיורינג עם סרט אחד, מכונה שעוצרת על כל קלט.

הגדרה 9. עבור בעיית חיפוש R וקלט x נסמן ב-$R(x)$ את קבוצת הפתרונות החוקיים עבור x. כלומר,

$$R(x)=\{y|(x,y)\in R\}$$

כעת, נאמר כי אלגוריתם A פותר את בעיית החיפוש R אם מתקיימים התנאים הבאים לכל $x\in\{0,1\}^*$:

1. אם $R(x)\neq\emptyset$ (ל-x יש איזשהו פתרון ביחס), אזי $A(x)\in R(x)$ (A יחזיר פתרון כלשהו, אם יש כמה יחזיר שרירותי).
2. אם $R(x)=\emptyset$, אזי $A(x)=\perp$.

הערה 10. הסימון $\perp$ הוא סימון מוסכם לכך שאלגוריתם מחזיר "אין פתרון".

הגדרה 11. נאמר כי אלגוריתם A פותר בעיית הכרעה S אם לכל x מתקיים:
1. אם $x\in S$ אזי $A(x)=1$
2. אם $x\notin S$ אזי $A(x)=0$

1.4 פתרון יעיל וסיבוכיות זמן

הגדרה 12. תהי M מכונת טיורינג בעלת סרט בודד. עבור קלט כלשהו x נסמן ב$t_M(x)$ את מספר הצעדים שM מבצעת בריצה על x, בהנחה שM אכן עוצרת על x (אם לא עוצרת נאמר כי $t_M(x) = \infty$). **סיבוכיות הזמן** של M הינה פונקציה $T_M : \mathbb{N} \to \mathbb{N}$ שמוגדרת ע"י $T_M(n) = max_{x\in\{0,1\}^n}\{t_M(x)\}$. כלומר, מס' הצעדים המקסימלי שM מבצעת על קלט באורך n.

הערה 13. הmax מגיע על מנת לתאר את הworst case שהקלט שלנו יכול להגיע אליו, שהרי אנחנו תמיד מעוניינים להסתכל על המקרה הגרוע ביותר. לכן, עבור קלט ספציפי נסתכל על מס' הצעדים המקסימלי שהמכונה תבצע עליו, וזו סיבוכיות זמן הריצה.

הגדרה 14. נאמר כי מכונת טיורינג M הינה פולינומית אם קיים פולינום p כך שלכל n מתקיים: $T_M(n) \le p(n)$.

הגדרה 15. נאמר כי בעיית חיפוש/הכרעה ניתנת לפתרון **יעיל** אם קיימת מכונת טיורינג פולינומית עם סרט בודד שפותרת אותה.

הערה 16. **מוטביציה:** למה פתרון פולינומי הוא פתרון יעיל? פולינום חסום יחסית היטב ולא גדל מהר כמו אקספוננציאל, וכן ניתן להרכיב בעיות פולינומיות זו בזו, כלומר ניתן להשתמש באלגוריתם אחד בכמה וכמה אלגוריתמים פולינומיים ולקבל סה"כ אלגוריתם פולינומי. זה לא נכון עבור אלגוריתם לינארי, שהרי אם נריץ אותו בלולאה נקבל זמן ריבועי.

הערה 17. האם אנחנו מפסידים מכך שהדיון שלנו מתעסק במ"ט בעלת סרט בודד? כן. השפה הפשוטה הבאה מראה זאת:

$$S = \{xx | s \in \{0,1\}^*\}$$

במכונת טיורינג עם שני סרטים, היינו מעתיקים את הקלט לסרט השני, ומוודאים באמצעות שני הסרטים באמצעות סריקה עד אמצע הקלט שאכן $xx \in S$, זה יעלה כמובן $O(n)$

במכונת טיורינג בעלת סרט בודד לעומת זאת, נצטרך לבצע "הלוך חזור" על גבי הסרט, על מנת להשוות בכל שלב איבר בודד ולסמן ולחזור אחורה. זמן הריצה במקרה זה הוא $\Omega(n^2)$.

כלומר: אנחנו מפסידים בזמן הריצה. אך, נזכר בטענה הבאה ממודלים חישוביים:

משפט 18. (התזה של צ'רץ טיורינג) כל בעיה שניתנת לפתרון במודל חישובי כלשהו סביר", ניתנת לפתרון באמצעות מכונת טיורינג בעלת סרט בודד.

ונראה תזה חזקה אף יותר:

משפט 19. (התזה של *cobham-edmonds*) אם ניתן לפתור בעיה כלשהי במודל חישובי "סביר" בזמן $T(n)$, אזי ניתן לפתור אותה באמצעות מכונת טיורינג בעלת סרט בודד בזמן $Poly(T(n))$.

מסקנה. למרות שמפסידים בזמן הריצה לפעמים במ"ט בעלת סרט בודד, הפער בין פתרון הבעיה במודל זה יהיה פולינומי בלבד (לפי משפט 17), וזה נחשב יעיל לפי מה שהגדרנו קודם.

1.5 מחלקות של בעיות חיפוש

ישנן בעיות כמו "בהינתן קבוצה S, מהם כל תתי הקבוצות של S?". זו לא בעיה קשה, היא אפילו ממש קלה, אך רק הזמן שידרוש לי לכתוב את כל הקלט שלי ($2^{|S|}$) הוא אקספוננציאלי, ולכן זו אינה בעיה יעילה. לכן מראש, נתמקד בבעיות שאורך הפלט שלהן הוא פולינומי.

הגדרה 20. נאמר כי יחס R הינו חסום פולינומית אם קיים פולינום p כך שלכל זוג $(x, y) \in R$ מתקיים $|y| \le P(x)$.

הגדרה 21. נגדיר את המחלקה PF=Polynomial Find:

$PF = \{R|$ R חסום פולינומית, וקיים אלגוריתם פולינומי שפותר את R $\}$

למשל, $R_{ST-Conn} \in PF$.

הגדרה 22. נגדיר את המחלקה **PC=Polynomial Check**:

$PC = \{R|$ R חסום פולינומית, וקיים אלגוריתם פולינומי A שמקיים $(x, y) \in R \iff A(x, y) = 1$ $\}$

כלומר, מחלקת הבעיות שקיים להן אלגוריתם פולינומי שיודע להכריע לכל x, y האם y פותר את x. למשל, $R_{3-Color} \in PC$ כי בהינתן צביעה ניתן לעבור על קשתות הגרף ולוודא כי הצביעה חוקית בזמן לינארי אפילו, ובפרט פולינומי, אך זו שאלה פתוחה האם $R_{3-color} \in PF$.

נרצה לשאול את השאלה, מהו היחס בין המחלקות הללו? האם $PC \subseteq PF$? למעשה, איננו יכולים להכריע. לצורך העניין, $R_{3-color} \in PC$ אך איננו יודעים האם $R_{3-color} \in PF$. אך, האם $PF \subseteq FC$? התשובה היא לא. יש בעיות שאנחנו יודעים לפתור בזמן יעיל - אך לא יכולים לוודא פתרון בזמן יעיל. האינטואיציה לכך, היא שעל מנת לפתור עלינו להחזיר פתרון בודד. על מנת להיות במחלקה PC עלינו לדעת להתמודד עם כל זוג שהוא, זו בעיה קצת יותר קשה פתאום.

משפט 23. $PF \not\subseteq FC$

הוכחה: על מנת להוכיח, נגדיר את היחס הבא (בעיית חיפוש) R: $R = R_1 \cup R_2$, כאשר:

$R_1 = \{((M,x),$"עוצרת"$) |$ M מכונת טיורינג, $x \in \{0,1\}^*$, והמכונה עוצרת בריצה על $x\}$

$R_2 = \{((M,x),$"לא יודע"$) |$ M מכונת טיורינג, $x \in \{0,1\}^*$, כאשר תמיד ניתן להגיד על המכונה "לא יודע"$\}$

נטען כי $R \in PF$: לכן, נציג את האלגוריתם הבא: לכל קלט נחזיר "לא יודע", מדובר באלגוריתם פולינומי שרץ בזמן $O(1)$ שפותר את R. הוא פותר את הבעיה כיוון שבפרט הזוג $((M,x),$"לא יודע"$)$ תמיד נותן פתרון חוקי ביחס ליחס R (שמכיל הרי את R_2). שהרי, אלגוריתם שפותר בעיית חיפוש צריך להחזיר פתרון חוקי ביחס לR, הפתרון "לא יודע" אכן נחשב חוקי.

נטען כי $R \notin PC$: נב"ש כי קיים אלגוריתם A כך שבודק נכונות של פתרונות. נבנה אלגוריתם B שפותר את $HALT$ (בעיית העצירה) באופן הבא:

$B(M,x)$ =: יריץ את $A((M,x),$"עוצרת"$)$.

כיוון שA בודק נכונות של פתרונות על R, הוא ידע לבדוק ולהכריע האם M עוצרת על x (שהרי זה חלק מR). במילים אחרות, הכרחנו את אלגוריתם B באמצעות אלגוריתם A לוודא האם אכן M עוצרת על x. כלומר, B הוא אלגוריתם שפותר את $HALT$. בסתירה לכך ש$HALT$ לא כריעה.

■

הערה 24. **האם** $PC \subseteq PF$**?** זוהי שאלה פתוחה, בפרט זו ה־שאלה (שמיד נדון בה). כיום לא ידוע אם כל אלגוריתם שניתן לוידוא נכונות בזמן פולינומי גם ניתן לפתרון בזמן פולינומי.

1.6 מחלקות של בעיות הכרעה

הגדרה 25. נגדיר את המחלקה P:

$P = \{S |$ קיים אלגוריתם פולינומי שמכריע את $S\}$

למשל, $S_{st-conn} \in P$ ע"י הרצת דייקסטרה שאם הוא החזיר מסלול הוא בפרט יודע להחזיר "כן" או "לא". כמו כן, לא ידוע האם $S_{3-color} \in P$.

נבחין כי הגדרת המקבילה לPC היא מעט קשה בבעיות הכרעה, שהרי בבעיית הכרעה קיבלנו פתרון אפשרי ובדקנו אם הוא חוקי. כאן, בבעיות הכרעה אנחנו רק בודקים האם "כן" או לא, ולכן נעבוד קצת יותר קשה:

הגדרה 26. נאמר כי לבעיית הכרעה S יש **מערכת הוכחה יעילה** אם קיים אלגוריתם פולינומי V וקיים פולינום p כך שלכל x:

1. **שלמות:** אם $x \in S$ אזי "ישנה הוכחה לכך שהתשובה היא כן", כלומר: קיים y באורך $|y| \le p(x)$ כך ש$V(x,y) = 1$.
2. **נאותות:** אם $x \notin S$ אזי לכל y מתקיים כי $V(x,y) = 0$.

הערה. לא מובטח כי נמצא את ההוכחה בצורה יעילה, רק שקיימת כזו שמוודה פתרון בזמן יעיל.

V יקרא **מוודא מסוג** NP**.** הy נקרא עדות עבור x". y הוא עד לכך ש$x \in S$. אם נרחיב קצת יותר, אזי V מקבל קלט x והוכחה y באורך פולינומי לכך ש$x \in S$ (יתכן שההוכחה שגויה). אם $x \in S$ אזי תהיה איזושהי הוכחה y קצרה כך שנביא אותה לV הוא יאשר שהיא נכונה, ואם $x \notin S$ אזי כל הוכחה שנביא לV הוא ידע שהיא לא נכונה.

הגדרה 27. נגדיר את המחלקה NP: כל הבעיות שניתנות לוידוא בזמן פולינומי.

$NP = \{S |$ קיים לS מוודא מסוג $NP\}$

משפט 28. $P \subseteq NP$

הוכחה: תהי $S \in P$, ולכן קיים אלגו' A פולינומי שמכריע את S. נגדיר מוודא עבור S כך: $\forall y : V(x,y) = A(x)$. V אכן פולינומי, כיוון שA פולינומי. וכן הוא מקיים את 2 התכונות של מוודא NP:

א. $x \in S$: לפי הגדרה, צריך שיהיה קיים עד פולינומי y כך שהמוודא V יחזיר 1. כיוון ש$x \in S$ והרי מדובר בבעיית הכרעה, $A(x) = 1$. מכאן, לפי ההגדרה, $V(x,y) = A(x) = 1$, לכן לכל y שנבחר (לא משנה מי יהיה העד), V יחזיר 1.

ב. $x \notin S$: לפי הגדרה, לכל עד y יהיה צריך להתקיים $V(x,y) = 0$. אם כן, כיוון ש$x \notin S$ הרי ש$A(x) = 0$, ולפי הגדרתנו לכל y יתקיים $V(x,y) = A(x) = 0$, כנדרש.

■

הערה 29. נשים לב כי לפי הגדרת בעיית הכרעה, אם קיים אלגוריתם פולינומי A שיודע להכריע את S, כלומר $S \in P$, הרי שאותו אלגוריתם יכול לשמש גם כמוודא את הפתרון בזמן פולינומי. כלומר, אם יש לי אלגוריתם שפותר את הבעיה (מכריע), הוא יודע גם לוודא ולכן אכן $P \subseteq NP$. זה לא נכון ב$PF \not\subseteq PC$ כיוון שכפי שאמרנו בעוד שצריך לוודא שקיים פתרון אחד על מנת להיות בPF, על מנת להיות בPC צריך לדעת להתמודד עם כל קלט אפשרי. כיוון שכאן ישנם 2 מקרים בדיוק ־ הכרעה של כן, והכרעה של לא, אזי כן מתקיים $P \subseteq NP$. כמו כן, נשים לב כי אלגוריתם בNP מקבל גם את ההוכחה לכך ש$x \in S$ ורק צריך להכריע אם היא נכונה בזמן פולינומי (שקול לכך שאלגוריתם בPC קיבל פתרון ודרש להכריע נכונותו).

דוגמה: $S_{3-color} \in NP$. במקרה הזה הרי ש$x = G$, y הוא צביעה של הגרף ב3 צבעים, כלומר y הוא פונקציית צביעה, והמוודא V הוא אלגוריתם שמקבל את x, y ומחזיר כן או לא. במקרה הזה: V יעבור על כל הקשתות $(a,b) \in E$ ויוודא כי $\phi(a) \ne \phi(b)$. אם כן, הוא יחזיר שכן. אם קיימת לפחות קשת אחת שזה לא נכון לגביה הוא יחזיר שלא. מדובר באלגוריתם פולינומי הרי שעלותו $O(|E|)$, לינארי, ולכן המוודא פולינומי, ולכן אכן $S_{3-color} \in NP$.

הערה 30. **האם** $NP \subseteq P$**?** שאלת מליון הדולר ־ שאלה פתוחה: כל התאוריה של מדעי המחשב מתפתחת סביב השאלה הזו. ההשערה הרווחת היא שלא.

1.7 תרגול 1

בקורס נגדיר זמן יעיל כזמן ריצה $O(n^i)$ כאשר i הוא קבוע. לצורך העניין - $O(n^{100000000000})$ הוא זמן יעיל. כפי שנבין בהמשך הקורס, זו לא בעיה פשוטה לדעת האם אנחנו יכולים למצוא פתרון בזמן פולונימאלי.

בעיה 31. נתבונן ביחס הבא:

$$R_{4-Clique} = \{(G,S)|\text{G Graph, S clique and |S|=4}\}$$

כלומר, G גרף לא מכוון וS קליקה בגודל 4 בגרף. הוכיחו כי $R_{4-Clique} \in PF$.
פתרון: עלינו להראות למעשה אלגוריתם פולינומי שפותר את הבעיה. בתור התחלה, נבחין כי R אכן חסום פולינומית כי $|S| \le |V| \le |G|$.
כעת, נתבונן באלגו' הבא:
$A(G)$:
1. לכל תת קבוצה S של 4 בקודקודים בG בצע:
אם לכל $u,v \in S$ יש קשת בניהם, החזר את S.
2. החזר $\perp$

האלגוריתם נאיבי למדי, אך יש להוכיח פורמלית את נכונותו. כלומר, נרצה להראות פורמלית כי A פותר את $R_{4-Clique}$. נבחין כי אם אין קליקה בגודל 4 בגרף, האלגוריתם אכן יצליח במשימתו כי על כל קבוצה באורך 4 שהוא יבדוק הוא לא ימצא קליקה, ואכן יחזיר $\perp$. לכן, נניח בשלילה כי קיימת קליקה בגרף נסמנה $S = \{x_0, x_1, x_2, x_3\}$ שהאלגוריתם לא מוצא. אזי, בעת שיעבור על הקבוצה S הנ"ל בשלב 1 הוא ימצא כי $\forall i \neq j \in [3] : (x_i, x_j) \in E$ ולכן יחזיר את S, בסתירה להנחה.
כעת, זמן הריצה: נראה כי האלגוריתם מבצע $O(1)$ עבודה לכל קבוצה (בדיקת הקשתות שלה), ומס' תתי הקבוצות בגודל 4 הינו:

$$\binom{|V|}{4} = \frac{|V|!}{(|V|-4)! \cdot 4!} \le |V|^4 = O(|G|^4)$$

מסקנה: $R_{4-Clique} \in PF$.

■

בעיה 32. נתבונן ביחס הבא:

$$R_{Clique} = \{(G,k),S)|\text{G Graph, S Clique and |S|=k}\}$$

הראו כי $R_{Clique} \in PC$
פתרון: באופן דומה, נראה כי מדובר ביחס חסום פולינומית. שהרי, $|S| = k \le |V| \le |G|$. כעת, נרצה להציג אלגוריתם שמקבל G,k,S ובודק אם אכן S קליקה בגודל k:
1. ראשית, נבדוק כי $|S| = k$, אחרת נחזיר שלא.
2. לכל $u,v \in S$ נבדוק האם $(u,v) \in E$. אם נמצא זוג שלא, נחזיר לא. אחרת לבסוף: נחזיר כן.

האלגוריתם רץ בזמן ריצה $O(|S|^2) \le O(|G|^2)$. מכאן: $R_{Clique} \in PC$.

■

הערה 33. בבירור, האלגוריתם שהצגנו קודם לא יעבוד עבור R_{Clique} שכן זמן הריצה שלו יהיה $O(\binom{|V|}{k}) \le O(|V|^k)$. זהו לא אלגוריתם פולינומי - כי הוא תלוי בקלט. אך, זה שהאלגו' הקודם לא עבד לא אומר שהבעיה הזו לא ניתנת לפתרון. בפרט, אנחנו לא יודעים היום אלגוריתם פולינומי עבור הבעיה. כלומר - האם $R_{Clique} \in PF$ זו שאלה פתוחה.

בעיה 34. נגדיר "קבוצה שולטת" כקבוצת קודקודים S כך שלכל קודקוד $u \in V$ מתקיים או ש$u \in S$ או שלu יש שכן $v \in S$. ונגדיר את היחס:

$$\text{Dominating-Set} = \{(G,k)|\text{G has a dominating set of size k}\}$$

הראו כי $Dominating-Set \in NP$.
פתרון:
אינטואטיבית, הבעיה בNP כיוון שבהינתן (G,k) נוכל לקבל כעדות את הקבוצה השולטת והמוודא V יבדוק שאכן היא קבוצה שולטת לפי ההגדרה.
נגדיר $V((G,k),y)$: החזר 1 $\iff$ y קבוצה שולטת בG בגודל k.

נבחין כי V פולינומית כיוון שהבדיקה תהיה פולינומית - למעשה לכל קודקוד בגרף נבדוק האם $v \in S$, אחרת נעבור על כל שכניו ונבדוק אם יש לו שכן בS. זמן הריצה הוא

$$O(\sum_{v \in V} deg(v)) = O(|E|) \leq O(|V|^2) \leq O(|G|^2)$$

נבחין כי אם $(G,k) \in DS$ אזי קיים y כזה, המקיים $|y| \leq |G|$ עבורו $V((G,k),y) = 1$.
אחרת, אם $(G,k) \notin DS$, לכל y יתקיים $V((G,k),y) = 0$ שהרי לא קיימת קבוצה שולטת.

■

2 הרצאה 2

2.1 הקשר בין בעיות חיפוש לבעיות הכרעה

משפט 35. $PC \subseteq PF \iff P = NP$

הוכחה:

$\Longrightarrow$ נניח $PC \subseteq PF$. נוכיח $P = NP$. אנו יודעים כי $P \subseteq NP$. לכן נרצה להוכיח $NP \subseteq P$.
תהי $S \in NP$ בעיה כלשהי, נרצה להראות $S \in P$. הרי לפי הגדרה, קיים פולינום P ואלגוריתם פולינומי V כך ש:

$$x \in S \iff \exists y : |y| \leq p(x) : V(x,y) = 1$$

מה הפער שלנו בין NP לP? אנחנו יודעים בהינתן עדות, להכריע שייכות. אך איננו יודעים באופן כללי למצוא בצורה יעילה את העדות בהכרח. לכן, נעזר בהנחה שלנו $PC \subseteq PF$.

נגדיר את בעיית החיפוש ההבאה:

$$R_S = \{(x,y) | s.t\ |y| \leq p(x) \wedge V(x,y) = 1\}$$

נטען כי $R_S \in PC$. ראשית, לפי ההגדרה R_S חסום פולינומית (y חסום באורך של $p(x)$). שנית, האלגוריתם V הוא אלגוריתם שבודק שייכות לR_S (הרי הוא יודע להכריע האם y הוא עדות של x, כלומר האם $(x,y) \in R_S$).
לפי ההנחה שלנו $R_S \in PC \subseteq PF$. כלומר, $R_S \in PF$. לכן, קיים אלגוריתם פולינומי A שפותר את R_S. כלומר, אם $R_S(x) \neq \emptyset$ אזי האלגו' מחזיר y כך ש$y \in R_S(x)$. אחרת, מחזיר $\perp$.
כעת, נרצה להעזר באלגוריתם A על מנת להוכיח $S \in P$. נבנה אלגוריתם B כך שבהינתן קלט x מריץ את $A(x)$ ומחזיר $1 \iff A$ החזיר תשובה שאינה $\perp$.
נבחין כי B פולינומי כי זמן הריצה שלו כזמן הריצה של האלגו' A שהוא פולינומי. נבחין כי B מכריע את S שכן:
$x \in S \iff$ לפי הגדרה, קיים y המקיים $|y| \leq p(x)$ כך ש$V(x,y) = 1$. לכן $(x,y) \in R_S$. $\iff R_S(x) \neq \emptyset \iff A(x) \neq \perp \iff B(x) = 1$
כלומר, $x \in S \iff B(x) = 1$
לבסוף, האלגוריתם B אכן מכריע את S לפי הגדרה, והוא פולינומי, כנדרש. מסקנה: $S \in P$.

$\Longleftarrow$ נניח $P = NP$ ונוכיח $PC \subseteq PF$.
תהי $R \in PC$. נרצה להציג אלגוריתם פולינומי שפותר את R, כלומר, $R \in PF$.
אם כן, $R \in PC$ ולכן קיים פולינום P כך שלכל $(x,y) \in R$ מתקיים: $|y| \leq P(x)$. בנוסף, קיים אלגוריתם פולינומי A שבודק שייכות ליחס R.

בבירור, נצטרך להגדיר בעיית הכרעה ולהראות כי היא נמצאת בNP, ששווה לP. ולהתקדם משם.אינטואטיבית, נרצה למצוא את הy כמעבר אחד אחר אחד אבל! מדובר בזמן ריצה אקספוננציאלי. לכן נרצה לגלה את הy באמצעות מעבר ביט ביט, כלומר נגלה לאט לאט את y כאשר כל פעם נסתכל על $prefix(y)$ שונה שלו. עוד קצת אינטואיציה: בידינו אלגוריתם פולינומי שבודק אם y פתרון תקין עבור x. כמו כן, נבחין כי על מנת להראות שייכות לNP אנחנו רק צריכים להראות שקיים מוודא וקיימת עדות. לכן מגדירים יחס כפי שיתואר למטה, לפיו נכנסים רק $prefix(y)$. אנחנו יודעים מי יהיה העדות - y'', ואנחנו יודעים גם מי המוודא: A עצמו! שמסוגל פשוט לשרשר ולבדוק אם אכן השרשור נמצא בR.

נגדיר את בעיית ההכרעה הבאה:

$$S'_R = \{(x,y') | \exists y'' : (x, y' \circ y'') \in R\}$$

לדוגמה: $R=\{(10,011)\}$ אזי $S'_R=\{(10,\varepsilon),(10,0),(10,01),(10,011)\}$. כלומר ־ כל התחיליות של y.
נוכיח כי $S'_R\in NP$. מתקיים כי :

$$(x,y')\in S'_R \iff \exists y'':|y''|\le p(x+y'):A(x,y'y'')=1$$

שהרי, לפי הגדרה העד יכול להיות y'' והמוודא הוא האלגוריתם A שבודק שייכות לR, והרי יכול לשרשר $y'\circ y''$ ולבדוק אם ערך זה שייך לR.
כעת, לפי ההנחה, $P=NP$ ולכן גם $S'_R\in P$. כלומר, קיים אלגוריתם פולינומי D שמכריע את $S_{R'}$ (מחזיר כן אם קיים y'' כנ"ל ששרשורו יהיה בR).
נגדיר אלגוריתם B שיפתור את R:

$B(x)$:
א. ראשית ־ נבדוק אם בכלל קיים פתרון, לכן נריץ את $D(x,\varepsilon)$. אם הוא החזיר כן: אזי בבירור קיים y'' כך ש$(x,y'')\in R$. אחרת ־ החזיר לא: נחזיר $\perp$.
ב. אם הגענו לשלב הזה: בהכרח קיים פתרון, ונרצה לבנות אותו בצורה הדרגתית. נספן $y\leftarrow\varepsilon$
ג. כל עוד $D(x,y)=1$ (הy שאנו מסתכלים עליו עכשיו עודנו תחילית של פתרון):
1. אם $D(x,y1)=1$ (שרשור של הביט 1 לסיום y עדיין נותן פתרון) אזי נגדיר $y\leftarrow y1$.
2. אחרת, אם $D(x,y0)=1$ אזי $y\leftarrow y0$

נבחין כי האלגוריתם $B(x)$ הוא אלגוריתם פולינומי. בשלב 1 הוא מריץ אלגו' פולינומי D, בתוך הלולאה: הוא מריץ לכל היותר פעמיים אלגוריתם פולינומי. כמה פעמים מתבצעת הלולאה? לכל היותר אורך פולינומי של צעדים ־ האורך של y, ולכן זמן הריצה של הלולאה הוא מס' פולינומי של פעמים כפול ערך פולינומי: וסה"כ מדובר באלגוריתם פולינומי.
אם אין פתרון לx, בשלב 1 B יחזיר $\perp$. אחרת, אם קיים פתרון, הוא נבנה רק בצורה לפיה הוא נשאר תחילית של פתרון: לכן בהכרח יחזיר y כך ש$(x,y)\in R$.
סה"כ B אלגוריתם פולינומי שמכריע את R, ולכן $R\in PF$.

■

מסקנה 36. ראינו מהמשפט הקודם שקילות בין בעיות פתוחות לגבי בעיות הכרעה ובעיות חיפוש, לכן נרשה לעצמנו בקורס להתעסק יותר בבעיות הכרעה.

הערה 37. בכיוון השני של הוכחת משפט 35, השתמשנו ברדוקציה. צמצמנו את הבעיה שלנו לבעיה אחרת: אם אנחנו יודעים לפתור (כאן, בצורה פולינומית ויעילה) בעיה מסויימת אזי (בגלל שפולינום כפול פולינום הוא פולינומי) ניתן לפתור את הבעיה השנייה בזמן יעיל. מה שמוביל אותנו לדבר על:

2.2 רדוקציות

הגדרה 38. רדוקציה הינה דרך לפתור בעיה אחת, בעזרת פתרון עבור בעיה אחרת. אם ישנה רדוקציה מבעיה A לבעיה B ואנחנו יודעים לפתור את B, אזי אנחנו יודעים לפתור את A.

הערה 39. קודם ביצענו רדוקציה מR לS'_R ־ בידינו באמצעות ההנחה היה פתרון לS'_R ובאמצעותו בנינו אלגוריתם B ששימש מעבר לרדוקציה אל R.

2.3 רדוקציית קוק

הגדרה 40. רדוקציית קוק מבעיה A לבעיה B הינה אלגוריתם פולינומי שפותר את A בעזרת גישת אוקרל לB. כלומר, האלגוריתם שפותר את A יכול לשאול את האוקרל שאלות עבור B , כאשר זמן הריצה עבור כל **גישת אורקל** נחשבת כצעד אחד בודד. נסמן זאת ב $A\le_T^P B$ (P מייצג פולינומית וT מייצג מכונת טיורינג. ה$\le$ מתאר את הרדוקציה עצמה ־ A לא קשה יותר מB).

האורקל הוא מעין "קופסה שחורה" קסומה אשר ניתן לשאול אותה שאלות לגבי בעיה מסויימת, היא מחזירה תשובה נכונה באופן מיידי בצעד חישובי אחד מבלי שנדע כיצד הגיעה לפתרון.

הגדרה 41. רדוקציה עצמית: עבור בעיית חיפוש R נסמן בS_R את בעיית ההכרעה הבאה:

$$S_R=\{(x|R(x)\neq\emptyset\}$$

אינטואטיבית, לכל בעיית חיפוש ישנה בעיית הכרעה מתאימה לה ־ בעיית קדם: בהינתן x, האם קיים לה איזה שהוא y?
רדוקציה עצמית הינה רדוקציית קוק מR לS_R. כלומר: $R\le_T^P S_R$. (כלומר, מאפשר לפתור בעיית חיפוש באמצעות גישות לאורקל שפותר את בעיית ההכרעה).

מסקנה 42. אם יש לנו בעיית חיפוש קשה, אבל יש רדוקציה עצמית לבעיה זו. אזי נוכל לפתור את בעיית ההכרעה המתאימה, ובגלל שקיימת רדוקציה עצמית אזי פתרנו גם את בעיית ההכרעה.

דוגמה 43. נתבונן בדוגמה ראשונה לרדוקציה עצמית. נגדיר את בעיית החיפוש הבאה:

$$R_{SAT}=\{(\Phi,\tau)|\Phi \text{ is a Boolean formula in CNF , and } \tau \text{ is a satisfying assignment } \phi \text{ for the formula}\}$$

נראה רדוקציה עצמית עבור R_{SAT}.

אינטואיציה: נתחיל מלהציב למשל $x_1=T$, ונקבל נוסחה בוליאנית מצורת CNF מצומצמת יותר. כעת נשאל את האורקל: האם הנוסחה שבידי היא ספיקה? אם כן: מצאנו $x_1=T$ הוא "תחילית" של פתרון ונתקדם. אחרת: נציב $x_1=F$ ונתקדם בכל מקרה לנוסחה.

נציג את האלגוריתם הבא:
$A^{SAT}(\Phi)$:
1. שאל את האורקל אם $\Phi\in SAT$, אם לא נחזיר $\perp$.
2. נסמן n כמס' המשתנים בנוסחה Φ. כלומר, $\{x_i\}_{i=1}^{n}$ הם המשתנים ונגדיר $\tau\to\varepsilon$
3. לכל i מ1 עד n:

א. הגדר נוסחה חדשה: Φ' ע"ח הצבת τT (למשתנה הבא, הצב ערך T) ושאל את האורקל אם Φ' ספיקה. אם $\Phi'\in SAT$ נגדיר $\tau=\tau T$ ואחרת נגדיר $\tau\to\tau F$.

בבירור, האלגוריתם בונה את τ צעד אחר צעד ואם אכן קיימת השמה מספקת אזי הוא מחזיר אחת שכזו τ, ואם אין כזו מנכונות האורקל הוא מחזיר בשלב ה1 $\perp$.

זמן הריצה: כמובן מריצים את האורקל n פעמים, ולכן הפער פולינומי.

■

הערה 44. הערה מגניבה. אנחנו יודעים כי $R_{SAT}\in NP$, קל לבדוק בהינתן נוסחה האם היא אכן ספיקה. אך לא ידוע אם היא בP, כלומר האם קיים אלגו' פולינומי שיודע לומר: יש/אין פתרון. אם $P=NP$, אזי קיים אלגו' פולינומי שיודע לומר - יש/אין פתרון: ומהשקילות שראינו כאן עם רדוקציית קוק עצמית, נריץ אותו n פעמים, סה"כ פולינומי כפול n נותר פולינומי ולכן $R_{SAT}\in P$.

הערה 45. כסימון, $A^{SAT}(\Phi)$ מסמן שהאלגוריתם A מקבל גישות אורקל לבעייה SAT. כקונבציה, כך נסמן. כלומר: האלגוריתם A מסוגל לגשת בגישת אורקל אל בעיית ההכרעה.

הערה 46. **רדוקציית קוק** רלוונטית הן לבעיות הכרעה והן לבעיות חיפוש. כאשר נעבור לדון בסוג של רדוקציה שרלוונטי רק לבעיות הכרעה:

2.4 רדוקציות קארפ

הגדרה 47. רדוקציית קארפ מבעיית הכרעה S_1 לבעיית הכרעה S_2 הינה פונקציה **שניתנת לחישוב בזמן פולינומי** ומקיימת: $x\in S_1\iff f(x)\in S_2$.
במקרה זה, נסמן $S_1\leq_m^P S_2$ (P מייצג פולינומי וm מייצג many to one: היא לא חח"ע, יתכן שיהיו שני ערכים $x_2\neq x_1$ כך ש$f(x_1)=f(x_2)$, כמו כן היא לא צריכה להיות על).

הערה 48. הרדוקציות בהן התעסקנו בקורס "**מודלים חישוביים**" הן רדוקציות קארפ.

הערה 49. הפונקציה לא הפיכה בהכרח כפי שאמרנו - יתכן והפונקציה (רדוקציה) הינה $S_1\leq_m^P S_2$, כך שבהינתן קלט בS_1 ניתן להמירו לקלט בS_2 אך הכיוון ההפוך אינו נכון!

משפט 50. אם קיימת רדוקציית קארפ $S_1\leq_m^P S_2$ וכן $S_2\in P$ אזי $S_1\in P$.
הוכחה: נגדיר אלגו' פולינומי עבור S_1 -
1. הפעל את $f(x)$ [נקבל כי $f(x)\in S_2$]
2. קיים אלגו' פולינומי A_2 שמכריע את S_2. נחזיר את $A_{S_2}[f(x)]$.

■

משפט 51. אם קיימת רדוקציית קארפ $S_1\leq_m^P S_2$ וקיימת רדוקציית קארפ $S_2\leq_m^P S_3$, אזי קיימת רדוקציית קארפ $S_1\leq_m^P S_3$. (טרנזיטיביות)
הוכחה: יהיו f_1,f_2 פונקציות הרדוקציות בהתאמה. נראה כי $f_1\circ f_2$ היא פונקציית רדוקצייה $S_1\leq_m^P S_3$ המקיימת $x\in S_1\iff f_3(x)\in S_3$.
■

דוגמה 52. (**דוגמה חשובה**). נתבונן בדוגמה ראשונה לרדוקציית קארפ. נזכר בבעיית SAT ונגדיר את הבעיה הבאה:

$$IS=\{(G,k)|\text{G Graph, G has independent set of size at least k}\}$$

נרצה להראות רדוקציית קארפ $SAT\leq_m^P IS$.

מה אנחנו רוצים להראות? בהינתן נוסחה בוליאנית ϕ בצורת CNF כקלט לבעיית SAT, נרצה לבנות גרף G ולהגדיר מס' k כך שהבנייה תהיה בזמן פולינומי, כך שיתקיים: $(G,k) \in IS \iff \phi \in SAT$. כלומר בשלב ראשון נראה את הבנייה, נוכיח שהיא פולינומית. בשלב שני נוכיח את השקילות הנ"ל.

נסמן: $\phi = \bigwedge_{i=1}^{m}(\bigvee_{j=1}^{n_i} \ell_{i,j})$ כנוסחה הבוליאנית בצורת CNF, כך שעבור הפסוקית הi יש בו n_i ליטרלים מופרדים ב$\vee$ כך ש$n_{i,j} \in \{x_r, \overline{x_r}\}$ עבור $r \in [n]$ כך שאנו יודעים שהנוסחה מורכבת מהמשתנים $\{x_i\}_{i=1}^{n}$.

נגדיר $G = (V,E)$ כך:

$$V = \{u_{i,j} | \forall 1 \leq i \leq m, 1 \leq j \leq n_i : \ell_{i,j} = u_{i,j}\}$$

כלומר, כל ליטרל מיוצג ע"י קודקוד. נגדיר את הקשתות E כך:

1. סוג ראשון של קשתות: לכל פסוקית $1 \leq i \leq m$ נרצה להוסיף קשתות בין כל הקודקודים של אותה הפסוקית, כך שהם יהוו קליקה. כלומר:

$$E_1 = \{(u_{i,j}, u_{i,k}) | \forall 1 \leq i \leq m, \forall j \neq k : j,k \in [n_i]\}$$

2. סוג שני של קשתות: עבור על משתנה $x_r \in \phi$ כך ש$r \in [n]$ נוסיף קשתות בין כל קודקוד שמתאים לx_r לבין כל קודקוד שמתאים ל$\overline{x_r}$. כך:

$$E_2 = \{(u_{i,j}, u_{a,b}) | \ell_{i,j} = \overline{\ell_{a,b}}\}$$

וסה"כ $E = E_1 \cup E_2$.

כמו כן, נגדיר $k = m$. כלומר k הוא מס' הפסוקיות. עד לשלב זה הייתה הבנייה של הגרף G, ונבחין שהיא אכן פולינומית (אין צורך ממש לחשב בכמה זמן), אך בבירור זה פולינומי. כעת נרצה להראות כי מתקיים $(G,k) \in IS \iff \phi \in SAT$.

אינטואטיבית, הרדוקציה הנ"ל בונה משחק שבו כדי לנצח חייבים לספק את הנוסחה. בנוסחת CNF על מנת שהיא תהיה אמת כל פסוקית חייבת לקבל ערך של אמת. כדי שפסוקית תהיה אמת, מספיק שליטרל אחד יהיה אמת בתוכה. בגרף: כל פסוקית הפכה לקליקה. נבחין כי בIS, אפשר לבחור לכל היותר קודקוד אחד מתוך כל קליקה כזו. כיוון שהגדרנו $k = m$, אנחנו למעשה "מכריחים" את האלגוריתם לבחור בדיוק נציג אחד מכל פסוקית, הבחירה הזו שקולה להחלטה: "זה הליטרל שיספק פסוקית זו". בנוסף, חיברנו כל משתנה לשאר מופעיו בצורת השלילה שמופיעים בפסוקיות אחרות. מדוע? על מנת למנוע מצב בו הגדרנו $x_1 = T, \overline{x_1} = T$. כעת, נפרמל אינטואיציה זו:

$\Longrightarrow$ נניח כי $\phi \in SAT$. אזי, קיימת השמה τ שמספקת את ϕ. נרצה להראות כי $(G,k) \in IS$. כלומר, בגרף זה ישנה קבוצה ב"ת מגודל לפחות k. אנו יודעים כי τ מספקת את ϕ ולכן בכל פסוקית יש ליטרל אחד לפחות $\ell_{i,j}$ שמספק אותה. נטען כי הקבוצה הבאה: $S = \{u_{i,j}\}_{i=1}^{m}$ המקיימת $|S| = m = k$ היא קבוצה ב"ת בG. נבחין כי בין כל זוג קודקודים שכאלו אין קשת כיוון שהם מפסוקיות אחרות, פרט למקרה היחיד שיש קשת בין פסוקיות שונות ובו קיים משתנה שגם הוא מספק את הפסוקית, וגם השלילה שלו מספק את הפסוקית. שהרי במקרה זה אם נב"ש שאכן קיים כזה אזי $\ell_{i,j} = T$ וגם $\ell_{a,b} = T$ אבל $\ell_{a,b} = \overline{x_r} \wedge \ell_{i,j} = x_r$ ונקבל סה"כ $\ell_{a,b} = F$ בסתירה. ולכן: אכן הקבוצה הנ"ל S היא קבוצה ב"ת בG בגודל k, כנדרש.

$\Longleftarrow$ נניח כי $(G,k) \in IS$. אזי, קיימת בG קבוצה ב"ת S בגודל k. נרצה להגדיר השמה τ לנוסחה ϕ ולהוכיח כי היא אכן מספקת את ϕ.

נגדיר השמה τ לϕ בהתאם לקבוצה S, כך: אם $u_{i,j} \in S$ וכן $u_{i,j} = x_r$ אזי נגדיר $\tau(x_r) = T$. אם $u_{i,j} = \overline{x_r}$ אזי נגדיר $\tau(x_r) = F$.

יתכן כי לפי ההשמה הנ"ל לא כל המשתנים קיבלו ערך, שאר המשתנים שלא קיבלו ערך יקבלו ערך שרירותי, למשל T. נטען כי בהכרח ההשמה הנ"ל מוגדרת היטב, שכן לא יתכן כי x_r קיבל גם T וגם F. בשלילה שכן, אזי קיים x_r עבורו הייתה השמה T, ולכן $u_{i,j} \in S$ וגם קיימת השמה F כלומר $u_{a,b} \in S$. אך קודקודים אלו מייצגים אותם קודקודים - ולכן ישנה קשת $(u_{i,j}, u_{a,b}) \in E$, בסתירה לכך שS היא קבוצה ב"ת. מסקנה: ההשמה הנ"ל מוגדרת היטב.

כעת, נרצה להוכיח שאכן לפי ההשמה τ כעת ϕ היא ספיקה. כלומר, $\tau(\phi) = T$. אנו יודעים כי $|S| = k$, וכן לא יכולים להיות זוג קודקודים מאותה פסוקית (שכן אחרת ישנה קשת בניהם, כי כל פסוקית היא קליקה וS ב"ת), לכן בהכרח S מכילה קודקוד אחד מכל פסוקית: אותו ליטרל שמתאים לפסוקית הנ"ל ונמצא בS מספק את הפסוקית המתאימה לו, וסה"כ כל הפסוקיות מסופקות: לכן ϕ מסופקת ע"י τ, כנדרש.

■

2.5 תרגול 2

תזכורת קלה. לכל בעיית חיפוש R ניתן להגדיר בעיית הכרעה מתאימה: $S_R = \{x | \exists y : (x,y) \in R\}$.

כלומר, לכל בעיית חיפוש ניתן לבנות ממנה בעיית הכרעה שמכילה את כל הקלטים שיש להם לפחות פלט אחד.

משפט 53. לכל בעיית חיפוש כך ש$R \in PF$, מתקיים כי $S_R \in P$.

הוכחה: תהי בעיית חיפוש R כך ש$R \in PF$, אזי קיים אלגו' פולינומי A_R שפותר אותה. נרצה להציג אלגו' פולינומי A_{S_R} שפותר את S_R.

A_{S_R}: יריץ את $A_R(x)$ ויחזיר 0 אם $A(x)=\perp$ ואחרת, יחזיר 1.
בבירור, A_{S_R} פותר את S_R ורץ בזמן פולינומי כי מריץ פעם אחת אלגו' פולינומי, לכן $S_R \in P$.

■

תרגיל 54. תהי S בעיית הכרעה. הראו רדוקציית קוק מS ל$\overline{S}$.
פתרון: תהי S בעיית הכרעה. נציע את האלגו' הבא -
1. שאל את האורקל של $\overline{S}$ האם $x \in \overline{S}$: החזר תשובה הפוכה ממה שהאורקל אמר.
נכונות: יהי $x \in S$, אזי $x \notin \overline{S}$ ולכן האורקל יחזיר לא: ולכן נחזיר כן. יהי $x \notin S$ אזי $x \in \overline{S}$ ולכן האורקל יחזיר כן: ולכן נחזיר לא.
זמן ריצה: גישת האורקל פולינומית (או $O(1)$, תלוי גישה), וכן הפיכת התשובה גם כן פולינומית. סה"כ אלגו' פולינומי.

■

תרגיל 55. יהיו זוג בעיות הכרעה, $S_1, S_2 \in P$. הוכיחו: קיימת רדוקציית קוק מS_1 לS_2.
פתרון: יהיו S_1, S_2 זוג בעיות הכרעה בP. אזי קיים אלגוריתם פולינומי A_{S_1} שמכריע את S_1.
נגדיר למעשה רדוקציית קוק מS_1 לS_2 כך שהאלגוריתם יריץ את $A_{S_1}(x)$ ויחזיר את תשובתו, מבלי לפנות לאורקל של S_2. בבירור: פולינומי.

■

ìéâøú 56. יהיו זוג בעיות הכרעה, $S_1, S_2 \in P$. הפריכו: קיימת רדוקציית קארפ מS_1 לS_2.
פתרון: הטענה לא נכונה. נקח $S_1 = \{0,1\}^*$ וכן $S_2 = \emptyset$. בבירור $S_1 \in P$ (האלגו' תמיד מחזיר 1) ובבירור $S_2 \in P$ (האלגו' תמיד מחזיר 0). בשלילה שקיימת רדוקציית קארפ מS_1 לS_2. אזי, קיימת פונקציה פולינומית f עבורה $\forall x: f(x) \in S_2 \iff x \in S_1$. מכאן, לכל x מתקיים $x \in S_1$ ולכן $f(x) \in S_2 = \emptyset$ בסתירה להגדרת קבוצה ריקה.

■

תרגיל 57. הראו רדוקציית קארפ $VC \leq_m^P DS$ כאשר VC מוגדרת בצורה הבאה:

$$\text{Vertex-Cover} = VC = \{(G,k)|\text{G has set } S \subseteq V \text{ of size k (|S|=k) s.t} : \forall e=(u,v) \in E : u \in S \vee v \in S\}$$

פתרון:
מה אנחנו רוצים להראות? אנו רוצים בהינתן קלט $(G,k) \in VC$, לבנות (או להציג המרה) לגרף G ומס' כלשהו k' כך שיתקיים $(G,k) \in VC \iff (G',k') \in DS$.
בשלב ראשון נרצה להציג את הבנייה הפולינומית ("הפונקציה" f).
נגדיר $G' = (V', E')$ בצורה הבאה: G' יכיל את כל קודקודי הגרף G פרט לקודקודים המבודדים בגרף. כמו כן, לכל קשת $(u,v) \in E$ נוסיף קודקוד $u_{(u,v)}$. כלומר:

$$V' = V \backslash \{u \in V \text{ s.t } \forall v \in V : (u,v) \notin E\} \cup \{u_{(x,y)} | \forall (x,y) \in E\}$$

ונגדיר את הקשתות בצורה הבאה:

$$E' = E \cup \{(u, u_{(x,y)}), (u_{(x,y)}, v) | \forall (x,y) \in E\}$$

כמו כן נגדיר $k' = k$. בבירור, נשים לב כי הבנייה פולינומית כפי שרצינו (לכל היותר $O(|E|^2 \leq |G|^2)$). כעת נרצה להוכיח את נכונות הבנייה -

$\Longrightarrow$ נניח כי $(G,k) \in VC$. אזי קיימת בו קבוצת קודקודים S שמכסה את כל הקשתות. בה"כ נניח כי S לא מכילה קודקודים מבודדים (הרי ניתן למחוק אותם לאחר מכן והיא עדיין תכסה את כל הקשתות). נרצה לטעון כי S היא קבוצה שולטת.
יהי $v \in G'$ קודקוד מקורי. לפי הבנייה, איננו מבודד לכן קיים לו שכן u. כלומר $(v,u) \in E$. S היא כיסוי קשתות בG ולכן נקבל כי $u \in S$ או $v \in S$. כעת נסתכל על קודקוד חדש מסוג $u_{(u,v)}$. לפי הבנייה, ישנה קשת $(u,v) \in G$ ולכן לפי מה שראינו על קודקוד מסוג מקורי, או ש$u \in S$ או ש$v \in S$. לכן ל$u_{(u,v)}$ ישנו לפחות שכן אחד בS. (שהרי מחובר לu, v).
אם נסכם - עבור קודקודים בG: קודקוד רגיל מקיים - או שהוא בS, או שבהכרח יש לו שכן בS (כי קיימת קשת והסרנו קודקודים מבודדים). קודקוד חדש מהצורה $u_{(u,v)}$ מקיים - אחד משכניו בהכרח בS. סה"כ כל קודקוד בG' נשלט ע"י הקבוצה S שהיא בגודל $k = k'$, כנדרש.

$\Longleftarrow$ נניח כי $(G',k') \in DS$. אזי קיימת בG' קבוצה שולטת S' בגודל $k' = k$. נניח בה"כ כי קבוצה זו לא מכילה קודקודים מהצורה $u_{(u,v)}$ שכן אחרת ניתן להחליף אותו פשוט באחד מן השכנים שלו - ועדיין נשאר עם קבוצה שולטת (קודקוד שכזה יכול לשלוט רק על שכניו - הם שכנים בעצמם, לכן אם נחליפו לu או v עדיין נקבל קבוצה שולטת). כעת תהי $(u,v) \in G$ קשת כלשהי. לפי הבנייה, קיים בG' קודקוד $u_{(u,v)}$ שנשלט ע"י הקבוצה S אך לא שייך אליה. (קיים לפחות אחד, כי מניחים שהגרף לא ריק וכן זה הקודקוד שמייצג את הקשת הנ"ל ולא שייך אליה לפי בה"כ שלנו). מכאן ש$u \in S$ או ש$v \in S$ - ולכן הקשת (u,v) מכוסה ע"י S.

סה"כ הראינו רדוקציית קארפ מVC לDS, כנדרש.

אם נסכם במעט אינטואיציה: בVC אנחנו צריכים לספק קשתות, אבל בDS אנחנו צריכים לספק קודקודים (לדאוג שכל אחד יישלט ע"י S). אם היינו מגדירים $G=G'$ אין דרך להתקדם שכן אם נבחר קודקוד כיסינו את כל שכניו ־ אך לא את הקשתות ביניהם (ולכן נהיה בבעיה מבחינת VC). אנחנו רוצים להכריח את DS להתנהג כמו VC, לכן הוספנו קשת־קודקוד חדשה, כעת DS חייב לכסות את הקודקוד הנ"ל $u_{(u,v)}$. אך קודקוד זה מחובר רק לu או לv ־ ולכן DS יכסה את הנציג אמ"מ יבחר בS $u \in S$ או $v \in S$ (כפי שאמרנו אין סיבה להכניס קודקודי קשת לקבוצה). נבחין כי הוצאנו קודקודים מבודדים ־ שכן הם חסרי תועלת: לא מכסים שום קשת, לכן לא ייבחרו לVC אופטימלי וכן בDS הוא נטל ־ הוא חייב להבחר: כי אין לו שכנים שיכסו אותו. לכן בשביל לשמור על הגודל k, הסרנו מראש את הקודקודים המבודדים. (ושוב: נבחין כי בחיים לא יהיה קודקוד מבודד בVC! לכן מראש טיפלנו בכך ע"י הסרתו).

■

3 הרצאה 3

3.1 NP־שלמות

משפט 58. נאמר כי בעיית הכרעה S היא NP־קשה אם לכל $S' \in NP$ יש רדוקציית קארפ לS. (כלומר, היא קשה לפחות כמו כל בעיה בNP ויש דרך להמיר בעיה מNP אל S). במקרה זה, נאמר כי $S \in NPH$ באשר $NPH = \text{NP Hard}$.

הערה 59. אם נדע לפתור בעיית NP־קשה נדע לפתור את כל הבעיות בNP.

משפט 60. נאמר כי בעיית הכרעה S היא NP־שלמה אם $S \in NP$ וגם S היא NP־קשה. במקרה זה, נאמר כי $S \in NPC$.

משפט 61. קיימת בעייה NP־שלמה.

הוכחה: נגדיר את הבעיה "האוניברסלית" הבאה:

$$S_u = \{< M, x, 1^t > |\text{M is Turing machine}, \exists y : |y| \leq t : M(x,y) = 1\text{within t steps} \wedge 1^t = 11....1\}$$

כלומר, מכונת טיורינג, x כקלט למכונה, קיים y כך שהמכונה על (x,y) מחזירה 1 לאחר לכל היותר t צעדים. אינטואטיבית, נבחין שהיא אכן מייצגת את כל NP (עדות y באורך לכל היותר t וכו'). נרצה להוכיח כי S_u היא NP־שלמה.

1. $S_u \in NP$: נגדיר את המוודא הפולינומי הבא: המוודא יקבל את העדות y ואת השלשה.
$V((M,x,1^t,y)$: אם $|y| > t$ החזר אפס. אחרת, הרץ $M(x,y)$ למשך t צעדים: אם M החזירה 1 החזר 1 ואחרת החזר אפס.
בבירור האלגוריתם עונה על הדרוש, ונבחין כי כיוון שמדובר בייצוג אונארי של t אחדות, נדע כי בהכרח t הוא פולינומי ולא אקספוננציאלי (אם t היה בייצוג בינארי, יתכן והיינו מקבלים מספר שמס' הצעדים שלו הוא $2^{|o|}$ עבור o כלשהו וזה כבר אקספוננציאלי). זמן הריצה הוא $O(t)$ וכיוון שt הוא פולינומי, בהכרח זמן ריצת האלגוריתם פולינומי. סה"כ אכן $S_u \in NP$.

2. $S_u \in NPC$: $S_u \in NP$. לכן, כל שנדרש הוא להראות שלכל $S' \in NP$ קיימת רדוקציית קארפ מS' לS_u.
תהי $S \in NP$. נרצה להראות כי קיימת רדוקציית קארפ מS לS_u. אם כן, כיוון ש$S \in NP$ קיים פולינום $p(n)$ וכן אלגוריתם פולינומי V כך ש:

$x \in S \Longleftarrow \exists y : |y| \leq p(|x|) : V(x,y) = 1$
$x \notin S \Longleftarrow \forall y : V(x,y) = 0$.

V פולינומי ולכן קיים פולינום $Q(x)$ שחוסם את זמן הריצה של V. כלומר ־ כשV מקבל קלט כלשהו (x,y) כאשר $|y| \leq p(|x|)$ אזי זמן הריצה של V הוא לכל היותר $Q(|x|+|y|) \leq Q(|x|+p(|x|))$. אם כן, נסמן $t(n) = Q(n+p(n))$. מכאן, זמן הריצה של V הוא לכל היותר $t(|x|)$.

כעת, נגדיר את הרדוקצייה f:

$$f(x) = < V, x, 1^{t(|x|)} >$$

כלומר, מכונת הטיורינג כעת היא מוודא (אלגו' פולינומי), x הוא אותו הקלט ומס' האחדות הוא $t(|x|)$.
נרצה להוכיח שהרדוקציה היא רדוקציית קארפ חוקית, כלומר: פולינומית ומקיימת נכונות.
זמן הריצה: V כזמן קבוע לשליחת האלגו', x כזמן קבוע וסה"כ מדובר בזמן קבוע של פולינום באורך של x. לכן סה"כ הרדוקציה פולינומית.
נרצה להוכיח $x \in S \Longleftrightarrow f(x) \in S_u$.
$\Longrightarrow$ $x \in S$, אזי $\exists y : |y| \leq p(|x|) : V(x,y) = 1$. אם כן, 1 $V(x,y) = 1$ לאחר לכל היותר $t(|x|)$ צעדים. מכאן, $f(x) = < V, x, 1^{t(|x|)} > \in S_u$.
$\Longleftarrow$ $x \notin S$, אזי $\forall y : V(x,y) = 0$. בפרט, לכל $y : |y| \leq t(|x|)$ יתקיים $V(x,y) = 0$ ובפרט V לא יעצור לאחר לכל היותר $t(x)$ צעדים. לכן, $f(x) = < V, x, 1^{t(|x|)} > \notin S_u$.
סה"כ, הרדוקצייה נכונה ולכן אכן ישנה רדוקציית קארפ וקיבלנו $S_u \in NPC$.

■

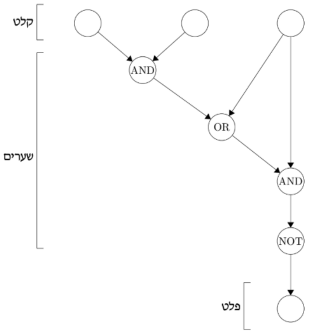
3.2 משפט קוק־לוין

משפט 62. (משפט קוק־לוין) $SAT \in NPC$, כלומר SAT היא NP- שלמה.
הוכחה: כהערה ראשונה, מדובר בהוכחה ארוכה אז שתו מים ונצא לדרך. תוכנית הפעולה שלנו תהיה מורכבת מכמה חלקים:
1. אנחנו נגדיר בעיה בשם $CSAT$, נוכיח כי $CSAT$ היא NP-שלמה.
2. נראה רדוקציית קארפ מ$CSAT$ לSAT על מנת להוכיח כי SAT היא NP-קשה. (**מסקנה:** אם יש לנו רדוקציית קארפ מבעיה A לבעיה B, וגם A היא NP-Hard אזי B היא $NP-Hard$ - כלומר מדובר בטרנזטיביות רדוקציית קארפ).
3. נזכר כי SAT היא ב NP.
4. שילוב 3 ו2 יובילו לכך ש$SAT \in NPC$.

חלק ראשון של ההוכחה $(CSAT)$**:**

הגדרה 63. מעגל בוליאני הינו גרף מכוון ללא מעגלים (DAG) שקודקודיו מתחלקים ל3 סוגים:
א. קודקודי קלט - קודקודים $v \in V$ המקיימים $deg_{in}(v) = 0$ (ללא קשתות נכנסות) וכן $deg_{out}(v) \geq 1$ (לפחות קשת יוצאת אחת).
ב. קוקודי פלט - קודקודים $v \in V$ המקיימים $deg_{out}(v) = 0$ (ללא קשתות יוצאות) וכן $deg_{in}(v) = 1$ (יש קשת נכנסת אחת).
ג. שערים: קודקודים $v \in V$ המקיימים $1 \geq deg_{in}(v) \geq 2$ (בין 1 ל2 קשתות נכנסות) וכן $deg_{out}(v) \geq 1$.

לדוגמה (לקוח מתוך הסיכום של עידו קצב, 2023).



הגדרה 64. נגדיר את הבעיה $CSAT$ בצורה הבאה:
$CSAT = \{$ C | C מעגל בוליאני עם קודקוד פלט יחיד, וקיימת השמה לקודקודי הקלט שמספקת את המעגל $\}$

הערה 65. בהשמה, אנחנו מתכוונים שקודקודי הקלט יקבלו ערך של 0 או 1, ונבצע את החישוב בהתאם למעגל הבוליאני ($\wedge, \vee,$ וכו') ונבדוק אם הפלט הוא 1 (= השמה מספקת).

משפט 66. $CSAT \in NP$.
הוכחה: נגדיר מוודא פולינומי V בצורה הבאה: V יקבל את המעגל C ואת העדות y להיות פונקציית השמה עבור קודקודי הקלט V_1: $\phi : V_1 \to \{0,1\}$ ונגדיר את המוודא בצורה הבאה -
$V(C,\phi)$: הצב לפי פונקציית ההשמה את הקודקודים ובצע את החישובים הבוליאניים והחזר את פלט החישוב.
בבירור, החישוב פולינומי בגודל הגרף G ולכן מדובר באלגו' פולינומי.
שלמות: אם $C \in CSAT$, אזי קיימת פונקציית השמה ϕ שאורכה לכל היותר כמס' הקודקודים בגרף וכן בפרט באורך פולינומי, עבורה $V(C,\phi) = 1$ שכן היא מספקת את המעגל הבוליאני.
נאותות: אם $C \notin CSAT$, לכל פונקציית השמה ϕ יתקיים $V(C,\phi) = 0$ שכן C לא ספיקה.
לכן סה"כ $CSAT \in NP$.

■

לאחר שהראינו כי $CSAT \in NP$, נרצה להוכיח כי $CSAT \in NPC$. כלומר, נרצה להציג רדוקציית קארפ מבעיה $S \in NP$ ל$CSAT$. לשם כך, נעזר בטענת העזר הבאה:

משפט 67. (ללא הוכחה) לכל פונקציה $f : \{0,1\}^n \to \{0,1\}^m$ קיים מעגל בוליאני שמחשב אותה, אך הגודל שלו יכול להיות אקספוננציאלי בn. (פונקציה שממירה מחרוזת באורך n למחרוזת באורך m).

משפט 68. $CSAT$ היא NP-קשה.
הוכחה: תהי $S \in NP$. נרצה לבנות פונקציה שבהינתן קלט x מחזירה מעגל C_x כך שמתקיים $x \in S \iff f(x) = C_x \in CSAT$.
$S \in NP$ ולכן קיים פולינום p ומ"ט עם סרט בודד V כך ש: $x \in S \iff \exists y : |y| \leq p(|x|) : V(x,y) = 1$.
באופן דומה להוכחה הקודמת על קיום בעיית NP-קשה, קיים פולינום $t(n)$ כך שזמן הריצה של V חסום ע"י $t(|x|)$.
אם כן, ננסה לחשוב איך נראה תהליך החישוב של מכונת טיורינג V על קלט (x,y). המכונה עושה לכל היותר $t(|x|)$ צעדים, ובמקרה הגרוע מדובר בהליכה של הראש קורא כותב ימינה ושמאלה, הסרט שבאורך $t(|x|)$ יידרש להעתקה $t(|x|)$ זמן לכן סה"כ לכל היותר זמן פולינומי של $t(|x|)^2$.

את המכונה, נתעד באמצעות טבלה כך שנבצע השמה של $t(|x|)$ המקומות הרלוונטיים בסרט בשורה של טבלה, ולתחזק $t(|x|)$ שורות (עבור כל צעד). בכל מיקום בטבלה, (x_i, q_i) נחזיק זוג כך שx_i הוא משתנה שעליו ראש הסרט הקורא־כותב מצביע וq_i הוא מצב המכונה. אם נסכם ונדייק - אנחנו מחזיקים $t(|x|)$ שורות לצעדים, ו$t(|x|)$ עמודות עבור גודל הסרט בכל צעד.

בהינתן תא (x_i, q_j) - מה אנחנו צריכים בשביל לדעת לחשב אותו? אנחנו זקוקים לשלושת התאים: $(x_{i+1}, q_j), (x_{i-1}, q_j), (x_{i-1}, q_{j-1})$. מדוע? בכל צעד בודד של המכונה, היא יכולה לשנות רק דבר אחד: את התא שבו הראש נמצא ברגע זה. היא יכולה לכתוב תו חדש וכן לשנות מצב פנימי ולזוז צעד ימינה או שמאלה. בהינתן מיקום i וזמן j, מה ישפיע על מיקומו?

התא מעליו - (x_{i-1}, q_j): הוא מכיל את המצב הקיים, צריך לדעת מה היה בתוך המיקום הקודם בסרט (x_{i-1}) על מנת לשנות אותו כרגע.

התא משמאלו מעלה - (x_{i-1}, q_{j-1}): חשוב בגלל תנועה ימינה. אם ראש המכונה היה ב$i-1$ והחליט לזוז קדימה בצעד הקודם, הוא הגיע לתא i.

התא מימינו מעלה - (x_{i-1}, q_{j+1}): אם זזים שמאלה.

כלומר סה"כ אנחנו נדרשים לכל תא לדעת את התא מעליו (מכיל מה היה במצב הנוכחי בזמן הקודם) ומעליו משמאלו ומימינו - על מנת לדעת אם לזוז ימינה או שמאלה.

נסמן בN את אורך הקידוד של תא בטבלה ונבחין כי N תלוי רק בתיאור המכונה V ולא תלוי בקלט x. לכן, ישנה פונקצייה $f : \{0,1\}^{3N} \to \{0,1\}^N$ שמחשבת ערך של תא בהינתן הערכים של שלושת התאים שמעליו. לפי טענת העזר, קיים מעגל בוליאני שמחשב את הפונקציה f. כעת נתאר את הרדוקצייה:

בהינתן קלט x, נבנה את המעגל C_x ב"שכבות":

השכבה הראשונה תהיה מורכבת מ$t(|x|)$ תאים כאשר אנחנו מציבים ב$|x|$ התאים הראשונים את הקלט x ומשאירים $p(|x|)$ קודקודים עבור הקלט y.

את השכבות השנייה עד $t(|x|)$ נבנה כך שכל תא יורכב ממעגל שמחשב אותו כתלות בתאים שמעליו.

בשכבה האחרונה נגדיר קודקוד פלט יחיד שמקבל את ערך התא השמאלי ביותר.

נבחין כי גודל המעגל סופי כמובן ובהינתן קלט x הגודל שלו הינו $O((t(|x|))^2)$.

וכן מתקיים $C(x,y) = 1 \iff V(x,y) = 1$ (בגלל הצורה שבנינו את המעגל, פלט המכונה יהיה 1 אמ"מ כנשציב במעגל C שמדמה את המכונה נקבל 1).

לכן, $x \in S \iff$ קיים y באורך $p(|x|)$ לכל היותר כך ש$V(x,y) = 1 \iff C(x,y) = 1 \iff C_x \in CSAT$ כנדרש.

■

כל שנותר על מנת להוכיח כי SAT היא בעיית NP־קשה הוא להראות רדוקציית קארפ מ$CSAT$ לSAT. ובכן:

משפט 69. מתקיים $CSAT \leq_m^P SAT$

הוכחה: נרצה בהינתן מעגל בוליאני C כקלט ל$CSAT$, לבנות נוסחה בוליאנית ϕ בצורת CNF כקלט לSAT כך שיקיים: $\phi \in SAT \iff C \in CSAT$.

למען האינטואיציה, נבחין כי מעגל שמדמה CNF הוא מעגל "שטוח" שיש לו שורה אחת של OR (עם NOT בניהם) ואז שורה נוספת של AND. לכן - אינטואטיבית נרצה "לשטח" את המעגל.

כלומר, נפצל את המעגל לכמה מעגלים קטנים. מכל אחד מהם נבנה נוסחה בצורת CNF ולבסוף נחזיר נוסחה שתהיה מורכבת מכל הנוסחאות הקטנות.

יהיו קודקודי הקלט $\{x_i\}_{i=1}^n$ ויהיו השערים הלוגיים $\{g_i\}_{i=1}^m$. נבנה נוסחה עם $n+m$ משתנים $x_1,..,x_n,g_1,..,g_m$ שייצגו את קודקודי הקלט והשערים.

פסוקיות: לכל שער i במעגל C נגדיר נוסחה בצורת CNF: ϕ_i שתהיה מסופקת אמ"מ הערך שמקבל המשתנה g_i עקבי עם הערך שמקבל השער g_i במעגל C.

למשל: עבור $g_1 = x_1 \wedge x_2$ טבלת האמת נראית כך:

x_1	x_2	g_1	$g_1 =_? x_1 \wedge x_2$
T	T	T	T
T	T	F	F
T	F	T	F
T	F	F	T
F	T	T	F
F	T	F	T
F	F	T	F
F	F	F	T

אזי אם נסתכל על השורות בטבלה שערכן F, ונשים מעליהן אלו בדיוק ערכי הT. כלומר:

$$(g_1 = x_1 \wedge x_2) \equiv \overline{(x_1 \wedge x_2 \wedge \overline{g_1})} \wedge \overline{(x_1 \wedge \overline{x_2} \wedge g_1)} \wedge \overline{(\overline{x_1} \wedge x_2 \wedge g_1)} \wedge \overline{(\overline{x_1} \wedge \overline{x_2} \wedge g_1)} \equiv$$

$$(\overline{x_1} \vee \overline{x_2} \vee g_1) \wedge (\overline{x_1} \vee x_2 \vee \overline{g_1}) \wedge (x_1 \vee \overline{x_2} \vee \overline{g_1}) \wedge (x_1 \vee x_2 \vee \overline{g_1})$$

והרי שמהשער g_1 הצלחנו להפיק נוסחת CNF בשם ϕ_1.
בדומה, כך נעשה על כל השערים ונקבל נוסחאות $\{\phi_1, ..., \phi_m\}$. בנוסף, עבור הקודקוד שנכנס לקודקוד הפלט נוסיף פסוקית ϕ_{m+1} עם המשתנה שלו. (ממש משתנה x_i שיתקיים $\phi_{m+1} = x_i$).
לבסוף, נחזיר את הנוסחה הבאה:

$$\phi = \bigwedge_{i=1}^{m+1} \phi_i = \phi_1 \wedge \phi_2 \wedge ..... \wedge \phi_{m+1}$$

זמן הבנייה: מס' השערים בC חסום לינארית באורך הקלט. עבור כל שער ביצענו כמות קבועה של צעדים על מנת לבנות נוסחה ϕ_i לכן סה"כ מדובר באורך פולינומי בגודל הקלט.
נוכיח נכונות:
$\Longrightarrow$ נניח $C \in CSAT$. לכן, קיימת השמה $z = z_1, ..., z_n$ עבור קודקודי הקלט במעגל שמספקת אותו. נסתכל על ההשמה לנוסחה ϕ בה כל משתנה x_i מקבל את הערך z_i וכל משתנה g_i מקבל את ערך השער g_i במעגל C. לפי הבנייה, כל פסוקית ϕ_i מסופקת וכן הפסוקית ϕ_{m+1} מסופקת כי $C(z) = T$ (והרי ϕ_{m+1} זה המשתנה שנכנס אל שער הפלט שמסופק) וסה"כ ϕ מסופקת ומתקיים $\phi \in SAT$.
$\Longleftarrow$ נניח כי $\phi \in SAT$. לכן קיימת השמה $z = z_1, ..., z_n, a_1, ..., a_m$ למשתנים בϕ שמספקת אותה. נסתכל על ההשמה $z_1, ..., z_n$ לקודקודי הקלט במעגל. לפי הבנייה, כל שער g_i במעגל מקבל את הערך a_i (כי הפסוקית ϕ_i שמוודאית זאת מסופקת) ובנוסף ϕ_{m+1} מסופקת שכן הערך שמקבל קודקוד הפלט הוא T). סה"כ כל הקודקודים מסופקים ולכן $C \in CSAT$.
סה"כ הראינו את נכונות הרדוקציה. כנדרש.

■

מסקנה: SAT היא NP־שלמה.

4 הרצאה 4

הערה 70. כיצד מראים שבעיה $S \in NP$ היא NP שלמה? מספיק להראות רדוקציית קארפ מבעיה X שכבר ראינו שהיא NP שלמה לבעיה S. מדוע? כל בעיה בNP ניתן להמיר (רדוקציה) לבעיה X, ואת X ניתן להמיר לS - ומטרנזטיביות הרדוקציה.

4.1 בעיות NP שלמות שעוסקות בגרפים

משפט 71. IS היא בעייה NP שלמה.
הוכחה:
ברור כי $IS \in NP$ (המוודא יקבל את הקבוצה S ויבדוק לכל $u, v \in S$ אם $(u, v) \notin E$).
כמו כן, ראינו בדוגמה 52 שמתקיים: $SAT \leq_m^P IS$. SAT היא בעיה NP שלמה, ולכן כל בעיה $S \in NP$ ניתן להמיר לSAT, וכן ניתן להמיר את SAT לIS. מכאן:

$$\forall S \in NP : S \leq_m^P SAT \leq_m^P IS \Longrightarrow \forall S \in NP : S \leq_m^P IS$$

כנדרש.

■

72 èôùî. $Clique$ היא בעייה NP שלמה.
הוכחה:
ברור כי $Clique \in NP$ שכן המוודא יקבל את הקבוצה S ויבדוק לכל $u, v \in S$ אם $(u, v) \in E$.
כמו כן, ראינו כי $IS \leq_m^P Clique$ וכיוון שIS בעיה NP שלמה, מטרנזטיביות הרדוקציה נקבל כי:

$$\forall S \in NP : S \leq_m^P Clique$$

ולכן $Clique$ NP שלמה.

■

73 äøãâä. יהי גרף $G = (V, E)$. כיסוי קודקודים היא קבוצה S כך ש$\forall (u, v) \in E : u \in S \vee v \in S$. פורמלית:

$$VC = \{(G, k) | \text{G has Vertex Cover of size} \leq k\}$$

èôùî 74. $VC \in NPC$, כלומר VC היא בעיה NP שלמה.

הוכחה:

ראשית, $VC \in NP$. נגדיר את המוודא הבא:

$V((G,k),S)$: אם $|S| \neq k$ החזר 0. אחרת, בדוק אם S היא קבוצה שולטת ב־G:

לכל $(u,v) \in E$: בדוק אם $u \in S$ או $v \in S$. אם שניהם לא ב־S החזר 0.

אחרת, החזר 1.

נראה כי V פולינומי שכן מבצע בדיקות בעלות קבועה לכל צלע וסה"כ $O(|E|)$. נראה נכונותו:

שלמות ־ נניח כי $(G,k) \in VC$, אזי קיימת קבוצה S מגודל k כך שלכל $(u,v) \in E$ מתקיים $u \in S$ או $v \in S$ ולכן קיימת קבוצה S עבור V יחזיר 1.

נאותות ־ אם $(G,k) \notin VC$ אזי לא קיימת אף קבוצה מגודל S שתקיים את הדרוש ובכל מקרה האלגוריתם יחזיר שלא.

כעת, נוכיח כי $IS \leq_m^P VC$:

בהינתן קלט $(G,k) \in IS$ נרצה לבנות $(G',k') \in VC$ כך שיתקיים $(G,k) \in IS \Longleftrightarrow (G',k') \in VC$

נגדיר את הגרף $G' = G$ ונגדיר $k' = |V| - k$.

הבנייה פולינומית ואף עלותה לינארית. כעת, נראה נכונות:

נניח כי $(G,k) \in IS$. אזי, קיימת קבוצה S בגודל k כך שלכל $u,v \in S$ מתקיים $(u,v) \notin S$. אם כן, נבחין כי הקבוצה $S' = V \backslash S$ היא כיסוי קודקודים ב־G'. נוכיח זאת:

תהי קשת $e = (u,v)$. אזי, נבחין כי בהכרח לא יתכן כי $u \in S$ וגם $v \in S$ כי אז זו סתירה לכך ש־S קבוצה ב"ת. לכן, לפחות אחד מן הקודקודים בה"כ $u \notin S$. כלומר, $u \in V \backslash S$. כלומר, $u \in S'$. וכן מתקיים כי $|S'| = |V \backslash S| = |V| - |S| = |V| - k$, כפי שרצינו.

בכיוון השני, נניח כי $(G,k) \notin IS$. אם כן, לא קיימת קבוצה ב"ת בגודל k ב־G. נב"ש כי $(G',k') \in VC$. אזי, קיים כיסוי קודקודים ב־$G' = G$ בגודל $|V| - k$. נסמנו S. נטען כי במקרה זה $V \backslash S$ היא קבוצה ב"ת. בשלילה כי קיימים $u,v \in V \backslash S$ כך ש־$(u,v) \in E$. אזי, הקשת (u,v) לא מכוסה ע"י S בסתירה לכך ש־S כיסוי קודקודים. לכן, בהכרח $(G',k') \notin VC$.

כעת, כיוון ש־$IS \in NPC$ והראינו $VC \in NP$ והראינו רדוקציית קארפ מ־IS ל־VC נקבל כי $VC \in NPC$. כנדרש.

■

èôùî 75. $3SAT$ היא בעיה NP שלמה.

הוכחה: נו, ברור. (קוק לוין, מקרה מצומצם בו כל פסוקית בגודל 3).

■

äøãâä 76. (בעיית מעגל ההמילטון המכוון). נגדיר את בעיית מעגל ההמילטון המכוון (Directed hamilton cycle) DHC. יהי גרף $G = (V,E)$ מכוון. מעגל המילטון בגרף הוא מעגל $(v_0, ..., v_{k-1}, v_k = v_0)$ שעובר על כל הקודקודים בגרף בדיוק פעם אחת. נגדיר:

$$DHC = \{(G = (V,E) | \text{G directed graph, G has hamilton cycle}\}$$

משפט 77. $DHC \in NPC$.

הוכחה: ראשית, נראה כי $DHC \in NP$. לשם כך, יהי המוודא $V(G = (V,E), C)$ הבא:

$V(G,C)$: V יקבל כקלט את הגרף וכמוודא מעגל בגרף C. ראשית, V יבדוק כי אכן C מעגל. אם C אינו מעגל החזר 0. אחרת: V יבדוק אם כל קודקוד מופיע בדיוק פעם אחת ב־C, ואם כן ישיב שכן ולא אחרת.

בבירור ש־V פולינומי שכן מדובר במעבר על גודל המעגל (נעיר כי אם נראה שגודל המעגל גדול מ־$O(|V|)$ נוכל להפסיק את הריצה שכן לא מדובר במעגל המילטון). מכאן, בבירור אם קיים מעגל המילטון $V(G,C) = 1$. אחרת, אם לא קיים מעגל המילטון מכוון לכל מעגל C כנ"ל נקבל כי $V(C,G) = 0$ ולכן $DHC \in NP$.

כעת, נרצה להראות $SAT \leq_m^P DHC$

הרדוקציה תקבל כקלט נוסחה בוליאנית ϕ ותחזיר גרף מכוון $G = (V,E)$ כך ש: $G \in DHC \Longleftrightarrow \phi \in SAT$

בהינתן נוסחה ϕ עם n משתנים ו־m פסוקיות. הרעיון יהיה לבנות "גרף שכבות". בכל שכבה יהיה משתנה $\{x_i\}_{i=1}^n$. כלומר ־ n שכבות סה"כ. הקודקודים של הגרף ייוצגו ע"י מופעים שונים של אותו משתנה.

לכל משתנה x_i ב־ϕ יהיו $3m + 4$ קודקודים ב־G.

כפי שאמרנו, ישנן n שכבות. תהי השכבה ה־i של המשתנה ה־x_i. אזי, הקשתות באותה השכבה יוגדרו באופן הבא (באשר הקודקודים המתאימים הם $(v_{i,1}, v_{i,2}, ..., v_{i,3m+4}$):

$$E_i = \{(v_{i,j}, v_{i,j+1}) | \forall j \in [1, 3m+3]\} \cup \{(v_{i,j}, v_{i,j-1}) | \forall j \in [2, 3m+4]\}$$

(כלומר, קשת בין כל זוג קודקודים באותה השכבה, עם כיוון השכבה לצד ימין וקשתות בין כל זוג קודקודים עם כיוון השכבה לצד שמאל).

הרעיון הוא, שמיד נסביר אך אם $n = 3, m = 2$ נקבל 10 קודקודים כמו בדוגמה מטה. ישנן 2 פסוקיות, הרעיון יהיה ששני הקודקודים הראשונים באותה שכבה יהיו לא קשורים לפסוקיות, וכך גם השניים האחרונים. אך שאר $3m$ הקודקודים יחולקו כך שכל 3 קודקודים מייצגים פסוקית אחת.

דוגמה:

כמו כן, פרט לקבוצת הקודקודים $\{(v_{i,1},....,v_{i,3m+4})\}_{i=1}^{n}$ שמיוצגים בשכבות, נוסיף קודקודים נוספים $\{b_i(c_i)\}_{i=1}^{m}$ באשר כל קודקוד ייצג פסוקית אחת, וישנן m פסוקיות.

כעת, נניח כי בפסוקית מסוימת קיבלנו $(\overline{x_1}\vee x_2\vee x_3)$. ונניח ואנחנו בשכבה של x_1 ־ הראשונה בגרף. אזי, נניח שזו הפסוקית הi. אזי נלך ל$3i$ הקודקודים שמתאימים לפסוקית הi, ונעביר את הקשתות בצורה הבאה אל $b_i(c_i)$ (הקודקוד שמייצג את הפסוקית הנ"ל): נעביר קשת מהקודקוד השני של הפסוקית אל הקודקוד, כלומר: $(v_{1,3i}\,, b_i(c_i))$ (מדוע $3i$? ננסה להבין ־ ישנם 2 קודקודים בתחילה, ועוד $3(i-1)$ קודקודים שהיו קודם לכן באותה השורה, ועוד קודקוד נוסף (כי אנחנו מעבירים מהשני). לכן סה"כ $3i = 1 + 3(i-1) + 2$). כמו כן, נעביר גם קשת מקודקוד $b_i(c_i)$ אל הקודקוד הראשון של הפסוקית. כלומר, $(b_i(c_i), v_{1,3i-1})$. את ההכללה הנ"ל ניתן לבצע לכל משתנה. כלומר, עבור משתנה j בפסוקית i עם not נעביר את הקשתות:

$$(v_{j,3i}, b_i(c_i)), (b_i(c_i), v_{j,3i-1})$$

עבור משתנה j בפסוקית i ללא not, מחברים אותו מהמשתנה הראשון של השלשה אל משתנה הפסוקית, וממשתנה הפסוקית אל השני (כלומר הפוך):

$$(v_{j,3i-1}, b_i(c_i)), (b_i(c_i), v_{j,3i})$$

בנוסף, נוסיף שני קודקודי התחלה וסיום: s,t.

כיצד אותם נחבר? נסביר רעיונית. קשת ראשונה מs אל $v_{1,1}$ ואל $v_{1,3m+4}$. (כלומר, אל הקודקודים הראשון והאחרון של השכבה הראשונה). משם, קשת $(v_{1,3m+4}, v_{2,1})$ [כלומר מסוף השכבה הראשונה לתחילת השניה] וכן קשת $(v_{1,1}, v_{2,3m+4})$ [קשת מתחילת ראשונה לסוף אחרונה] וכן קשתות $(v_{1,1}, v_{2,1}), (v_{1,3m+4}, v_{2,3m+4})$ וככה באופן כללי. בנוסף נוסיף קשת (t,s) כלומר:

$$E_{connect} = \{(s, v_{1,1}), (s, v_{1,3m+4})\} \cup \{(v_{i,1}, v_{i+1,1}) | \forall i \in [1, n-1]\} \cup \{(v_{i,3m+4}, v_{i+1,3m+4}) | \forall i \in [1, n-1]\} \cup \{(v_{i,1}, v_{i+1,3m+4}) | \forall i \in [1, n-1]\}$$

$$\cup \{(v_{i,3m+4}, v_{i+1}, 1) | \forall i \in [1, n-1]\} \cup \{(v_{n,1}, t), (v_{n,3m+4}, t)\} \cup \{t, s\}$$

$$V = \{s, t\} \cup \{(v_{i,1}, ..., v_{i,3m+4})\}_{i=1}^{n} \cup \{b_i(c_i)\}_{i=1}^{m}$$

$$E = \bigcup_{i=1}^{n} E_i \cup \{(v_{j,3i}, b_i(c_i)), (b_i(c_i), v_{j,3i-1}) | \forall j \in [n], i \in [m] \wedge x_j \ with \ 'not'\} \cup$$

$$\{(v_{j,3i-1}, b_i(c_i)), (b_i(c_i), v_{j,3i}) | \forall j \in [n], i \in [m] \wedge x_j \ without \ 'not'\} \cup E_{connect}$$

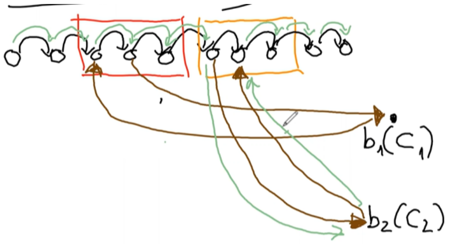
וקיבלנו גרף $G = (V, E)$. אמנם, הפרטים נראים מורכבים אך ברור כי הבנייה פולינומית.

כעת, נראה את נכונות הרדוקציה.

עוד לפני שנגיע להוכחה, ננסה להבין מדוע רצינו לבנות גרף מסובך כל כך: לכל קודקוד היו שרשראות ־ עולה ויורדת. נבחין בדבר הבא: ללא הקשתות שמחברות בין קודקודים $1,2$ לפסוקיות $b_i(c_i)$ ־ תמיד היה מעגל המילטון! לפי מבנה הגרף ־ תמיד אפשר להתחיל מהקודקוד בשכבה הראשונה, לעבור בשרשראות ולעבור בין השכבות. הקודקודים שמחברים בין קודקודים $1,2$ לכל פסוקית הם אלו שמגבילים אותנו.

כעת, נתבונן בדוגמה הבאה המצורפת על מנת להמחיש את האינטואיציה טוב יותר. מסלול עולה הוא מסלול משמאל לימין ־ בו עוברים על הקודקודים. ניתן לבצע כזה, אם הולכים אל $b_2(c_2)$ וחוזרים. נזכיר ־ זהו המצב שבו הופיע $\overline{x_1}$. לא ניתן, אם הולכים אל $b_1(c_1)$ (ללא). עם זאת, ניתן לבצע מסלול יורד: ואז משתמשים ב$b_1(c_1)$ ולא ב$b_2(c_2)$. מסקנתנו: אם עושים מסלול עולה/ יורד ־ ניתן לשלב רק אחד מן הפסוקיות. נרצה לומר כי

אם הנוסחה היא ספיקה נוכל תמיד לבחור מסלול עולה/יורד ולהצליח למעשה ליצור מעגל המילטון מכוון.



הרעיון הוא ־ אם קיימת השמה מספקת, אם משתנה x_i קיבל T נרצה להסתכל על משתניו במסלול עולה, ואם קיבל F נרצה להסתכל על משתניו בגרף במסלול יורד. נבחין כי אם בדוגמה מלמעלה הרי המצב השני הוא בו היה ללא not, ולכן במסלול עולה נצליח לעבור דרכו. מסקנתנו היא כי אנחנו נצליח לבקר במסלול עולה בכל הפסוקיות שסופקו ע"י x_1! (ללא not) ובמסלול יורד בכל הפסוקיות שסופקו ע"י x_1! (שכן היה not). כיוון שאנחנו יודעים שכל הפסוקיות סופקו ־ זה שקול לבקר בכל הפסוקיות: ויש מעגל המילטון. אך נצטרך לפרמל זאת ־

לאחר כל האינטואיציה נתקדם להוכחה.

בכיוון ראשון, נניח כי $\phi \in SAT$ ונראה כי ב-G יש מעגל המילטון מכוון. מעגל ההמילטון יראה כלהלן:

נתחיל בקודקוד s, לאחר מכן נעבור לקודקודי x_1 ונבקר אותם במסלול עולה אם $x_1 = T$ או במסלול יורד אם $x_1 = F$. במהלך המסלול נשלב ביקור בקודקודי הפסוקיות שאותם x_1 סיפק כפי שכבר אמרנו קודם. אילו יהיו פסוקיות בהן x_1 הופיע כחיובי (במקרה של השמת T) או פסוקיות בהן x_1 הופיע כשלילי (במקרה של השמת F). וכך נתקדם בין קודקודי המשתנים השונים תוך כדי הרווית קודקודי הפסוקיות, המעבר בין המשתנים יהיה באמצעות קשתות העזר (בהתאם יש קשתות בסוף וכל תחילת שכבה בגרף). לאחר מכן, לאחר סיום במסלול האחרון נגיע אל t ונעזר בקשת (t, s) לחזור ל-s. מכיוון ש-ϕ ספיקה וקבענו את כיוון המסלולים על השמה מספקת, לכל קודקוד של פסוקית יש לפחות קודקוד אחד שהרווה אותו. המסלול של משתנה זה משלב את קודקוד הפסוקית במעגל ההמילטון שיצרנו. סה"כ, קיבלנו מעגל המילטון ב-G.

בכיוון שני, נניח כי קיים מעגל המילטון ב-G. נרצה להוכיח כי ϕ במקרה זה ספיקה.

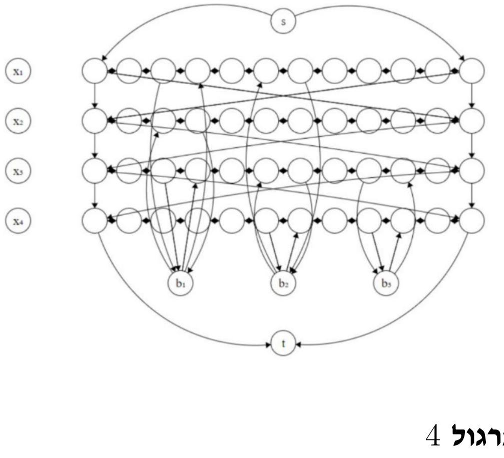
נניח ראשית כי שכל מעגל המילטון כזה מבקר בסדר עולה/יורד בכל קודקודי משתנה ואינו קופץ בין קודקודי משתנים שונים (מיד נוכיח זאת). אם זה אכן המצב ־ אכן נוכל לקבוע השמה ע"פ הכיוון של כל משתנה. אם מסלול עולה נגדיר $x_i = T$ ואם מסלול יורד נגדיר $x_i = F$. כיוון שזהו מסלול המילטון כל אחד מקודקודי הפסוקיות שולב במסלול כלשהו. סה"כ נקבל כי המשתנה שהמסלול שלו שילב פסוקית כלשהו יגרום לאותה פסוקית להסתפק ולכן מכיוון שכל הפסוקיות שולבו הן כולן מסתפקות וקיבלנו כי ההשמה אכן מספקת. רק נותר להוכיח כי מעגל המילטון כזה לא קובץ בין קודקודי משתנים שונים (וזה לא מספיק פורמלי הועבר בהרצאה לכן נשאיר את ההוכחה לקורא.)

סה"כ כיוון ש-$SAT \in NPC$ מתקיים:

$$\forall S \in NP : S \leq_m^P SAT \leq DHC \Longrightarrow S \leq_m^P DHC \Longrightarrow DHC \in NPC$$

■

הערה 78. דוגמה להמחשת ההוכחה לעיל:



4.2 תרגול 4

משפט 79. $DS \in NPC$

הוכחה: בתרגולי עבר ראינו כי מתקיים:

1. $DS \in NP$
2. $VC \leq_m^P DS$

כיוון ש$VC \in NPC$ קיבלנו כי $\forall S \in NP : S \leq_m^P VC \leq DS \Longrightarrow S \leq_m^P DS$ ולכן $DS \in NPC$.

■

משפט 80. נגדיר:

$$Hamiltonian - Cycle = HC = \{G | \text{G undirected graph, G has Hammilton cycle}\}$$

אזי, $HC \in NPC$

הוכחה:

ראשית, נוכיח כי $HC \in NP$: אם כן נגדיר את המוודא הבא: $V(G,C)$: V יקבל את הגרף G ואת המעגל C. V יוודא כי C אכן מעגל וכן עובר בכל קודקוד פעם אחת בדיוק ואם כן יחזיר כן ואחרת לא. בבירור האלגו' פולינומי וכן נראה נכונות: אם אכן $G \in HC$ אזי קיים מעגל המילטון C_H כנ"ל עבורו $V(G, C_H) = 1$. אם $G \notin HC$ אזי לא קיים מעגל המילטון כנ"ל ולכן לכל C יתקיים $V(G,C) = 0$.

כעת, נרצה להראות רדוקציה מDHC. כלומר נוכיח $DHC \leq_m^P HC$.

בהינתן גרף מכוון כקלט לDHC, נסמנו G נבנה גרף G' כך ש: $G' \in HC \Longleftrightarrow G \in DHC$.

לכל קודקוד $u \in G$ נגדיר 3 קודקודים תואמים. כך:

$$V' = \{v_{in}, v_{mid}, v_{out} | v \in V\}$$

כמו כן, נגדיר:

$$E' = \{(u_{in}, u_{mid}), (u_{mid}, u_{out}) | u \in V\} \cup \{(u_{out}, v_{in}) | \forall (u,v) \in E\}$$

וסה"כ $G' = (V', E')$. בבירור, הרדוקציה הנ"ל פולינומית שכן מתקיים $|V'| = 3V, |E'| = 2|V| + |E|$ וסה"כ מדובר בכמות פולינומית של קודקודים וקשתות.

כעת נוכיח $G \in DHC \Longleftrightarrow G' \in HC$

נניח כי $G \in DHC$. אזי, קיים בG מעגל המילטון מכוון ונסמנו: $C = (v_0, ..., v_{n-1}, v_n = v_0)$. מכאן ניתן לבנות את המעגל הבא שעובר בכל הקודקודים בG':

$$((v_0)_{in}, (v_0)_{mid}, (v_0)_{out}, (v_1)_{in}, ..., (v_{n-1})_{in}, (v_{n-1})_{mid}, (v_{n-1})_{out}, (v_0)_{in})$$

בכיוון השני. נניח כי $G' \in DHC$. לכן קיים בG' מעגל המילטוני. נניח בה"כ כי המעגל מתחיל מ$(v_1)_{mid}$. נניח בה"כ כי משם הוא הולך ל$(v_1)_{out}$ [אפשר להגיע רק אליו או ל$(v_1)_{in}$ ואם לא ילך ל$(v_1)_{out}$ נסתכל על המעגל מהכיוון ההפוך]. מ$(v_1)_{out}$ המעגל חייב ללכת אל עותק in כלשהו. נניח בה"כ כי זה $(v_2)_{in}$ שכן הוא חייב ללכת משם ע"פ אל $v_2)_{mid}$ בדומה בה"כ. רעיון זה חוזר על עצמו באופן דומה לכל הקודקודים. סה"כ המעגל בהכרח מהצורה $(v_{1,mid}, v_{1,out}, v_{2,in}, ..., v_{1,mid})$ מעגל המילטוני בG.

סה"כ, קיבלנו כי $HC \in NPC$.

■

משפט 81. נגדיר את הבעיה הבאה:

$AC = ALMOST - Clique = \{(G,k) \mid$ קיימת בG קבוצת קודקודים בגודל k שכולם מחוברים ביניהם מלבד זוג קודקודים יחיד. $\}$

אזי, $AC \in NPC$

הוכחה:

ראשית, בבירור $AC \in NP$ שכן המוודא $V((G,k), S)$: יקבל כקלט את הגרף ואת המס' k וקבוצה S ויבדוק ראשית כי $|S| = k$ ואם לא יחזיר 0. לאחר מכן, יעבור על כל $(u,v) \in S$ ויבדוק אם אכן $(u,v) \in E$. יזכור במשתנה עזר האם ראה עד כה זוג $(u,v) \in S$ וגם $(u,v) \notin E$. אם ימצא זוג כנ"ל ומשתנה העזר שווה 0 יעדכנו לאחד. אם ימצא זוג כנ"ל ומשתנה העזר שווה 1 יחזיר 0 ויעצור. אם יסיים את הריצה וערך המשתנה בדיוק 1 יחזיר 1, ואחרת יחזיר 0. בבירור האלגו' פולינומי שכן זמן הריצה $O(k^2)$ והנכונות ברורה. לכן $AC \in NP$.

כעת נראה $Clique \leq_m^P AC$.

בהינתן קלט (G,k) לבעייה $Clique$ נגדיר גרף (G', k') כקלט לAC כך שיתקיים $(G,k) \in Clique \Longleftrightarrow (G', k') \in AC$

נגדיר $V' = V \cup \{x_1, x_2\}$ וכן: $E' = E \cup \{(x_1, v) | v \in V\} \cup \{(x_2, v) | v \in V\}$ ־מחברים את הקודקודים החדשים למקוריים אך לא ביניהם.

וכן נגדיר: $k' = k + 2$.

נבחין כי האינטואיציה לכך היא שיצרנו בדיוק מבנה של קליקה ללא הקשת (x_1, x_2).

הרדוקציה פולינומית כמובן שכן $|V'| = |V| + 2$ וכן $|E'| = |E| + 2|V|$.

כעת נוכיח שהרדוקציה נכונה.

נניח כי $(G,k) \in Clique$. אזי קיימת קבוצה בגודל k קליקה בG: $C = (v_0, ..., v_{k-1})$. נגדיר $C' = C \cup \{x_1, x_2\}$ ונבחין כי C' כמעט קליקה בG'. שכן: $|C'| = |C| + 2 = k + 2$. וכן, הקליקה המקורית עודנה קיימת וכן לכל קודקוד v מופיעות הקשתות $(x_1, v)(v, x_2)$. אך הקשת $(x_1, x_2) \notin E$ ולכן סה"כ מדובר בכמעט קליקה בG'.

נניח כי $(G',k')\in AC$. אזי, קיימת כמעט קליקה ב־G' בגודל $k+2$.
ראשית, אם קבוצה זו לא כוללת אף אחד מהקודקודים x_1,x_2 אזי אותה הקבוצה קיימת גם ב־G. נסיר את שני הקודקודים שאין צלע ביניהם ונקבל קבוצה בגודל k שמהווה קליקה ב־G כנדרש.
אם בקבוצה זו נמצאים שני הקודקודים הנ"ל, נסיר אותם ונקבל שוב קליקה בגודל k ב־G.
מקרה אחרון הוא שבקבוצה זו נמצא רק קודקוד אחד מבין השניים. נניח בה"כ כי מדובר ב־x_1. זה גורר שיש שני קודקודים ב־C u,v שמחוברים לכל הקודקודים ולא ביניהם. לכן פשוט נקח את $S\cup\{x_2\}\backslash\{u\}$ ונקבל כעת כי (x_1,x_2) הם אלו שלא מחוברים ביניהם וכל השאר כן מחוברים ולכן סה"כ אנחנו כמו במקרה הקודם ושוב נקבל קליקה בגודל k ב־G.
סה"כ $(G,k)\in Clique$.
קיבלנו סה"כ כי $AC\ \le_m^P\ Clique\ \le_m^P S : \forall S\in NP$ ולכן $AC\in NPC$.

■

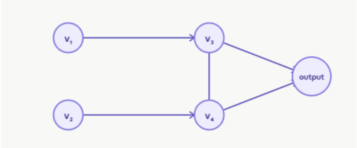
5 הרצאה 5

5.1 המשך בעיות בגרפים ב־NPC

הגדרה 82. עבור גרף $G=(V,E)$ נאמר כי $\phi : V\to\{1,..,k\}$ היא צביעה חוקית ב־3 צבעים אם לכל קשת $(u,v)\in E$ מתקיים $\phi(u)\neq\phi(v)$.

משפט 83. נגדיר: $3COL=\{\ G \mid$ ל־G יש צביעה חוקית ב־3 צבעים $\}$
אזי, $3COL\in NPC$

הוכחה: כפי שראינו בעבר, $3COL\in NP$. כעת, נוכיח $SAT\le_m^P SAT$.
בהינתן נוסחה ϕ בצורת CNF כקלט ל־SAT נבנה גרף G כך שיתקיים: $G\in 3COL \iff \phi\in SAT$
1. נגדיר שלושה קודקודים T,N,F ונחבר אותם בקשתות אחד לשני בצורת משולש. דהיינו: $(T,N),(N,F),(F,T)$.
2. לכל משתנה $x_i\in\phi$ נגדיר שני קודקודים: $v_i,\overline{v_i}$ ונחבר אותם בקשת זה לזה. כלומר, $(v_i,\overline{v_i}) : \forall i\in[n]$ (כאשר n הוא מס' המשתנים). נבחין שזה אומר כי כל אחד מהם יקבל צבע שונה (טוב לנו ־ לא נרצה ש־x_1 יקבל T וגם $\overline{x_1}$ יקבל T). את כל הקודקודים הנ"ל נחבר לקודקוד הניטרלי ־ N. כלומר, $(v_i,N),(\overline{v_i},N) : \forall i\in[n]$
כעת נבחין כי אם אכן ישנה צביעה חוקית ־ יש דרך ליצור השמה חוקית לנוסחה. מדוע? כיוון שכל המשתנים מחוברים לקודקוד הניטרלי, הם שונים מהצבע שלו. הצבע שלו ־ שונה מהצבע של T,F לכן כל אחד מהמשתנים יקבל T,F. אבל עלינו כעת לדאוג שכל הפסוקיות יהיו מסופקות: כי עד לשלב זה ־ תמיד יש צביעה חוקית.
3. עבור כל פסוקית בעלת ליטרל אחד ־ נוסיף קשת מהקודקוד המתאים לליטרל אל F. מדוע? זה ידאג כי הם לא יקבלו את אותה השמה (צבע) ־ שכן יהיו מחוברים בקשת. לכן אותו ליטרל יקבל T (הוא לא יקבל N כי מחובר ל־N).
4. נגדיר רכיב "OR (גאד'אט) שידמה פעולה של OR בין המשתנים ־ עבור כל פסוקית עם יותר מליטרל אחד.
כיצד נגדירו?
עבור $k=2$ (משתנים) נגדיר:



עבור $k>2$ קודקודי קלט נגדיר $k-1$ רכיבי OR בינאריים כך שקודקוד הפלט של רכיב OR ה־i הוא אחד מקודקודי הקלט של רכיב ה־OR ה־$i+1$ (כך הם נבנים במעין שכבות").
5. כל פסוקית עם יותר מליטרל אחד, נגדיר רכיב OR עם הליטרלים שמתאימים לאותה פסוקית. כלומר, קודקודי הקלט של הרכיב הינם הקודקודים המתאימים לליטרלים שמופיעים בפסוקית. וכן נחבר את קודקוד הפלט של הרכיב לקודקודים F ו־N. (כי נרצה שקודקוד הפלט של הרכיב יצא T, ולכן יהיה בצבע שונה מ־F,N).
ראשית, נשים לב כי זמן הבניה אכן פולינומי. כעת נעבור להוכחת הנכונות:
לפני כן נפתח בטענת עזר.

טענה: עבור רכיב OR ־ אם קיים i כך ש־$\sigma(in_i)=T$ (קודקוד קלט ברכיב הוא T) אזי קיימת צביעה חוקית לכל רכיב ה־OR כך שקודקוד הפלט מקבל צבע T. כלומר, $\sigma(Output_i)=T$.

הוכחה: ההוכחה באינדוקציה. נראה את הבסיס בלבד שכן לאחר מכן זה רקורסיבי באינדוקציה. נניח כי $k=2$. בה"כ הקודקוד שקיבל הוא התחתון. ניתן אם כן להשלים לצביעה חוקית כך שזוג הקודקודים בעמודה המקבילה יקבלו את הערכים N,F ואז אכן הקודקוד פלט מקבל T.
כעת נעבור להוכחת האמ"פ.
$\Longrightarrow$ נניח $\phi\in SAT$. אזי קיימת השמה מספקת τ. נראה שקיימת צביעה חוקית $\sigma : V\to\{T,F,N\}$ כך:

$$\sigma(T)=T,\sigma(F)=F,\sigma(N)=N$$

לכל משתנה x_i נגדיר: $\sigma(v_i)=\tau(x_i)$ וכן $\sigma(\overline{x_i})=\overline{\tau(x_i)}$ [כלומר צובעים את הליטרלים בהתאם להשמה. אם משתנה קיבל ערך T, השלילה שלו יקבל F וכו'.]

τ מספקת את ϕ ולכן בכל פסוקית קיים משתנה שמספק אותה, לכן לפי טענת העזר ניתן לצבוע את רכיב הOR שלה. כך, סה"כ הצלחנו לצבוע את כל רכיבי הOR ואת הקודקודים שהגדרנו ולכן סה"כ קיימת צביעה חוקית לG ולכן $G \in 3COL$.

$\Longleftarrow$ נניח כי $G \in 3COL$.

נעזר בטענת עזר: עבור רכיב OR, אם $\sigma(in_1) = \sigma(in_2) = ... = \sigma(in_k) = F$ אזי בכל צביעה חוקית $\sigma(output_i) = F$. ההוכחה בדומה לקודם, באינדוקציה. אם $k = 2$ בבירור שני הקודקודים באותה שורה יכולים לקבל רק T, N ולכן עבור קודקוד הפלט נותר רק F.

כעת, נניח כי קיימת לG צביעה חוקית ב3 צבעים: $\sigma : V \to \{T, N, F\}$. בה"כ נניח כי $\sigma(T) = T, \sigma(F) = F, \sigma(N) = N$. נרצה להגדיר את ההשמה הבאה:

נשים לב כי לכל i $\sigma(v_i), \sigma(\overline{v_i}) \in \{T, F\}$ ולכן בפרט הצבעים שלהם שונים. אחד מהם קיבל T והשני F. נגדיר השמה $\tau(x_i) = \sigma(v_i)$.

נרצה להוכיח כי אכן ההשמה מספקת. שכן, אם נניח בשלילה כי קיימת פסוקית אחת לא מסופקת - אזי כל הליטרלים בה הם F ומטענת העזר אזי רכיב ה$output$ שלה הוא F. אבל מהבנייה, רכיב ה$output$ מחובר לF, N שקיבלו את הצבעים N, F בהתאמה. ולכן זו סתירה לכך שישנה צביעה חוקית (שכנים עם F). לכן, ההשמה אכן מספקת.

סה"כ הראינו רדוקציה מ$SAT \in NPC$ ולכן סה"כ $3COL \in NPC$.

■

5.2 בעיות מספרים בNPC

הגדרה 84. נגדיר את הבעיה הבאה:

$SSS = Subset-Sum = \{\ (A, b)\ |$ A היא רב קבוצה של מספרים טבעיים , $b \in \mathbb{N}$, קיימת בA תת קבוצה של מספרים טבעיים שסכומם b $\}$

הערה 85. רב קבוצה איננה קבוצה. הסדר איננו משנה - מס' המופעים של כל איבר כן. בפרט, $\{1, 2, 3\} \neq \{1, 1, 2, 3\} = \{1, 2, 3, 1\}$.

משפט 86. $SSS = Subset - Sum \in NPC$.

הוכחה:

בבירור כי $SSS \in NP$, המוודא יקבל את האינדקסים של תת הקבוצה. יבדוק שאכן מדובר בתת קבוצה - יסכום את האיברים ויבדוק שערך זה שווה לb. מדובר בזמן פולינומי, ולכן סה"כ $SSS \in NP$.

כעת, נרצה להראות $VC \leq_m^P SSS$

בהינתן קלט (G, k) כקלט לVC נבנה קבוצה A וערך b קלט לSSS כך ש: $(A, b) \in SSS \iff (G, k) \in VC$.

נסמן $G = (V, E)$ כך ש: $V = \{v_i\}_{i=1}^n, E = \{e_i\}_{i=1}^m$. עבור כל קודקוד v_i נגדיר מספר:

$$x_{v_i} = 10^m + \sum_{j=v_i \in e_j} 10^{j-1}$$

(למעשה ערך זה נראה מאוד מוזר. אבל! אם נאנדקס בטבלה בינארית לכל קודקוד האם הוא נמצא בקשת מסוימת, ונוסיף אחד משמאל. זה בדיוק הערך שנקבל לכל v_i).

עבור כל קשת e_j נגדיר ערך: $x_{e_j} = 10^{j-1}$.

סה"כ הקבוצה $A = \{v_1, ..., v_n, e_1, ..., e_m\} = V \cup E$.

נגדיר $b = k \times 10^m + \sum_{j=1}^m 2 \times 10^{j-1}$. (מה הרעיון? דיברנו על טבלה קודם - אם קשת תהיה מכוסה, יהיה 1 לפחות בעמודה שלה כי קודקוד כלשהו יכסה אותה. אם תכוסה משני צדדיה - יהיה 2. לכסות את כל הקשתות בגרף - לשים אחד לפחות בכל עמודה. העמודה הראשונה בטבלה מיוצגת ע"י $k \times 10^m$ - זה יבטיח שגודל הקבוצה לא יעלה על k).

בבירור, הבנייה פולינומית. כעת נרצה להראות נכונות:

$\Longrightarrow$ נניח כי $(G, k) \in VC$. נניח כי בG כיסוי קודקודים S בגודל k. נתבונן בקבוצה הבאה:

$$B = \{x_{v_i} | v_i \in S\} \cup \{x_{e_j} | e_j = (v_{j_1}, v_{j_2}) \wedge (v_{j_1} \in S \vee v_{j_2} \in S) \wedge \{v_{j_1}, v_{j_2}\} \subsetneq S\}$$

כלומר - מספרי הקודקודים, וכל קודקודי הקשתות כך שרק קודקוד אחד מהצדדים נמצא בS (לא הקשתות ששני קודקודי הקשת מכוסים ע"י S).

נראה כי סכומה:

$$\sum_{b \in B} b = \sum_{v_i \in S} x_{v_i} + \sum_{e_j} x_{e_j}$$

$$\sum_{v_i \in S} (10^m + \sum_{j=v_i \in e_j} 10^{j-1}) = k \times 10^m + 2 \sum_{(v_{j_1} \in S \wedge v_{j_2} \in S)} 10^{j-1} + \sum_{(v_{j_1} \in S \vee v_{j_2} \in S)} 10^{j-1} +$$

$$\sum_{e_j} x_{e_j} = \sum_{(v_{j_1}\in S \vee v_{j_2}\in S)} 10^{j-1}$$

ואם נחבר:

$$\sum_{b\in B} b = k\times 10^m + 2\sum_{j=1}^{m} 10^{j-1}$$

כלומר ־ קבוצת הקשתות שלוקחים ־ אלו הערכים של הקשתות שמכוסות ע"י רק 1. זה מגיע על מנת לאזן את הסכום.
$\Leftarrow$ נניח כי $(A,b)\in SSS$. נניח כי קיימת בA תת קבוצה $B\subseteq A$ שסכום איבריה b. נרצה לטעון כי קיים בG כיסוי קודקודים בגודל k. נגדיר $S=\{v_i|x_{v_i}\in B\}$.
ראשית נטען כי היא בגודל k שכן נשים לב כי הדרך היחידה לקבל b כנ"ל שיש בו $k\times 10^m$ היא ע"י k מספרים שמתאימים לקודקודים לכן בהכרח מס' הקודקודים x_{v_i} הוא k.
שנית, נרצה להוכיח כי היא מכסה את כל הקשתות. שכן, הדרך היחידה להגיע לערך 2 כפול 10^{j-1} היא אם ורק אם מצאנו לפחות קודקוד אחד שמשתתף בקשת e_j (שהרי לא יכולנו להגיע לערך זה מבלי שלפחות אחד מהמספרים יופיע בקבוצה).
סה"כ כיוון ש$VC\in NPC$ הראינו כי $SSS\in NPC$.

■

5.3 תרגול 5

משפט 87. תהי A רב קבוצה. נגדיר את הבעיה הבאה:

$$Partition = \left\{ A|\exists A'\subseteq A : \sum_{a\in A'} a = \sum_{a\in A\backslash A'} a \right\}$$

כלומר, קיימת תת קבוצה שמחלקת את A לשתי קבוצות $(A', A\backslash A')$ בסכום זהה. אזי, $Partition\in NPC$.
הוכחה:
תחילה, נראה כי $Partition\in NP$. אם כן, נגדיר אלגוריתם V שיקבל את הקבוצה A וקבוצה $A'\subseteq A$. המוודא יפעל כך:
$V(A,A')$: ראשית בדוק אם אכן $A'\subseteq A$. אם לא, השב 0. אחרת: חשב $x=\sum_{a\in A'} a, y=\sum_{a\in A\backslash A'} a$. אם $x=y$ השב 1, אחרת השב 0.
זמן הריצה פולינומי, שכן הבדיקה על ההכלה עלותה $O(|A|)$ וכן חישוב x,y עולה $O(|A|)$ גם כן וסה"כ זמן פולינומי בגודל הקלט.
באשר לנכונות, שלמות: בבירור אם קיימת קבוצה כנ"ל A' $V(A,A')=1$. אם לא קיימת כזו, לכל תת קבוצה המוודא ישיב 0.
כעת, נראה כי $SSS\leq_m^P Partition$.
בהינתן קלט (A,b) לבעיה SSS נגדיר קבוצה B כך ש: $(A,b)\in SSS \Longleftrightarrow B\in Partition$.
היכן הפער שלנו? נרצה כי אם קיימת תת קבוצה שסכומה b, נוכל לחלק לשתי קבוצות. נרצה למעשה כי נוסיף איבר שסכומו x ואז: $\frac{S+x}{2}$ (סכום הקבוצה) זו חלוקה. נרצה שזה יקרה אמ"מ קיימת בA תת קבוצה שסכומה b. אם נניח כי x נמצא בה אזי $x+b=\frac{S+x}{2}$ ואם נפתור נקבל $x=S-2b$.
לכן, נוסיף איבר $\{x\}$ $B=A\cup$ וכן נגדיר $x=|2b-\sum_{a\in A} a|$. בבירור, הבניה פולינומית. כעת נראה נכונות:
$\Longrightarrow$ נניח כי $(A,b)\in SSS$. אזי, קיימת בA תת קבוצה שסכומה b. נסמנה S.
נחלק למקרים. אם $2b-\sum_{a\in A} a\geq 0$ אזי נגדיר את החלוקה הבאה: $(S, B\backslash S)$. נראה כי: b $\sum_{b\in B\backslash S} b = 2b-\sum_{a\in A} a\in A+\sum_{a\in A} a-b=b$ ואכן מדובר בשתי קבוצות בגודל b.
אחרת, כלומר $2b-\sum_{a\in A}\leq 0$ אזי $x=\sum_{a\in A} a-2b$. וכן סכום B הוא $2(\sum_{a\in A} a-b)=-2x$. לכן במקרה זה נתבונן בחלוקה $(S\cup\{x\}, B\backslash\{x\}\backslash S)$ ונראה כי:

$$S\cup\{x\} = b+\sum_{a\in A} a-b = \sum_{a\in A} a$$

$$B\backslash\{x\}\backslash S = \sum_{a\in A} a - (\sum_{a\in A} -b) - b = \sum_{a\in A} a$$

ואכן הקבוצות בגודל שווה.
$\Leftarrow$ נניח כי $B\in Partition$. לכן קיימת חלוקה של B לשתי קבוצות בעלות גודל שווה, נסמנן B_1, B_2. בה"כ כי $x\in B_1$. ושוב נחלק למקרים:
אם $2b-\sum_{a\in A} a\geq 0$ נראה כי:

$$\sum_{a\in B_2} = \frac{1}{2}(\sum_{a\in B_1} a + \sum_{a\in B_2} a) = 0.5\sum_{a\in B} a = 0.5\times 2b = b$$

לכן $B_2 \subseteq B$ ובגודל b.
אם $\sum_{a\in A} a - 2b \geq 0$. נראה כי:

$$x + \sum_{a\in A} a = 2(\sum_{a\in A} a - b) \Longrightarrow \sum_{a\in A_1} = \frac{1}{2}(2\sum_{a\in A} a - 2b) = \sum_{a\in A} a - b$$

$$\sum_{a\in A_1} a - x = \sum_{a\in A} a - b - \sum_{a\in A} a + 2b = b$$

לכן $A_1\backslash\{x\}$ היא תת קבוצה שמוכלת בA שסכומה b.
סה"כ קיבלנו כי $Partition \in NPC$.

■

משפט 88. נגדיר ADS באופן הבא:
$ADS = \{(G,k) \mid$ קיימת בG קבוצת קודקודים בגודל k ששולטת על כל הקודקודים פרט לאחד. $\}$
אזי, $ADS \in NPC$.
הוכחה:
בבירור כי $ADS \in NP$. כעת נראה ברדוקציה מDS: $DS \leq_m^P ADS$
בהינתן קלט (G,k) כקלט לDS נגדיר G',k' כקלט לADS כך ש$(G',k') \in ADS \Longleftrightarrow (G,k)\in DS$.
נגדיר $G' = G\cup\{x\}$ כאשר x הוא קודקוד מבודד וכן נגדיר $k'=k$. בבירור הבניה פולינומית. כעת נראה נכונות:
בכיוון ראשון, נניח $(G,k)\in DS$. אזי קיימת קבוצה S ששולטת על הקודקודים בG. נטען כי S היא קבוצה ששולטת על הקודקודים מלבד 1 בG'.
הרי, לכל קודקוד פרט לx היא שולטת עליו כי היא קבוצה שולטת בG, ולכן היא שולטת על כולם פרט לx. לכן סה"כ $(G',k')\in ADS$.
בכיוון שני, נניח כי $(G',k')\in ADS$. אזי קיימת קבוצה ששולטת על כל הקודקודים פרט לאחד. נסמנה S. הרי, קבוצה שולטת היא קבוצה שמקיימת לכל קודקוד כי הוא מכוסה ע"י הקבוצה או שאחד משכניו מכוסה. הרי, אם $x\notin S$ מדובר באותה קבוצה בדיוק ששולטת על כל הקודקודים בG והיא בגודל k וסיימנו. כעת, נניח כי $x \in S$. אזי, קיים קודקוד $x_i \in G$ שלא מכוסה ע"י S. כעת נוכל לטעון כי הקבוצה $S\backslash\{x\}\cup\{x_i\}$ היא עדיין קבוצה ששולטת על כל הקודקודים פרט לאחד. לכן, נחזור לאותו המקרה הקודם ונקבל סה"כ כי $(G,k)\in DS$.

■

6 הרצאה 6

6.1 רדוקציות עצמיות

נזכר: עבור בעיית חיפוש $R = \{ (x,y) \mid$ y פתרון עבור x $\}$ הגדרנו את בעיית ההכרעה המתאימה לה:

$$S_R = \{x|\exists y,(x,y)\in R\}$$

הגדרה 89. רדוקצייה עצמית הינה רדוקציית קוק מR לS_R. (פותרים את בעיית החיפוש בהינתן גישות לאורקל שפותר את בעיית ההכרעה).

משפט 90. לכל $R\in PC$ בעיית חיפוש, אם $S_R \in NPC$ אזי לR יש רדוקציה עצמית.
הוכחה: ראינו (בהרצאה 2) כי לכל בעיה $R\in PC$ ישנה רדוקציית קוק לבעיית ההכרעה $S_{R'} = \{(x,y')|\exists y'' : (x,y'y'')\in R\}$ (כלומר y' היא תחילית של פתרון). וכן ראינו כי $S_{R'}\in NP$ (העדות היא y''). כיוון ש$S_R \in NPC$, אזי לכל בעיה $S\in NP : S\leq_m^P S_R$. בפרט, $S_{R'}\leq_m^P S_R$.
לכן, בפרט קיימת רדוקציית קוק מ$S_{R'}$ לS_R (שכן פשוט אפשר להפעיל את רדוקציית הקארפ, ואז לשאול את האורקל). סה"כ קיבלנו כי ישנה רדוקציית קוק מR ל$S_{R'}$, וישנה רדוקציית קוק מ$S_{R'}$ לS_R ולכן ישנה רדוקציית קוק מR לS_R.

משפט 91. תהיינה A,B זוג בעיות. אם קיימת רדוקציית קארפ $A\leq_m^P B$ אזי ישנה רדוקציית קוק מA לB.
הוכחה: נציג את האלגו' הבא:
א. חשב את $f(x)$ באמצעות הרדוקציה.
ב. שאל את האורקל האם $f(x)\in B$, החזר את תשובתו.

משפט 92. המחלקה P סגורה לרדוקציית קארפ. כלומר, עבור $S\in P$ וקיימת בעיית הכרעה S' כך ש: $S' \leq_m^P S$ אזי $S'\in P$.
הוכחה: לכל קלט $x\in S'$ נחשב את $f(x)$ ונפעיל את האלגוריתם של S על $f(x)$ ונחזיר את תשובתו.

■

èôùî 93. אם $P \neq NP$, אזי $P \cap NPC = \emptyset$.

הוכחה: בשלילה כי קיימת בעיה $S \in P \cap NPC$. אזי, מצד אחד $S \in P$ ומהצד השני $S \in NPC$. לכן, $S \in NPC$ בפרט ולכל $S' \in NP : S' \leq_m^P S$. ממשפט 91, כיוון שישנה רדוקציית קארפ ו-$S \in P$ נקבל כי $S' \in P$ כלומר $\forall S' \in NP : S' \in P$ כלומר $NP \subseteq P$ ומהידע הקודם שלנו $P \subseteq NP$ ונקבל $P = NP$ בסתירה.

■

משפט 94. (ללא הוכחה) משפט לנדר. אם $P \neq NP$, אזי קיימת $S \in NP$ כך ש-$S \notin P$ וגם $S \notin NPC$. כלומר - קיימות בעיות שהן לא ב-P, לא NP שלמות, אך ב-NP.

6.2 המחלקה $CoNP$

הגדרה 95. נגדיר את המחלקה $CoNP$ כך:

$$Co - NP = \{S | \overline{S} \in NP\}$$

הערה 96. למשל, $\overline{SAT} \in coNP$ כי $SAT \in NP$.

משפט 97. $P \subseteq NP \cap Co - NP$ (בפרט אנו יודעים כי $P \subseteq NP$, ולכן $P \subseteq coNP$).

הוכחה: תהי בעיה $S \in P$. אזי, $S \in NP$ מההכלה $P \subseteq NP$. כעת, נוכיח $S \in Co - NP$. נרצה לטעון כי $\overline{S} \in NP$. אם כן, כיוון של-P יש סגירות למשלים (נחזיר את התשובה ההפוכה), אזי ש-$\overline{S} \in P$, כעת, $\overline{S} \in NP$ שוב מההכלה $P \subseteq NP$ וכן נקבל כי $S \in coNP$ לפי הגדרה.

■

èôùî 98. (ההשערה הרווחת, זה לא הוכח) כי $coNP \neq NP$.

נבחין כי השערה זו גוררת כי $P \neq NP$. שכן, אם נכ"ש $P = NP$ אזי אם $S \in NP$ נקבל $S \in P$ ומסגירות P למשלים גם $\overline{S}$ ב-P ולכן $\overline{S}$ ב-NP כלומר $S \in coNP$ ונקבל $P \subseteq coNP$. באופן דומה, אם $S \in coNP$ אזי $\overline{S} \in NP$ ששווה ל-P ולכן $\overline{S} \in P$ ולכן $S \in P$ כלומר $coNP \subseteq P$. סה"כ נקבל $NP = P = coNP$ כלומר $coNP = NP$ בסתירה להשערה. לכן:

$$coNP \neq NP \Longrightarrow P \neq NP$$

משפט 99. אם קיימת $S \in coNP$ כך ש-$S \in NPH$ (NP קשה) אזי $NP = coNP$. (לכן בכיוון ההפוך, לפי ההשערה הרווחת $NP \neq coNP$ אזי לא קיימת בעיה NP-קשה ב-$coNP$). - **מסקנה:** לכל הבעיות שראינו ב-NPC, תחת ההשערה הרווחת הבעיה המשלימה שלהן בהכרח לא ב-NP.

הוכחה: נראה בהכלה דו כיוונית. אך נשים לב כי אם $NP \subseteq coNP$ אזי $NP = coNP$ כי ההכלה נכונה גם בכיוון ההפוך שכן: $S \in NP \Longleftrightarrow S \in coNP \Longleftrightarrow \overline{S} \in NP \Longleftrightarrow \overline{S} \in coNP \subseteq NP$ כלומר נקבל לפי ההכלה כי $NP \subseteq coNP$.

כעת נראה $NP \subseteq coNP$. נניח שקיימת אכן $S \in coNP$ כך ש-S היא NP קשה. אזי, לכל $S' \in NP : S' \leq_m^P S$ (נבחין כי S לא בהכרח ב-NP ! ביקשנו NP קשה ולא שלמה).

טענת עזר: לכל זוג בעיות הכרעה, S, S' אם $S' \leq_m^P S$ אזי $\overline{S'} \leq_m^P \overline{S}$

הוכחת טענת העזר: בבירור, לפי הגדרת הרדוקציה הרי הכל באמ"מ. ולכן אותה רדוקציה נכונה גם עבור המשלימות.

תהי $S' \in coNP$. אזי, $\overline{S'} \in NP$. S היא NP קשה, לכן ישנה רדוקציית קארפ $\overline{S'} \leq_m^P S$. מטענת העזר, $S' \leq_m^P \overline{S}$. אך $\overline{S} \in NP$ (כי $S \in coNP$). לפי משפט שהראינו, NP סגורה לרדוקציית קארפ. ולכן $S' \in NP$ כלומר אכן הראינו את ההכלה.

■

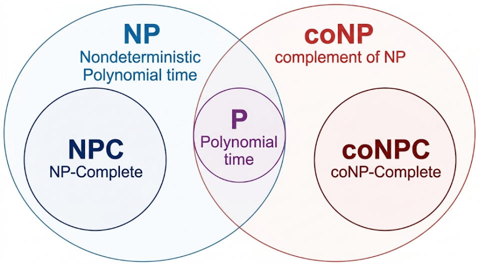
משפט 100. לכל זוג בעיות הכרעה, S, S' אם $S' \leq_m^P S$ אזי $\overline{S'} \leq_m^P \overline{S}$.

משפט 101. $S \in coNP$ אם קיים פולינום P ואלגוריתם פולינומי V כך ש:

$$x \in S \Longleftrightarrow \forall y : |y| \leq p(|x|) : V(x, y) = 1$$

כלומר, לכל y שניקח נקבל 1. זה שונה מההגדרה של NP נבחין, שם הכמת היה $\exists$ וכן הוא $\forall$.

הערה 102. הדיאגרמה הבאה ממחישה את העולם שלנו תחת ההנחה כי $NP \neq coNP$: $P \subseteq NP \cap coNP$, וכפי שראינו NPC זרה ל-$coNP$ במקרה זה.



משפט 103. אם קיימת $S \in NP \cap coNP$ כך שמכל $S' \in NP$ יש רדוקציית קוק לS (שואלים שאלות את S), אזי $NP = coNP$.

הוכחה: נראה בהכלה דו כיוונית. אך נשים לב כי אם $NP \subseteq coNP$ אזי $NP = coNP$ כי ההכלה נכונה גם בכיוון ההפוך שכן: $S \in NP \iff$ $S \in coNP \iff \overline{S} \in NP \iff \overline{S} \in coNP \subseteq NP$ כלומר נקבל לפי ההכלה כי $NP \subseteq coNP$.

לכן תהי $S' \in coNP$. נרצה לטעון כי קיימת רדוקציית קוק מS' לS. יש כאן פער בהוכחה עליו נצטרך להתגבר ־ $S' \notin NP$ (עוד לא הראינו זאת). אך: תמיד יש רדוקציית קוק מS' ל$\overline{S'}$ (ע"י אלגוריתם ששואל את האורקל שמחזיר תשובה הפוכה). נבחין כי $S' \in coNP$ ולכן $\overline{S'} \in NP$. לכן, מההנחה קיימת רדוקציית קוק מ$\overline{S'}$ לS. סה"כ קיבלנו רדוקציית קוק $S' \to \overline{S'} \land \overline{S'} \to S$ ומטרנזטיביות רדוקציית קוק נקבל רדוקציית קוק $S' \to S$.

מה קיבלנו? יש לנו בעיה $S' \in coNP$. וקיים אלגוריתם פולינומי A עם גישות אורקל לבעיה $S \in NP \cap coNP$ אשר מכריע את S'. נרצה להראות כי $S' \in NP$ ולכן נבנה מוודא V פולינומי עבור S' שעונה להגדרות NP (ללא גישות אורקל!). לכן אנחנו נרצה שהמוודא שנריץ יריץ את A עם שינויים. נשים לב כי $S \in NP \cap coNP$ ובפרט $S, \overline{S} \in NP$. **הרעיון יהיה:** לשים לב כי A מבצע לכל היותר מס' פולינומי של שאלות לאורקל. עבור כל שאלה האורקל מחזיר תשובה נכונה ולפיה A ממשיך את ריצתו. אך כיוון ש$S, \overline{S} \in NP$ אזי לכל תשובה שהאורקל מחזיר (יותר נכון: לכל שאלה ששואלים אותו) יש עדות קצרה שניתן בעזרתה להשתכנע. לכן המוודא שנבנה עבור S' יקבל כעדות רשימה של עדויות עבור כל השאלות שA שואל את האורקל במהלך הריצה. כעת נפרמל:

$S \in NP \cap coNP$ ולכן קיימים פולינומים $P_S, P_{\overline{S}}$ ומוודאים $V_S, V_{\overline{S}}$ כך ש:

$$x \in S \iff \exists y : |y| \leq P_S(|x|) : V_S(x,y) = 1$$

$$x \notin S \iff \exists y : |y| \leq P_{\overline{S}}(|x|) : V_{\overline{S}}(x,y) = 1$$

נגדיר מוודא V' עבור S': יקבל כעדות רשימה של עדויות:

$$y = ((\sigma_1, w_1), , ..., (\sigma_t, w_t))$$

באשר $\sigma_i \in \{0,1\}$ היא התשובה לשאלה הi של A, וכן w_i היא העדות שאמורה לשכנע אותנו כי σ_i התשובה לשאלה הi. נניח כי ישנן t (פולינומי) שאלות ולכן סה"כ y חסום בפולינום גם הוא.

V' בהינתן (x,y) כנ"ל מריץ את $A(x)$ וכאשר A ניגש לאורקל בפעם הi ושואל שאלה מהצורה "$q_i \in^? S$" אזי V' בודק אם $\sigma_i = 1$ (ישנה עדות שאומרת לנו אם אכן $\sigma_i = 1$ והיא w_i). כעת המוודא V' ינסה להשתכנע ויריץ $V_S(q_i, w_i)$. אם $V_S(q_i, w_i) = 1$ אזי זו אכן התשובה הנכונה וממשיך את הריצה של A תחת ההנחה כי אכן $q_i \in S$ (האורקל השיב "כן" מבחינת A). אם, $V_S(q_i, w_i) = 0$ (המוודא לא השתכנע). אך זה לא אומר שעדיין $q_i \notin S$) ובמקרה זה V' עצמו יחזיר 0 ויסיים! אם $\sigma_i = 0$ נעשה באופן דומה. המוודא V' יריץ $V_{\overline{S}}(q_i, w_i)$. אם ערך זה שווה 1 אזי זו אכן התשובה הנכונה וממשיך את ריצת A תחת ההנחה כי אכן $q_i \notin S$ (האורקל השיב "לא" מבחינת A). אם $V_{\overline{S}}(q_i, w_i) = 0$ (המוודא לא השתכנע) שוב V' עצמו ישיב 0.

נבחין שהאלגוריתם אכן פולינומי שכן מריץ אלגו' פולינומי A (וגישות האורקל מוחלפות באלגוריתמים פולינומיים). נראה כעת את נכונותו:

שלמות: אם $x \in S'$ אזי בהכרח $A(x) = 1$ (שכן A מכריע את S'). לכן קיימת סדרה של תשובות לשאלות שA שואל את האורקל עבורם A בסוף הריצה משיב 1. אנו יודעים שכל אחת מהתשובות הללו בהכרח תשובה נכונה (כן או לא בS) ־ ולכן לכל אחת מהתשובות קיימת עדות מתאימה עבורן $V_S, V_{\overline{S}}$ יחזירו אכן 1. לכן בסוף הריצה V' אכן ישיב 1.

נאותות: אם $x \notin S'$ לא קיימת סדרת שאלות כנ"ל, לכן לכל עדות באחת השאלות המוודא V_S או $V_{\overline{S}}$ ישיבו 0 וכך גם V'.

סה"כ בנינו מוודא עבור S' פולינומי ולכן $S' \in NP$ כלומר סה"כ הראינו $NP \subseteq coNP$ וכפי שראינו זה גורר $NP = coNP$.

■

6.3 תרגול 6

משפט 104. המחלקה NP סגורה לרדוקציית קארפ. כלומר תהי $S \in NP$ כך ש$S \leq_m^P S'$. אזי $S' \in NP$.

הוכחה: תהי $S \in NP$ ותהי S' כך ש$S \leq_m^P S'$. נוכיח כי $S' \in NP$. אם כן $S \in NP$ לכן קיים לה מוודא V ואלגוריתם פולינומי V כך ש:

$$x \in S \iff \exists y : |y| \leq p(|x|) : V(x,y) = 1$$

נבנה את המוודא הבא: $V'(x,y)$: המוודא יקבל כקלט x ועדות $y \le p(|x|)$. המוודא יחשב את $f(x)$ ויחזיר את $V(f(x),y)$.
המוודא אכן פולינומי שכן מחשב את $f(x)$ בזמן פולינומי (רדוקציית קארפ) ומריץ מוודא V שחסום בזמן פולינומי ולכן סה"כ פולינומי וכן:

$$x \in S' \iff f(x) \in S \iff \exists y : |y| \le p(|x|) : V(f(x),y) = 1 \iff \exists y : |y| \le p(|x|) : V'(f(x),y) = 1$$

לכן סה"כ $S' \in NP$.

■

תרגיל 105. נניח כי $P \ne NP$. הוכח או הפרך: NPC סגורה לרדוקציית קארפ. כלומר, אם $S \in NPC$ וכן $S' \le_m^P S$ אזי $S' \in NPC$.
הפרכה: נניח בשלילה שנכון הדבר ונראה כי $P = NP$. אם כן, תהי בעיה $S \in NPC$ ותהי $S' \in P \subseteq NP$. אם כן כיוון ש$S' \in NPC$ אזי $S' \in P \cap NPC$. כלומר, $S' \in P$ ולכן $\forall S'' \in NP : S'' \le_m^P S' \in P$ ולכן $NP \subseteq P$ וקיבלנו $P = NP$ בסתירה.

■

הערה 106. טריק נחמד: ראינו זאת בהוכחת השקילות של $coNP$ וההגדרה השקולה עם מוודא - אם נגדיר מוודא $V'(x,y) = 1 - V(x,y)$ הוא פולינומי בבירור וגם הופך את תשובת המוודא המקורי.

משפט 107. $coNP \ne \overline{NP}$.
הוכחה: תהי $S \in P$. אם כן ידוע כי $P \subseteq NP \cap coNP$. לכן $S \in NP$ וגם $S \in coNP$. כיוון ש$S \in NP$ אזי $S \notin \overline{NP}$ ולכן מצאנו בעיה $S \in coNP \land S \notin \overline{NP}$ ולכן אין שוויון.

7 הרצאה 7

השאלה המתבקשת היא מה יש מעבר לNP ול$coNP$? יש בעיות שנראות מאוד קרובות לNP ול$coNP$ אך (כנראה) לא נמצאות לא בNP ולא ב$coNP$. על זה נענה בהרצאה זו. נזכר כי ראינו כבר כי NP סגורה לרדוקציית קארפ:

משפט 108. NP סגורה לרדוקציית קארפ. כלומר אם $S \in NP$ וכן $S' \le_m^P S$ אזי $S' \in NP$.

נרצה לשים לב כי אין בהכרח סגירות (עקרונית) לרדוקציית קוק, זה פחות טריוואלי. כלומר אם $S \in NP$ וקיימת רדוקציית קוק $S' \le_T S$ (שואלים שאלות את S) זה לא גורר כי $S' \in NP$ (תחת ההנחה כי $NP \ne coNP$). שכן קשה לבנות מוודא כללי לS' על סמך גישות האורקל (שמשיב תשובה בינארית).
לכן ננסח את הטענה הבאה:

משפט 109. בהנחה כי $NP \ne coNP$ אזי NP לא סגורה לרדוקציית קוק.

הוכחה: נניח בשלילה כי NP כן סגורה לרדוקציית קוק. אזי, נרצה להוכיח כי במקרה זה $coNP \subseteq NP$ (נזכר כי זה גורר שמקבלים גם את הכיוון השני של ההכלה) ואז נקבל סתירה לכך ש$NP \ne coNP$. אם כן, תהי בעיה $S \in coNP$. אזי, $\overline{S} \in NP$. אם כן, בהכרח לכל בעיה בפרט לS יש רדוקציית קוק למשלים. כלומר $S \le_T \overline{S}$ (הרי תמיד ניתן לשאול את האורקל של הבעיה המשלימה ולהחזיר תשובה הפוכה למה שאמר).
לכן כיוון ש$\overline{S} \in NP$ והעובדה כי NP כן סגורה (מההנחה בשלילה) לרדוקציית קוק אזי $S \in NP$ וקיבלנו סה"כ $coNP \subseteq NP$. כעת נראה כיצד זה גורר את הכיוון השני: תהי בעיה $S \in NP$. אזי $\overline{S} \in coNP$. אך $coNP \subseteq NP$ ולכן $\overline{S} \in NP$. כלומר, $S \in coNP$. ולכן $NP \subseteq coNP$.
סה"כ קיבלנו $NP = coNP$ בסתירה. לכן, NP לא סגורה לרדוקציית קוק.

■

èôùî 110. לכל בעיה S קיימת רדוקציית קוק $S \le_T \overline{S}$.

אם כן, אין סגירות (כנראה) לרדוקציית קוק. אבל אם יש בעיה $S' \le_T S$ כאשר $S \in NP$, מה זה אומר על S'? ניתן לחסום את הקושי שלה - שכן ניתן לפתור אותה באמצעות אורקל. לכן, היא לא "מאוד מאוד קשה". נמשיך להעמיק את האינטואיציה והמוטיבציה להרצאה. נתבונן על הבעיות הבאות:

1. $Clique = \{(G,k) | \text{G graph, G has clique of size} = k\} \in NP$
2. $\overline{Clique} \in coNP$
3. $maxClique = \{(G,k) |$ האם הקליקה המקסימלית בגרף היא בגודל $k\}$

הבעיה הראשונה בNP, והשנייה ב$coNP$. מה עם הבעיה השלישית? הרי קל להשתכנע אם יש קליקה בגודל k, ניתן להביא את הקליקה. גם קל להשתכנע שאין קליקה בגודל k. כיצד משתכנעים (אם בכלל) שהקליקה המקסימלית בגרף בגודל k? הבעיה השלישית מורכבת משילוב זוג הבעיות 1 ו2. לכן נתחיל לדון בהיררכיה הפולינומית:

7.1 ההירככיה הפולינומית

הגדרה 111. עבור $k \in \mathbb{N}$ נאמר כי בעיית הכרעה $S \in \Sigma_k$ אם קיים פולינום P ואלגוריתם פולינומי V כך ש:

$$x \in S \iff \exists y_1 \forall y_2 \exists y_3 \forall y_4, ...Q_k y_k, \forall 1 \le i \le k : |y_i| \le P(|x|) : V(x, y_1, y_2, ..., y_k) = 1$$

כך ש:

$$Q_k = \begin{cases} \forall & k \equiv 0(mod2) \\ \exists & k \equiv 1(mod1) \end{cases}$$

משפט 112. נגדיר את בעיית ההכרעה הבאה:
$\{$ האם הקליקה המקסימלית ב-G היא מגודל k $| \ (G, k) \ \}$ $= maxClique$. אזי, $maxClique \in \Sigma_2$
הוכחה: נרצה להגדיר מוודא. המוודא יקבל כקלט $x = (G, k)$ ויקבל את y_1 ו-y_2. נגדיר את $y_1 = S_1 \subseteq V$ וכן $y_2 = S_2 \subseteq V$.
V יחזיר 1 אמ"מ: S_1 קליקה בגודל k, וכן S_2 אינה קליקה בגודל $k + 1$.
נבחין כי אכן המוודא הנ"ל פולינומי, שכן ניתן לבדוק קליקה/ אי קליקה בזמן $O(|S_1|^2 + |S_2|^2)$ ובפרט פולינומי. נראה את נכונותו:
נניח כי אכן $(G, k) \in maxClique$. אזי **קיימת** קבוצת קודקודים $S = S_1$ שמהווה קליקה בגודל מקסימלי, לכן: לכל קבוצה בגודל $k<$, כלומר **לכל** קבוצה בפרט בגודל $k + 1$ היא אינה קליקה. אכן המוודא ישיב 1 במקרה זה לפי הגדרה.
בכיוון שני, נניח כי $(G, k) \notin maxClique$ אזי או שהקליקה המקסימלית היא לא מגודל k, ומגודל קטן מ-k ואז ניפול כבר ב"קיים". או שהיא מגודל גדול מ-k ואז ניפול כבר ב"לכל". סה"כ אכן $maxClique \in \Sigma_2$.

■

נבחין כי הבעיה $maxClique$ אופיינה ע"י זוג כמתים ־ קיים ולכל. קיימת קבוצה, ולכל קבוצה גדולה ממנה: לא קליקה. הבעיה $Clique$ אופיינה ע"י כמת יחיד ־ קיים. הבעיה $\overline{Clique}$ אופיינה ע"י כמת יחיד ־ לכל. ישנן בעיות שמאופיינות ע"י יותר כמתים כמובן. לכן נצטרך להשתמש בהירככיה הפולינומית.

משפט 113. כל ההבאים מתקיימים:
1. $\Sigma_0 = P$
2. $\Sigma_1 = NP$
3. לכל k טבעי, $\Sigma_k \subseteq \Sigma_{k+1}$

הוכחה:
1. לפי הגדרה, הרי שמתקיים כי קיים אלגוריתם פולינומי V ופולינום $p(x)$ כך ש $V(x) = 1 \iff x \in S$ (ללא הכמתים). זו הגדרה שקולה לזו של P.
2. לפי הגדרה, קיים אלגו' פולינומי V ופולינום $p(x)$ כך ש$x \in S \iff \exists y_1 : |y_1| \le p(|x|) : V(x, y_1) = 1$. זו הגדרה של NP.
3. הרי ש(בצורה לא פורמלית) אם קיים מוודא V לבעיה $S \in \Sigma_k$ אזי ניתן להגדיר מוודא נוסף עם ערך y_{k+1} שיתעלם מהערך האחרון והכמת האחרון.

■

הגדרה 114. נגדיר את ההירככיה הפולינומית (Polynomial Hierarchy) באופן הבא: $PH = \bigcup_{k=0}^{\infty} \Sigma_k$

הגדרה 115. נגדיר לכל $k \in \mathbb{N}$ את $\Pi_k = co\Sigma_k = \{S | \overline{S} \in \Sigma_k\}$. באופן דומה ניתן להגדירה:
עבור $k \in \mathbb{N}$ נאמר כי בעיית הכרעה $S \in \Pi_k$ אם קיים פולינום P ואלגוריתם פולינומי V כך ש:

$$x \in S \iff \forall y_1 \exists y_2 \forall y_3 \exists y_4, ...Q_k y_k, \forall 1 \le i \le k : |y_i| \le P(|x|) : V(x, y_1, y_2, ..., y_k) = 1$$

כך ש:

$$Q_k = \begin{cases} \exists & k \equiv 0(mod2) \\ \forall & k \equiv 1(mod1) \end{cases}$$

משפט 116. מתקיים כי:
1. $\Pi_0 = P$ (נזכר כי P סגורה למשלים).
2. $\Pi_1 = coNP$

הערה 117. עבור MaxClique אין תלות בסדר הכמתים: קיימת קליקה בגודל k וגם לכל קבוצה מגודל גדול מk היא לא קליקה = לכל קבוצה מגודל k היא לא קליקה וגם קיימת קליקה בגודל k. שכן אין תלות בסדר הכמתים, יש בניהם וגם שהוא קומוטטיבי. ולכן בפרט $maxClique \in \Pi_2$ גם כן. עם זאת, לא כל הבעיות מקיימות קומוטטיביות על סדר הכמתים. למשל:

דוגמה 118. נגדיר את הבעיה הבאה:
$minExpr = \{ \phi \mid \phi$ נוסחה בוליאנית מינימלית $\}$. כאשר ϕ היא מינימלית אם לא קיימת ϕ' קצרה יותר (במס' הליטרלים שמופיעים בה) עם אותה טבלת אמת.
אזי $minExpr \in \Pi_2$ (לא ידוע אם $minExpr \in \Sigma_2$)
הוכחה: נרצה להתאימה למבנה של לכל ואז קיים. נבחין כי אם ננסה ללכת בכיוון של: לכל נוסחה ארוכה יותר, אז קיים מה? קשה לנו לקבוע. לכן נרצה ללכת בכיוון של: לכל נוסחה קצרה יותר, קיימת השמה שמספקת רק אחת מן הנוסחאות. פורמלית:
$\phi \in MinExpr \iff$ לכל נוסחה ϕ' קצרה יותר קיימת השמה a כך ש$\phi'(a) \neq \phi(a)$. מכאן נגדיר מוודא V שמקבל כקלט נוסחה ϕ, נוסחה נוספת $\phi' = y_1$ והשמה $a = y_2$ ומחזיר 1 אמ"מ $|\phi'| \geq |\phi|$ או $\phi(a) \neq \phi(a')$. V פולינומי ומתקיים:
$\phi \in MinExpr \iff \forall \phi' \exists a : V(\phi, \phi', a) = 1$.
נבחין כי a תלוי בϕ' ולכן לא ניתן להפוך כאן את סדר הכמתים. בפרט לא ידוע כיום אם $minExpr \in \Sigma_2$.

■

èôùî 119. לכל $k \in \mathbb{N}$ מתקיים $\Pi_k \subseteq \Pi_{k+1}$.

משפט 120. לכל $k \in \mathbb{N}$ מתקיים $\Sigma_k \subseteq \Pi_{k+1}$
הוכחה: בצורה לא פורמלית, אם ניתן לאפיין בk כמתים כאשר הראשון הוא קיים, ניתן לאפיין ב$k+1$ כמתים כאשר הראשון הוא לכל ונתעלם ממנו. פורמלית יותר, נוסיף ערך פקטיבי y_0 למען כמת לכל הראשון, ונדאג כי $V_2(x, y_0, y_1, ..., y_k) = 1 \iff V(x, y_1, y_2, ..., y_k) = 1$.

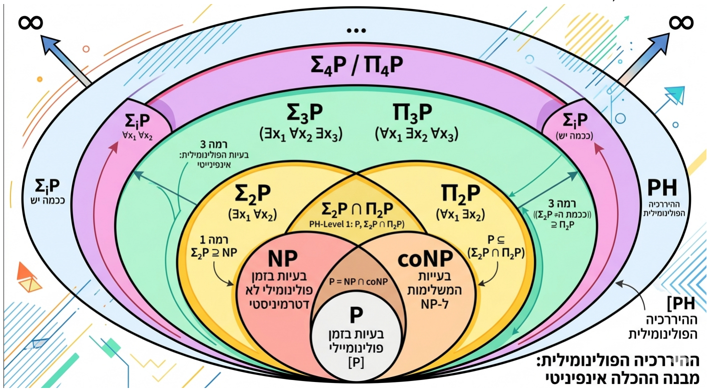
■

משפט 121. לכל $k \in \mathbb{N}$ מתקיים $\Pi_k \subseteq \Sigma_{k+1}$

משפט 122. (ההשערה הרווחת): לכל $k \in \mathbb{N} \backslash \{0\}$ מתקיים $\Sigma_k \neq \Pi_k$.

מסקנה 123. מתקיים $PH = \bigcup_{k=0}^{\infty} \Sigma_k = \bigcup_{k=0}^{\infty} \Pi_k$

הערה 124. כך נראית ההיררכיה תחת **ההשערה הרווחת** - כל רמה בהיררכיה מכילה את כל קודמיה.



7.2 תחת ההנחה $P = NP$: $PH = P$

כעת נעבור להוכיח כמה וכמה טענות עזר, כאשר מטרתן יהיה להוכיח כי אם $P = NP$ אזי כל ההיררכיה הפולינומית "קורסת" ונקבל $PH = H$. זה כמובן חיזוק לכך שאכן $P \neq NP$ (אך שוב, לא יודעים להוכיח זאת).

משפט 125. (טענה 1) לכל $S \in \Sigma_{k+1}$ קיים פולינום p ובעיה $S' \in \Pi_k$ כך ש: $x \in S \iff \exists y : |y| \leq p(|x|) : (x, y) \in S'$
הוכחה: נבחין שהטענה די טריוואלית, למעשה לא פורמלית היא טוענת כי אם ניתן לאפיין ב$k+1$ כמתים כאשר מתחילים בקיים, ניתן להתחיל בלכל ולהעזר בk כמתים אם מכניסים את הכמת הראשון (y) לבעיה, בבירור שהטענה כמעט טריוואלית אך נוכיחה באופן פורמלי שכן היא מקשרת לנו בין שתי רמות בהיררכיה:
תהי $S \in \Sigma_{k+1}$. לכן קיים פולינום p ואלגו' פולינומי V כך ש: $x \in S \iff \exists y_1 \forall y_2 \exists y_3 ....Q_{k+1} y_{k+1} : \forall 1 \leq i \leq k : |y_i| \leq p(|x|)$ וכן:

$$V(x, y_1, ..., y_{k+1}) = 1$$

נגדיר את S' באופן הבא:

$$S' = \{(x, y) | y \leq p(|x|)\} \wedge \{\forall y_2 \exists y_3 .....Q_{k+1} y_{k+1} : V(x, y, y_2, ..., y_{k+1})\}$$

נבחין כי מתקיים $S' \in \Pi_k$ בבירור לפי הגדרה (ישנם k כמתים שמתחילים בלכל). וכן מתקיים:

$$x \in S \iff \forall 1 \leq i \leq k : |y_i| \leq p(|x|) \wedge V(x, y_1, ..., y_{k+1}) = 1 \iff$$

$$\exists y : |y| \leq p(|x|) : (x, y) \in S'$$

כנדרש.

■

משפט 126. (טענה 2) עבור $k \geq 1$, אם $\Pi_k \subseteq \Sigma_k$ אזי $\Sigma_k = \Sigma_{k+1}$

הוכחה: אנחנו יודעים כי תמיד $\Sigma_k \subseteq \Sigma_{k+1}$. לכן נרצה להוכיח את ההכלה בכיוון השני.
תהי בעיה $S \in \Sigma_{k+1}$. אם כן, לפי טענה 1 הרי שקיים פולינום P ובעיה $S' \in \Pi_k$ כך ש: $x \in S \iff \exists y : |y| \leq p(|x|) : (x, y) \in S'$. אם כן, $\Pi_k \subseteq \Sigma_k$ ולכן $S' \in \Sigma_k$. כלומר, קיים פולינום P' ואלגוריתם פולינומי V' כך ש:

$$x \in S' \iff \exists y_1 \forall y_2 .....Q_k y_k, \forall 1 \leq i \leq k : |y_i| \leq p'(|x|) \wedge V'(x, y_1, ..., y_k) = 1$$

אם כן נחזור לבעיה שלנו, $(x, y) \in S'$. וזה פותח בכמת "קיים". אך גם S' פותחת בכמת קיים ־ אך תמיד ניתן לשרשר זוג כמתי קיים לכמת יחיד. כפי שנעשה כעת:

$$x \in S \iff \exists y : |y| \leq p(|x|) : (x, y) \in S' \iff \exists y, \exists y_1 \forall y_2 .....Q_k y_k, \forall 1 \leq i \leq k : |y_i| \leq p(|x|) \wedge y \leq p(|x|) :$$

$$V'(x, y_1, ..., y_k) = 1$$

לכן נוכל להגדיר $y_0 = (y, y_1)$ ולקבל:

$$\iff \exists y_0 \forall y_2 .....Q_k y_k, \forall 0 \leq i \neq 1 \leq k : |y_i| \leq p(|x|) \wedge y \leq p'(|x|) : V'(x, y_0, y_2, ..., y_k) = 1$$

נוכל לקחת פולינום $P'' = P(x) + P'(x)$ שיחסום את הגודל של $\{y, y_2, ..., y_k\}$. ולכן סה"כ הראינו מוודא עם k כמתים שפותחים בקיים, ולכן $S \in \Sigma_k$. סה"כ מתקיים $\Sigma_k = \Sigma_{k+1}$.

■

מסקנה 127. אם $NP = coNP$ אזי $NP = \Sigma_2$ (שכן זה גורר כי $coNP = \Pi_1 \subseteq \Sigma_1 = NP$ ולכן לפי הטענה הקודמת $\Sigma_1 = \Sigma_2$ כלומר $NP = \Sigma_2$).

משפט 128. (טענה 3) עבור $k \geq 0$. אם $\Sigma_k = \Sigma_{k+1}$ אזי $PH = \Sigma_k$ (כל ההירככיה קורסת).

הוכחה: נקבל גם כי $\Pi_k = \Pi_{k+1}$ (שהרי $S \in \Pi_k \iff \overline{S} \in \Sigma_{k+1} = \Sigma_k \iff S \in \Pi_{k+1}$).
לכן $\Pi_{k+1} = \Pi_k \subseteq \Sigma_{k+1}$ כלומר קיבלנו כי $\Pi_{k+1} \subseteq \Sigma_{k+1}$. לכן, $\Sigma_{k+1} = \Sigma_{k+2}$ לפי הטענה הקודמת.
באופן דומה באינדוקציה, ניתן להכליל את השורה הקודמת לכל k ולקבל כי $... = \Sigma_{k+2} = \Sigma_{k+1} = \Sigma_k$. ולכן סה"כ

$$PH = \bigcup_{k=0}^{\infty} \Sigma_k = \bigcup_{i=0}^{k} \Sigma_i = \Sigma_k$$

שהרי המעבר האחרון מיפין נכון כי $\Sigma_0 \subseteq \Sigma_1 \subseteq ... \subseteq \Sigma_k$.

■

משפט 129. אם $P = NP$ אזי $PH = P$

הוכחה: לפי הטענה הקודמת שהרי $\Sigma_0 = P, \Sigma_1 = NP$ נקבל כי $\Sigma_0 = \Sigma_1$ ולכן לפי הטענה הקודמת $PH = \Sigma_0 = P$. כלומר, כל ההירככיה קורסת.

■

7.3 תרגול 7

משפט 130. כל יחס $R \notin PF$ כך ש$S_R \in P$, אזי אין רדוקציה עצמית.

הוכחה: נניח בשלילה כי ישנה רדוקצייה עצמית לR^{S_R}. אזי, ניתן לפתור את R בזמן פולינומי באמצעות גישות אורקל ל$S_R \in P$, נקבל אלגוריתם שמכריע את R ולכן $R \in PF$ בסתירה.

משפט 131. אם $P \neq NP \cap coNP$ אזי קיימת בעיית חיפוש $R \in PC$ כך שאין לה רדוקציה עצמית.

משפט 132. נגדיר: $MIN-VC = \{(G,k) \mid k$ הוא מגודל Gכיסוי הקודקודים המינימלי ב$\}$

אזי, $MIN-VC \in \Sigma_2$.

משפט 133. קיימת בעיה Σ_2-שלמה.

הוכחה: נגדיר את הבעיה הבאה -

$S = \{\ (V,x,1^p,1^t)\ |\exists_{y_1:|y_1|\leq p}\forall_{y_2,|y_2|\leq p} : V(x,y_1,y_2)=1$ תוך לכל היותר t צעדים $\}$

ראשית, נבחין כי $S \in \Sigma_2$ שכן נגדיר את האלגוריתם:

$V_S = (x' = (V,x,1^p,1^t),y_1,y_2)$: הרץ את $V(x,y_1,y_2)$ לכל היותר t צעדים. אם קיבלת 1 החזר 1 ואחרת 0.

בבירור שV_S פולינומי שכן מבצע $|x| = |V|+p+t$ פעולות. וסה"כ $S \in \Sigma_2$.

כעת נראה כי לכל $S' \leq_m^P S : S' \in \Sigma_2$. תהי $S' \in \Sigma_2$. לכן קיים לה מוודא V ופולינום p כך שלכל x:

$$x \in S' \iff \exists y_1, |y_1| \leq p(|x|)\forall y_2, |y_2| \leq p(|x|) : V(x,y_1,y_2) = 1$$

נגדיר את הרדוקציה באופן הבא: $f(x) = (V,x,1^{p(|x|)},1^{t(|x|)})$ כאשר V פולינומי ולכן זמן ריצתו חסום ע"י $t(|x|)$. הרדוקציה פולינומית שכן $p(|x|),t(|x|)$ פולינומיים וV אלגו' קבוע דטרמיניסטי. כעת נראה את נכונות הרדוקציה -

$$x \in S' \iff \exists y_1, |y_1| \leq p(|x|)\forall y_2, |y_2| \leq p(|x|) : V(x,y_1,y_2) = 1 \iff$$

אמ"מ $\exists y_1 : |y_1| \leq p, \forall y_2 : |y_2| \leq p : V(x,y_1,y_2) = 1$ תוך לכל היותר t צעדים $\iff f(x) = (V,x,1^{p(|x|)},1^{t(|x|)}) \in S$

ולכן סה"כ קיבלנו כי $S \in \Sigma_2$, כנדרש. ■

8 הרצאה 8

8.1 חישוב לא דטרמיניסטי

הגדרה 134. אלגוריתם לא דטרמיניסטי הינו אלגוריתם שבכל צעד יתכנו כמה אפשרויות שונות. נאמר כי אלגוריתם לא דטרמיניסטי A מכריע בעיה S אם מתקיים כי:

$x \in S$ אזי קיימת הרצה של A כך ש$A(x) = 1$

$x \notin S$ אזי לכל הרצה של A יתקיים $A(x) = 0$

הגדרה 135. זמן הריצה של אלגוריתם לא דטרמיניסטי A על קלט x מוגדר להיות מס' הצעדים המקסימלי שA מבצע בריצה כלשהי על x. נסמן זאת ב$t_A(x)$.

נאמר כי A הוא אלגוריתם **פולינומי לא דטרמיניסטי** אם קיים פולינום p כך שלכל x: $t_A(x) \leq p(|x|)$.

הגדרה 136. (הגדרה שקולה למחלקה NP):

$NP = \{\ S \mid S$ קיים אלגוריתם לא דטרמיניסטי פולינומי שמכריע את $\}$

הוכחה (מהתרגול): נוכיח שהגדרה זו שקולה. (נסמן מחלקה של הגדרה זו בNP') ונוכיח $NP = NP'$.

בכיוון ראשון, נניח כי $S \in NP$. אזי, קיים מוודא V ופולינום p כך ש:

$$x \in S \iff \exists_{y,|y|\leq p(|x|)} : V(x,y) = 1$$

נבנה אלגוריתם לא דטרמיניסטי A - האלגוריתם יבחר באופן לא דטרמיניסטי y כלשהו המקיים $|y| \leq p(|x|)$ ויחזיר את $V(x,y)$.

קל לראות שהאלגוריתם הנ"ל פולינומי שכן בוחר בזמן פולינומי ומריץ אלגו' פולינומי. ונבחין כי:

אם $x \in S$ אזי $\exists_{y,|y|\leq p(|x|)} : V(x,y) = 1$ כלומר עבור אותו הy קיימת הרצה של A כך ש$A(x) = 1$.

אם $x \notin S$ אזי מההגדרה $\forall_{y,|y|\leq p(|x|)} : V(x,y) = 0$ ולכן לכל בחירה של y כנ"ל A ישיב אפס. לכן סה"כ $S \in NP'$.

בכיוון שני, נניח כי $S \in NP'$. אזי, קיים אלגוריתם לא דטרמיניסטי פולינומי A ויהי $p(n)$ הפולינום שחוסם את זמן הריצה של A.

המוודא $V(x,y)$ יוגדר באופן הבא - המוודא יריץ את $A(x)$ תוך שימוש בעדות y בתור סרט הבחירות הלא דטרמיניסטיות שהוא מבצע. (כלומר y זה סדר הבחירות הלא דטרמיניסטיות). בבירור שפולינומי, וסה"כ $NP = NP'$. ■

הגדרה 137. תהי בעיית הכרעה S.
א. נגדיר את המחלקה P^S להיות מחלקת כל הבעיות הניתנות להכרעה בעזרת אלגוריתם דטרמיניסטי פולינומי עם גישת אורקל לS.
ב. נגדיר את המחלקה NP^S כל הבעיות הניתנות להכרעה בעזרת אלגוריתם לא דטרמיניסטי פולינומי עם גישת אורקל לS.

הגדרה 138. עבור מחלקה של בעיות C נגדיר את $P^C = \bigcup_{S\in C} P^S$ ובדומה: $NP^C = \bigcup_{S\in C} NP^S$

8.2 הגדרה שקולה להירככיה הפולינומית

הגדרה 139. (הגדרה שקולה להירככיה הפולינומית). נגדיר:

$$\Sigma_0 = P, \Sigma_1 = NP, \forall k \geq 1 : \Sigma_{k+1} = NP^{\Sigma_k}$$

משפט 140. שתי ההגדרות של ההירככיה הפולינומית שקולות.

הוכחה: נראה את שני הכיוונים של ההכלה לכל $k \geq 1$: $\Sigma_{k+1} = NP^{\Sigma_k}$.

תהי $S \in \Sigma_{k+1}$ לפי ההגדרה המקורית. לפי טענה שראינו, זה גורר כי קיים פולינום P ובעיה $S' \in \Pi_k$ כך ש: $x \in S \iff \exists y : |y| \leq p(|x|) :$ $(x,y) \in S'$. אם כן, ניתן להגדיר אלגוריתם לא דטרמיניסטי שבהינתן קלט x בוחר ערך y באופן לא דטרמיניסטי (ניחוש) המקיים $|y| \leq p(|x|)$ ושואל את האורקל אם $(x,y) \in S'$. לכן, $S \in NP^{\Pi_k}$ (שכן $S' \in \Pi_k$). אך נשים לב כי $NP^{\Pi_k} = NP^{\Sigma_k}$ (שכן אם האורקל יכול לעזור עם בעיה $S' \in \Pi_k$ הוא יכול לעזור בצורה המשלימה ־ כאשר הופך את תשובות האורקל, והרי נקבל שהמשלימה היא בΣ_k). לכן סה"כ קיבלנו כי $S \in NP^{\Sigma_k} = \Sigma_{k+1}$ (לפי איך שהגדרנו), ולכן הכיוון הראשון הוכח.

בכיוון השני, תהי $S \in NP^{\Sigma_k}$ לפי ההגדרה החדשה. נרצה להוכיח כי $S \in \Sigma_{k+1}$ לפי ההגדרה המקורית. אם כן לפי הגדרה קיימת $S' \in \Sigma_k$ וקיים אלגוריתם לי"ד פולינומי $A^{S'}$ שמכריע את S (עם גישות אורקל לS').

נניח בה"כ שמהלך האלגוריתם מחולק לשני שלבים נפרדים ־

1. בכל פעם ש$A^{S'}$ רוצה לשאול את האורקל האם $q_i \in S'$, אז הוא בוחר באופן לא דטרמיניסטי תשובה $q_i \in \{0,1\}$ בלי לשאול את האורקל! וממשיך לפי תשובה זו.
2. לכל שאלה שהייתה במהלך הריצה, הוא שואל את האורקל האם $q_i \in S'$, ובודק שהתשובה שניחש זהה לתשובת האורקל.

נבחין שזה אכן בלי הגבלת הכלליות ־ שכן ניתן לשנות את האלגוריתם לפעול בצורה זו שקולה, והוא עדיין פולינומי. לבסוף: $A^{S'}$ מחזיר 1 אמ"מ כל התשובות היו נכונות וגם הוא הגיע למצב שמחזיר 1.

כלומר מתקיים כי:

$$x \in S \iff \exists y \exists \vec{a} : \left[A^{S'}(x) = 1 \land (q_1 \in S' \iff a_1 = 1) \land ...... \land (q_t \in S' \iff a_t = 1)\right]$$

באשר y הוא סדרת הבחירות הלא דטרמיניסטיות של האלגוריתם, וa הוא סדרת התשובות שניחש האלגוריתם וישנן t שאלות כאשר t קטן מפולינום שחוסם את זמן הריצה של $A^{S'}$. אם כן, מכאן יש לפתוח את התנאי הנ"ל ולקבל את התנאי שמתאים להגדרה שראינו:

$$x \in S \iff \exists y \exists \vec{a} \left[A^{S'}(x) = 1 \land ((q_1 \in S \land a_1 = 1) \lor (q_1 \notin S \land a_1 = 0)) \land .... \land ((q_t \in S \land a_t = 1) \lor (q_t \notin S \land a_t = 0)\right]$$

$$x \in S \iff \exists y \exists \vec{a} \left[A^{S'}(x) = 1 \land ((q_1 \in S' \land a_1 = 1) \lor (q_1 \notin S' \land a_1 = 0)) \land .... \land ((q_t \in S' \land a_t = 1) \lor (q_t \notin S' \land a_t = 0)\right]$$

נראה כי התנאי $q_1 \in S'$ כיוון שS' בΣ_k שקול ל: $\exists y_1 \forall y_2 .....Q_k y_k : V(q_1, y_1, ..., y_k) = 1$ והתנאי $q_1 \notin S'$ שקול ל: $\forall y_1' \exists y_2 ....Q_k y_k'$ $V'(q_1, y_1', ..., y_k') = 1$.

יש לשים לב שהשאלות $q_i \in S?$ או $q_j \in S?$ לא תלויות זו בזו. ולכן, ניתן להוציא את השאלות החוצה מהסוגריים וכן גם בסדר שבו מופיעים. לכן נקבל:

$$x \in S \iff \exists y \exists \vec{a} \exists y_1^1 \exists y_2^1 ....\exists y_k^1 \forall y_1^2 \forall y_2^2 \forall ...\forall y_k^2 .... : [A^{S'}(x) = 1 \land V'(x, y_1^1, .., y_t^1) = 1 \land ..... \land V'(x, y_1^t, ..., y_k^t) = 1]$$

מכאן ניתן לשים לב כי אם נחבר את הכמתים ונסדר את המוודאים למוודא יחיד (שניתן לאחדם לאלגוריתם פולינומי אחד) נקבל כי שייכות לS של קלט x מסויים מאופיינת באמצעות $k+1$ כמתים באשר הראשון הוא $\exists$ וניתן לבדוק את התנאים בתוך [] בזמן פולינומי. לכן סה"כ $S \in \Sigma_{k+1}$ לפי ההגדרה המקורית.

■

ñî. ä141÷מסקנה מהמשפט הקודם: אם ישנה רדוקציית קוק מS' לS וכן $S \in NP$. אזי $S' \in P^S \subseteq NP^S \subseteq NP^{NP} = \Sigma_2$ (שכן זה גורר כי אפשר לפתור את S' באמצעות גישת אורקל לבעיה S והרי ש$P^S \subseteq NP^S$ שכן כל אלגוריתם דטרמיניסטי הוא בפרט גם נכלל כלא דטרמיניסטי). ולכן: PH סגורה לרדוקציית קוק (לא בין הרמות בהכרח, כן בתוך כל ההירככיה).

8.3 חישוב לא יוניפורמי

לפני שנדבר מהו חישוב לא יוניפורמי, נרצה לדון מהו חישוב יוניפורמי: קיים אלגוריתם אחד לכל הקלטים שיודע להכריע את הבעיה. מכאן, נוכל להגדיר:

הגדרה 142. חישוב לא יוניפורמי הוא תהליך חישוב שונה לקלטים באורכים שונים.

8.3.1 מודל המעגלים

נזכר בהגדרת המעגל הבוליאני:

הגדרה 143. מעגל בוליאני הינו גרף מכוון ללא מעגלים (DAG) שקודקודיו מתחלקים ל3 סוגים:
א. קודקודי קלט ־ קודקודים $v \in V$ המקיימים $deg_{in}(v) = 0$ (ללא קשתות נכנסות) וכן $deg_{out}(v) \geq 1$ (לפחות קשת יוצאת אחת).
ב. קוקודי פלט ־ קודקודים $v \in V$ המקיימים $deg_{out}(v) = 0$ (ללא קשתות יוצאות) וכן $deg_{in}(v) = 1$ (יש קשת נכנסת אחת).
ג. שערים: קודקודים $v \in V$ המקיימים $2 \geq deg_{in}(v) \geq 1$ (בין 1 ל2 קשתות נכנסות) וכן $deg_{out}(v) \geq 1$.

בדומה ל$CSAT$, אנחנו נניח כי ישנו קודקוד פלט יחיד. נבחין כי מעגל מסויים יכול לטפל בקלט בגודל מסויים ספציפי ־ ולא כל גודל! כלומר כל מעגל יכול לחשב רק קלטים באורך של מס' קודקודי הקלט.

הגדרה 144. נאמר כי סדרת מעגלים $\{C_n\}_{n=1}^{\infty}$ (סדרה אינסופית) מכריעה בעיית הכרעה S אם לכל x מתקיים כי:

$$x \in S \Longleftrightarrow C_{|x|}(x) = 1$$

נרצה לחסום את הקושי (האורך) של כל מעגל.

הגדרה 145. נמדוד את סיבוכיות מעגל C במספר הקשתות של המעגל ונסמן זאת ב$|C|$.

הגדרה 146. נאמר כי S מוכרעת ע"י סדרת מעגלים פולינומית אם קיימת סדרת מעגלים $\{C_n\}_{n=1}^{\infty}$ שמכריעה את S וכן קיים פולינום p כך שלכל n: $|C_n| \leq p(n)$. נבחין, כי מס' המעגלים לא חסום.

8.4 תרגול 8

הערה 147. מעתה, נניח כי לאלגוריתם לא דטרמיניסטי בלי הגבלת הכלליות ישנן 2 אפשרויות לבחירה בכל שלב.

תרגיל 148. הראו אלגוריתם לא דטרמיניסטי שמכריע את SSS

פתרון:

נגדיר אלגוריתם $M(A,b)$:
1. בחר בצורה לא דטרמיניסטית $|A|$ ביטים של 0 או 1 ־ ביט לכל איבר, ושמור במערך B.
2. הגדר $s = 0$
3. לכל $1 \leq i \leq n$: אם $B[i] = 1$ בצע $s = s + B[i]$
4. החזר 1 אמ"מ $s = b$

נבחין כי האלגוריתם הנ"ל בהכרח פולינומי, שכן הבחירה מתבצעת בזמן פולינומי וכך המעבר הלינארי. נוכיח נכונות:
אם $(A,b) \in SSS$ אזי קיימת תת קבוצה כנ"ל של איברים $S' \subsetneqq S$ כך שעבור בחירה בשלב 1 של האינדקס שלהם להיות 1 (ושל השאר אפס) נקבל בדיוק s.
אם $(A,b) \notin SSS$ נב"ש כי $M(A,b) = 1$. אזי, קיימת הרצה כלשהי של האלגוריתם עבורה האלגוריתם השיב 1. זה גורר כי האלגוריתם מצא קבוצה של איברים שמוכלת בS (אלו שהוא בחר להם ביט 1) וסכומם בדיוק b. כלומר, $(A,b) \in SSS$ בסתירה.

משפט 149. לכל מחלקה $\mathcal{C}$ של בעיות הכרעה מתקיים $\mathcal{C} \subseteq P^{\mathcal{C}} \subseteq NP^{\mathcal{C}}$

משפט 150. אם $\Pi_1 \subseteq \Sigma_1$ אזי $\Sigma_1 = \Sigma_2$.

הוכחה: נוכיח את הטענה שוב, על אף שראינו את הוכחתה בהרצאה. הפעם בצורה שונה. הרי שידוע כי $\Sigma_1 \subseteq \Sigma_2$. לכן נצטרך להוכיח כי $\Sigma_2 \subseteq \Sigma_1$.

תהי $S \in \Sigma_2$. הרי שידוע לפי הגדרה שקולה להירככיה כי $\Sigma_2 = NP^{NP}$. כלומר $S \in NP^{NP}$. דהיינו, קיימת בעיה S' וקיים אלגוריתם $A^{S'}$ לא דטרמיניסטי פולינומי שמכריע את S באמצעות גישות אורקל לS'. הרי ש$S' \in NP$ ולכן $\overline{S'} \in coNP = \Pi_1 \subseteq \Sigma_1 = NP$ ולכן $\overline{S'} \in NP$. לכן סה"כ גם S' וגם משלימתה נמצאות בNP לכן קיימים אלגוריתמים לי"ד $A_{S'}, A_{\overline{S'}}$ שמכריעים אותן בהתאמה. כעת נבנה אלגוריתם לי"ד A_S שיכריע את S.

A_S יריץ את $A^{S'}$: כל קריאה לאורקל של S' עם שאלה הוא יחליף עם ההרצות של $A_{S'}, A_{\overline{S'}}$. אם $A_{S'}(q) = 1$ אזי הוא ימשיך להריץ את האלגוריתם כמו שקיבל תשובה חיובית מהאורקל. אם $A_{\overline{S'}}(q) = 1$ הוא ימשיך להריץ את האלגוריתם כאילו קיבל תשובה שלילית מהאורקל. אחרת, הוא קיבל בשתיהן 0 ויפסיק.

נבחין כי A_S אלגוריתם פולינומי לא דטרמיניסטי (הוא מריץ אלגו' פולינומי שקורא לאלגו' פולינומיים) והרי שמתקיים הדרוש (יש להוכיח זאת, נדלג כאן).

לכן סה"כ $S \in NP$ לפי ההגדרה השקולה, כלומר $S \in \Sigma_1$ ולכן הוכחנו $\Sigma_1 = \Sigma_2$, כנדרש. ■

9 הרצאה 9

9.1 מכונות טיורינג שמקבלות עצה

הגדרה 151. עבור פונקציה $\ell:\mathbb{N}\to\mathbb{N}$ נאמר כי בעיית הכרעה S שייכת למחלקה P/ℓ אם קיים אלגוריתם פולינומי A וסדרה אינסופית של מחרוזות עצה $\{a_n\}_{n=1}^{\infty}$ כך ש:

1. אורך העצה חסום ע"י ℓ. כלומר, $\forall n\in\mathbb{N}: a_n\le \ell(n)$.
2. לכל x, $A(a_{|x|},x)=1 \iff x\in S$ (כלומר, האלגוריתם מקבל את הקלט ועצה המתאימה לאורך הקלט).

הגדרה 152. המחלקה $P/Poly$ מוגדרת להיות:

$$P/Poly=\bigcup_{c=0}^{\infty} P/n^c$$

כלומר, כל הפונקציות הפולינומיות.

הערה 153. נשים לב, אנחנו מניחים שהעצה שאם קיבל האלגוריתם היא אכן נכונה. זה שונה מNP שם חיפשנו עדות נכונה ויתכנו עדויות לא נכונות. כאן, האלגוריתם A רק מריץ את אותה עצה. הקושי היא - האם קיימת עצה כנ"ל.

9.2 מכונת טיורינג עם עצה שקולה לסדרת מעגלים

משפט 154. תהי S בעיית הכרעה. אזי, $S\in P/Poly \iff$ קיימת סדרה פולינומית של מעגלים שמכריעה את S.

הוכחה:

$\Longrightarrow$ בכיוון ראשון, נניח שקיימת סדרה פולינומית של מעגלים $\{C_n\}_{n=1}^{\infty}$ שמכריעה את S. נרצה להוכיח $S\in P/Poly$.

נגדיר את סדרת העצות $\{a_n\}_{n=1}^{\infty}=\{C_n\}_{n=1}^{\infty}$. כלומר, העצה של a_n תהיה תיאור המעגל C_n.

נגדיר אלגוריתם A כך שבהינתן קלט x ועצה $a_{|x|}=C_{|x|}$ יחזיר את תוצאת $C_{|x|}(x)$.

נרצה להוכיח כי אכן כל התנאים מתקיימים. ראשית, כיוון שסדרת המעגלים היא פולינומית אזי נקבל שגם סדרת העצות חסומות ע"י פולינום p כלשהו. וכן בבירור:

$$x\in S\iff C_{|x|}(x)=1\iff A(a_{|x|},x)=1$$

לכן $S\in P/Poly$.

$\Longleftarrow$ בכיוון שני, נניח כי $S\in P/Poly$ ונרצה להוכיח שקיימת סדרה של מעגלים $\{C_n\}_{n=1}^{\infty}$ שמכריעה את S. אם כן, מההנחה קיים פולינום p ואלגוריתם פולינום A וסדרת עצות $\{a_n\}_{n=1}^{\infty}$ כך שלכל n: $|a_n|\le p(n)$ וכן לכל x: $A(a_{|x|},x)=1\iff x\in S$.

נזכר כי ראינו בהוכחה כי $CSAT\in NPC$ כי לכל מכונת טיורינג קיים מעגל בוליאני שמדמה את ריצת המכונה על קלט x.

לכן, לכל n נסתכל על המעגל C_n שמדמה את ריצת המכונה A כאשר היא מקבלת קלט באורך n ואת העצה a_n.

נבחין כי מס' קודקודי הקלט במעגל הוא $n+|a_n|$ (שכן כרגע המעגל מקבל כקלט x המקיים $|x|=n$ ועל מנת לגשת למעגל הוא צריך לקבל עצה בגודל $|a_n|$. עם זאת, אנחנו יודעים כי לכל מעגל אורך העצה תמיד זהה (עבור קלט n). לכן נסתכל על המעגל C_n^* המתקבל ע"י הצבת העצה a_n במעגל הבוליאני C_n ופישוט המעגל. כעת המעגל C_n^* מכיל n קודקודי קלט, גודלו חסום ע"י פולינום (שכן A חסום פולינומית והרי המעגל מדמה ריצתו של A על קלט באורך n). לכן, לכל x מתקיים: $C_n^*(x)=A(a_{|x|},x)$. בפרט, סדרת המעגלים הנ"ל מכריעה את S. כנדרש.

■

משפט 155. נבחין כי $P/0=P$ (העצות באורך 0).

משפט 156. המחלקה $P/1$ מכילה בעיות לא כריעות. (כלומר, אם הוספנו ביט 1 של עצה. נוכל להכריע בעיות לא כריעות).

הוכחה: רעיון ההוכחה היא להגדיר מראש בעיה שהתשובה לכל קלט באותו אורך תמיד זהה. כלומר - כל הקלטים באורך מסויים n מקיימים כי התשובה לכולם היא 1 או 0. תהי הבעיה הבאה: $S\subseteq\{1\}^*$ לא כריעה. (כלומר בעיה שמקודדת בקידוד אונארי בלבד. קיימת כזו). מדובר בקידוד אונארי של בעיה לא כריעה כלשהי. נגדיר בעיה S' מעל $\{0,1\}^*$ באופן הבא:

$$S'=\{x\ /\ 1^{|x|}\in S\}$$

כלומר, כל הקלטים x כך שמחרוזת של $|x|$ אחדות מופיעה בS. נבחין כי S' לא כריעה. שכן בשלילה שS' כן כריעה, נקבל קלט x עבור S, ונריץ את המכונה שמכריעה את S' ולכן היא מכריעה את S. אך S לא כריעה בסתירה. לכן S' לא כריעה. נבחין כי השייכות לS' תלויה אך ורק באורך של $|x|$ - שכן לכל זוג $x\neq y$ אם $|x|=|y|$ אזי שתיהן שייכות לS' אמ"מ $1^{|x|}\in S$.

עם זאת, נבחין כי $S'\in P\backslash 1$. שכן: נגדיר את סדרת העצות באופן הבא:

$$a_n=\begin{cases}1 & 1^n\in S\\ 0 & o.w\end{cases}$$

נבחין כי אכן גודל כל עצה הוא 1, והעצה למעשה אומרת עבור קלט n אם המחרוזת שמתאימה לn אחדות נמצאת בS. כעת, האלגוריתם A יפעל כך:

$A(a_{|x|}, x)$: החזר את $a_{|x|}$.

בבירור כי:

$$x \in S' \iff 1^{|x|} \in S \iff a_{|x|} = 1$$

ולכן האלגוריתם A מכריע את S'. וכן, אורך העצה הוא 1 בדיוק (ביט בודד) ולכן נקבל כי הכרענו את S'. ■

מסקנה 157. $P \subsetneq P/1$

9.3 $NP \subseteq P/Poly \implies PH = \Sigma_2$

משפט 158. אם $NP \subseteq P/Poly$ אזי $PH = \Sigma_2$ (זה לא גורר $P = NP$, אלא כל ההירככיה קורסת לרמה ה2).

הוכחה: נזכר במשפט שראינו, כי אם $\Pi_k \subseteq \Sigma_k$ אזי $\Sigma_k = \Sigma_{k+1}$ ואם $\Sigma_k = \Sigma_{k+1}$ אזי $PH = \Sigma_k$. לכן, נוכיח כי מתקיים $\Pi_2 \subseteq \Sigma_2$ ונראה שכל ההירככיה קורסת לרמה השניה $PH = \Sigma_2$.

תהי $S \in \Pi_2$. אזי, קיים אלגוריתם פולינומי V ופולינום p כך ש:

$$(*) : x \in S \iff \forall y : |y| \leq p(|x|) \exists z : |z| \leq p(|x|) : V(x, y, z) = 1$$

נרצה להוכיח כי $S \in \Sigma_2$ (כלומר להפוך את סדר הכמתים, בהינתן ההנחה שלנו). בשלב הראשון, נרצה להגדיר בעיה S':

$$S' = \{(x, y) |\ |y| \leq p(|x|) \land \exists z : |z| \leq p(|x|) \land V(x, y, z) = 1\}$$

כעת נבחין כי

$$(x, y) \in S' \iff \exists z : |z| \leq p(|x|) : V(x, y, z) = 1$$

ונטען כי $S' \in NP$. שכן, לפי הגדרה המוודא יקבל את העדות z. מההנחה שלנו, נקבל כי $S' \in NP \subseteq P/Poly$. בפרט, $S' \in NP$ ולכן קיימת רדוקציית קארפ $S' \leq_m^P SAT$. כלומר,

$$(**) : (x, y) \in S' \iff f(x, y) \in SAT.$$

מ$(*)$ ומ$(**)$ נקבל:

$$x \in S \iff (x, y) \in S' \iff \forall y : |y| \leq p(|x|) : f(x, y) \in SAT$$

בפרט, $x \in S \iff \forall y : |y| \leq p(|x|) : f(x, y) \in SAT$

מההנחה כי $NP \subseteq P/Poly$ בפרט $SAT \in P/Poly$ ולכן קיים אלגוריתם פולינומי A ופולינום q וסדרה של עצות $\{a_n\}_{n=1}^{\infty}$ כך ש: לכל n מתקיים $|a_n| \leq q(n)$ וכן לכל φ מתקיים כי:

$$\varphi \in SAT \iff A(a_{|\varphi|}, \varphi) = 1$$

הרעיון אליו נרצה להגיע הוא להעזר ב$x \in S \iff \forall y : |y| \leq p(|x|) : f(x, y) \in SAT$, ולהגיע לצורה של $x \in S$ אמ"מ **קיימת** עצה כך **שלכל**... ואז נגיע לצורה של Σ_2 ונסיים. עם זאת - ישנה בעיה פורמלית ברעיון זה שכן הקיים מגיע לפני הלכל - לכן עצה שקיימת צריכה להיות תלויה בגודל y שמגיעה אחריה. לכן העצה צריכה להכיל בתוכה מידע לכל האורכים האפשריים שיכולים להיות לy.

f ניתנת לחישוב בזמן פולינומי ולכן קיים פולינום p_f שחוסם את זמן הריצה שלה. לכן,

$$|f(x, y)| \leq p_f(|x| + |y|)$$

עבור קלט x נגדיר את העדות $b_{|x|}=a_1\#a_2\#.....\#a_{p_f(|x|+p(|x|))}$. **נרצה כי יתקיים** $x\in S\iff\exists b\forall y...V'(x,b,y)=1$ **ואז נקבל כי** $S\in\Sigma_2$. לכן ננסה להגדיר את V'. נבחין כי יתכן מצב שבו העצות יעבדו עלינו (כלומר נקבל כי b אינו שרשור של עצות ולכן הרצת A יחזיר תשובה לא נכונה). לכן, אנחנו נרצה להזכר כי לכל בעיה $S\in NPC$ קיימת רדוקציה עצמית לבעיה. בפרט ראינו כי קיימת רדוקציה עצמית לבעיה SAT. אנחנו נבצע מהלך דומה:

$V'(x,b,y)$:

1. חשב את $\varphi=f(x,y)$.
2. אם $A(a_{|\varphi|},\varphi)=0$ אזי החזר 0. (אם אמר שיש השמה מספקת, אנחנו לא בהכרח נאמין לו ־ אנחנו נבדוק שאכן יש השמה מספקת:
3. בעזרת האלגוריתם A וסדרת העצות $\{a_n\}_{n=1}^{|\varphi|}$ נבצע רדוקציה עצמית כדי למצוא השמה מספקת כנ"ל ל־φ כאשר נחליף כל שאלת אורקל מהצורה "האם $\varphi'\in SAT$" להרצת האלגוריתם A עם העצה $a_k=a_{|\varphi'|}$ כלומר $A(a_{|\varphi'|},\varphi')$. נבחין שבהינתן שאכן העצות לא עבדו עלינו ־ האלגוריתם ישיב תשובה נכונה (יחזיר השמה מספקת).
4. אם ההשמה שהתקבלה בשלב הקודם מספקת את φ החזר 1. אחרת, החזר 0

זמן הריצה: נבחין כי זמן הריצה פולינומי. א' עלותו חישוב פולינומי שכן מדובר ברדוקציית קארפ. 2 מדובר בהרצת אלגוריתם A פולינומי. בשלב 3 אנחנו מריצים אלגוריתם A (פולינומי) מס' פעמים ברדוקציה עצמית (פולינומי) וסה"כ מדובר בפולינומי. 4 גם כן מדובר בבדיקה על השמה מסוימת (כמו בדיקה ב־NP) ולכן גם כן פולינומי. סה"כ V' אלגוריתם פולינומי.

כעת נוכיח את נכונות האלגוריתם V' (עבור Σ_2).

$x\in S$: אזי לכל y המקיים $|y|\le p(|x|)$ מתקיים $f(x,y)\in SAT$ (מנכונות רדוקציית קארפ, שהרי $S\in NP$). עבור העדות $b_{|x|}$ (שרשור העצות שהאלגוריתם מצפה לקבל) לכל y האלגוריתם V' ימצא השמה מספקת עבור $f(x,y)$ ויחזיר 1. ולכן,

$$x\in S\Longrightarrow\forall y:|y|\le p(x):f(x,y)\in SAT\Longrightarrow\exists b:|b|\le p_f(|x|+p(|x|))\forall y:|y|\le p(|x|):V'(x,y,b)=1$$

$x\notin S$: אזי קיים $y:|y|\le p(|x|)$ כך ש־$f(x,y)\notin SAT$. עבור אותו y, V' תמיד ישיב 0 שכן V' מחזיר 1 אמ"מ קיימת השמה מספקת לנוסחה. אך $f(x,y)=\varphi$ לא ב־SAT ולעולם לא ימצא לה השמה מספקת. לכן:

$$x\notin S\Longrightarrow\forall b:|b|\le p_f(|x|+p(|x|))\exists y:|y|\le p(|x|):V'(x,y,b)=0$$

למעשה, כל הסיבוך באלגוריתם היה בגלל השלב הנ"ל של הנכונות ־ למנוע מצב שהאלגוריתם יקבל עצה לא טובה והוא יתבלבל ויחזיר תשובה לא נכונה. כיוון שהוא תמיד בודק אם קיים השמה מספקת לפני (באמצעות הרדוקציה העצמית) הוא לא יטעה לעולם.

לכן סה"כ קיבלנו כי $S\in\Sigma_2$. ראינו כי $\Pi_2\subseteq\Sigma_2$. לכן נקבל כי $\Sigma_2=\Sigma_3$. ממשפט קריסת ההירככיה, $PH=\Sigma_2$. כנדרש.

■

מסקנה 159. הוספת כוח חישובי ־ חישוב לא יוניפורמי ("עצות") מוסיפה כוח חישובי. אם היה מתקיים כי $NP\subseteq P/Poly$ (כל בעיה ב־NP ניתן לתאר כאלגוריתם פולינומי שמקבל סדרת עצות) אזי היה ניתן להכריע בעיות כמו SAT באמצעות אלגוריתם פולינומי שמקבל סדרת עצות. כלומר ־ אכן חישוב לא יוניפורמי לכל בעיה ב־NP היה גורר קריסת ההירככיה. לכן, קיבלנו עוד אינטואיציה למה (כנראה) $NP\neq P$.

9.4 תרגול

משפט 160. לכל בעיית הכרעה $S\subseteq\{0,1\}^*$ מתקיים $L\in P/_{2^n}$.

הוכחה: תהי $S\in\{0,1\}^*$. לכל n נגדיר את a_n להיות מחרוזת בינארית כאשר הביט ה־i הוא 1 אמ"מ המחרוזת באורך n בסידור הלקסיקוגרפי שייך לשפה. כעת, נגדיר מ"ט M שבהינתן x ועצה $a_{|x|}$ מחזירה את הביט בעצה במיקום הרלוונטי בסידור הלקסיקוגרפי של המחרוזת באורך $|x|$. מתקיים:

1. לכל n $|a_n|=|\{0,1\}^n|=2^n$
2. לכל x: המכונה M מחזירה תשובה נכונה.

זמן הריצה של M פולינומי בקלט ולכן $S\in P/_{2^n}$.

הגדרה 161. קבוצה S נקראת דלילה אם קיים פולינום p כך שלכל n:

$$|S\cap\{0,1\}^n|\le p(n)$$

כלומר כמות המילים בכל אורך חסומה פולינומית.

בהתאמה נגדיר: $\text{sparse}=\{S\mid$ קבוצה דלילה $S\}$

משפט 162. $P^{sparse}=P/_{Poly}$. כלומר, $S\in P/_{Poly}$ אמ"מ יש רדוקציית קוק מ־S לקבוצה דלילה.

הוכחה:

בכיוון ראשון, תהי $S\in P/_{Poly}$. לכן קיים אלגו' פולינומי A, סדרת עצות אינסופית $\{a_n\}_{n=0}^{\infty}$ ופולינום p כך ש:

לכל n: $|a_n|\le p(n)$

לכל x: $x\in S\iff A(a_{|x|},x)=1$

נרצה למצוא קבוצה דלילה $S' \in Sparse$ ואלגו' פולינומי $B^{S'}$ שמכריע את S באמצעות גישות אורקל ל-S'.
נניח בה"כ כי לכל n מתקיים $|a_n| = p(n)$ וכי p פולינום עולה ממש (ניתן להניח זאת, אחרת נחליפו בכזה שכן). אחרת, פשוט נרפד באפסים.
כעת לכל n ולכל $1 \le i \le p(n)$ נגדיר a_n^i מחרוזות באורך $p(n)$ באופן הבא:

$$a_n^i[j] = \left\{ \begin{array}{cc} 1 & i = j \\ 0 & o.w \end{array} \right\}$$

לדוגמה עבור $a_n = n^2$ אזי $a_1^1 = 1$. וכן $a_2^1 = 1000, a_2^2 = 0100, a_2^3 = 0010, a_2^4 = 0001$.
נגדיר את הקבוצה:

$$S' = \{a_n^i, n \in \mathbb{N}, a_n[i] = 1\}$$

נסביר. אם למשל $a_2 = 1011$ אזי במקרה שלנו, a_2^1 יכנס כי $a_2^1 = 1 \wedge a_2[1] = 1$. וגם a_2^3 יכנס כי $a_2^3 = 1 \wedge a_2[3] = 1$. גם a_2^4 מסיבה דומה. a_2^2 לא יכנס כי $a_2[2] = 0$.
נטען - S' היא קבוצה דלילה: מס' המילים באורך n חסום ע"י מס' האפשרויות למקם את הביט 1 במחרוזת באורך $p(n)$ ולכן $|S' \cap \{0,1\}^n| \le p(n)$. ובהסבר יותר טוב, מי שנכנס לקבוצה אלו מחרוזות באורך n גם כן, שביט כלשהו חייב להיות בהן דלוק. יש בדיוק $p(n)$ מיקומים בהם ניתן למקם את הביט הבודד, לכן מס' האפשרויות לכל n ב-S' הוא לכל היותר $p(n)$.
כעת נגדיר את רדוקציית הקוק:
$B^{S'}(x)$: לכל $1 \le i \le p(n)$ שאל את האורקל האם $a_{|x|}^i \in S'$ ושמור את התשובה כ-σ_i.
הגדר $a = \sigma_1\sigma_2...\sigma_{p(|x|)}$ והחזר את תשובת $A(\sigma, x)$
$B^{S'}$ מבצע כמות פולינומית של גישות אורקל, ולבסוף מריץ אלגוריתם פולינומי. לכן גם פולינומי. בנוסף מתקיים $B^{S'}(x) = A(a_{|x|}, x)$ לכל x לכן הוא מכריע את S, כנדרש. נבחין שהאורקל פשוט משחזר את העצה המקורית, ואז מריץ את A איתה.
בכיוון שני, תהי $S \in P^{sparse}$. לכן קיימת $S' \in Sparse$ עם פולינום p שחוסם את כמות המילים ואלגוריתם פולינומי $A^{S'}$ שמכריע את S.
נסמן ב-t את הפולינום שחוסם את זמן הריצה שלו. בוודאי כי אורך המילה הארוכה ביותר ש-A שואל את האורקל חסום ב-$t(n)$, שכן לכתוב את השאלה לוקח זמן. בהתאם לכך נגדיר את העצה a_n להיות שרשור של כל המילים ב-S' עד אורך $t(n)$, מופרדות ע"י תו שאינו בא"ב של S'. אזי:

$$|a_n| = \sum_{i=1}^{t(n)} \left[|S' \cap \{0,1\}^i| \cdot (i+1) \right] \le \sum_{i=1}^{t(n)} [p(i) \cdot (i+1)] \le \sum_{i=1}^{t(n)} [p(t(n)) \cdot (t(n)+1)]$$

$$\le [p(t(n)) \cdot (t(n)+1)] \cdot t(n)$$

וזה אכן פולינומי, כלומר גודל העצות חסומות.
כעת נגדיר אלגוריתם B שבהינתן קלט ועצה מריץ את $A^{S'}$ ומחליף כל גישת אורקל בבדיקה של העצה האם המילה מופיעה בעצה או שלא.
בעצם, האלגוריתם דאג שהעצות יהיו כל המילים ב-S' עד אורך $t(n)$. לכן כל פעם שהאלגו' יבצע שאילתא מהצורה האם $q \in S'$? פשוט נבצע סריקה של העצה המתאימה - שכוללת את כל המילים ב-S' שיכולות להיות רלוונטיות - כיוון ש-S' דלילה, הצלחנו ליצור עצה כזו באורך קטן.
האלגוריתם הנ"ל מבצע אותו מס' צעדים כמו $A^{S'}$ ובכל צעד סורק לכל היותר את כל מחרוזות העצה שהיא באורך פולינומי, לכן גם האלו הנ"ל פולינומי, בנוסף האלגוריתמים מבצעים פעילות זהה לכן גם B מכריע את S. סה"כ $S \in P/_{Poly}$, כנדרש.

10 הרצאה 10

10.1 סיבוכיות מקום

בהקשר של סיבוכיות מקום נעסוק במודל של מכונת טיורינג עם שלושה סרטים.
1. סרט קלט - Read Only
2. סרט פלט - Write Only. נניח כי ניתן לנוע קדימה בלבד.
3. סרט עבודה - קריאה וכתיבה.

בהקשר של סיבוכיות מקום - אנחנו לא נמדוד את אורך הקלט או הפלט שלנו. מבחינתנו - סיבוכיות המקום היא סרט העבודה.

הגדרה 163. עבור פונקציה $s : \mathbb{N} \to \mathbb{N}$ נאמר כי בעיה $A \in DSPACE(s(n))$ אם קיימת מכונת טיורינג דטרמיניסטית M שמכריעה את A כך שלכל x מס' התאים בסרט העבודה ש-M משתמשת בו חסום ע"י $s(|x|)$.

הגדרה 164. עבור פונקציה $t : \mathbb{N} \to \mathbb{N}$ נאמר כי בעיה $A \in DTime(t(n))$ אם קיימת מכונת טיורינג דטרמיניסטית M שמכריעה את A כך שלכל x זמן הריצה חסום ע"י $t(|x|)$.

משפט 165. מתקיים כי $P = \bigcup_{t \in Poly} DTIME(t(n))$

משפט 166. לכל $t : \mathbb{N} \to \mathbb{N}$ מתקיים כי $DTIME(t(n)) \subseteq DSPACE(t(n))$. (שכן אם מס' הצעדים שהמכונה מבצעת הוא $t(n)$ אזי בהכרח מס' התאים שהיא יכולה להשתמש בהם מוגבל ב$t(n)$).

משפט 167. לכל $s(n) \geq log(n)$ מתקיים כי $DSPACE(s(n)) \subseteq DTIME(2^{O(s(n))})$ [באופן כללי מתקיים כי $DSPACE(s(n)) \subseteq DTIME(n \cdot 2^{O(s(n))})$].

הוכחה: נתאר קונפיגורציה של המכונה לאחר t צעדים בתור כל המידע שצריך לשמור כדי להמשיך את ריצת המכונה. נשים לב שאם המכונה הגיעה לאותה קונפיגורציה פעמיים במהלך ריצתה היא בהכרח תכנס ללולאה אינסופית (שהרי לאחר עוד t צעדים תשוב לשם - וכו'). לכן בהינתן שהמכונה עוצרת - מס' הצעדים חסום ע"י מספר הקונפיגורציות האפשריות. אם כן, נרצה לשאול את עצמנו איזה מידע צריך להכלל בקונפיגורציה: 1. מיקום ראש סרט הקלט, מיקום ראש על סרט העבודה, וכיוון שסרט העבודה יכול להשתנות תוך כדי הריצה נרצה את תוכן סרט העבודה, וכן המצב הפנימי הנוכחי של המכונה. לכן מספר הקונפיגורציות האפשריות הינו לכל היותר מס' הצירופים של מידע הקונפיגורציות -

ישנן n אפשרויות למיקום הראש על סרט הקלט, $s(n)$ אפשרויות למיקום סרט העבודה, $2^{s(n)}$ אפשרויות לתוכן סרט העבודה (בכל תא $\{0,1\}$) ולמצב הפנימי של המכונה ישנו קבוע $c = |Q|$ (מס' מצבי המכונה). לכן סה"כ: $n \cdot s(n) \cdot 2^{s(n)} \cdot c = n \cdot 2^{O(s(n))}$ (שכן $s(n)$ נבלע כי בפרט $s(n) \leq 2^{s(n)}$).

במקרה שבו $s(n) \geq log(n)$ אזי $n = 2^{log(n)} \leq 2^{s(n)}$ ולכן נקבל כי זמן הריצה הוא לכל היותר $2^{O(s(n))}$.

■

הגדרה 168. נגדיר את המחלקה הבאה:

$$L = \bigcup_{c=1}^{\infty} DSPACE(clogn)$$

להיות כל הבעיות S שניתנות להכרעה בזמן מקום של $O(logn)$ [עבור $c \in [1, \infty)$].

מסקנה 169. $L \subseteq P$

הוכחה: תהי בעיה $S \in L$ שניתנת להכרעה ב$clogn$ מקום. אזי, אנו יודעים כי לפי הטענה לעיל: $DSPACE(clogn) \subseteq DTIME(2^{O(clogn)}) = DTIME(n^c)$ ולכן S ניתנת להכרעה בזמן פולינומי.

■

הערה 170. השאלה האם $L = P$ (כלומר האם $P \subseteq L$ - כל בעיה שניתנת להכרעה בזמן פולינומי ניתנת להכרעה במקום לוגריתמי) היא שאלה פתוחה במדעי המחשב. לא ידוע כיום, מעריכים כי $P \neq L$ אך למרות השוני (כנראה) בניהן - הן מאוד קרובות. L היא ה"מקבילה" של P בהקשר של מקום.

הגדרה 171. עבור פונקציה $s : \mathbb{N} \to \mathbb{N}$ נאמר כי $S \in NSPACE(s(n))$ אם קיימת מכונת טיורינג לא דטרמיניסטית M שמכריעה (לפי הגדרת ל"ד) את S כך שלכל x מס' התאים המקסימלי בסרט העבודה שM משתמשת בריצה על x חסום ע"י $s(|x|)$.

הגדרה 172. נגדיר את המחלקה הבאה: $NL = \bigcup_{c=1}^{\infty} NSPACE(clogn)$ להיות כל הבעיות שניתנות להכרעה במקום לוגריתמי ע"י מכונת טיורינג לא דטרמיניסטית. (הערה: זו המקבילה של NP בהקשר של מקום).

משפט 173. $L \subseteq NL$ (זה ברור מאליו, שכן כל מכונת טיורינג דטרמיניסטית היא בפרט דטרמיניסטית). וכן - זו שאלה פתוחה האם $NL = L$.

10.2 NL - שלמות

הגדרה 174. רדוקציית Log-space מבעיית הכרעה A_1 לבעיית הכרעה A_2 הינה פונקציה f שניתנת לחישוב במקום **לוגריתמי** כך שלכל x מתקיים:

$$x \in A_1 \iff f(x) \in A_2$$

משפט 175. יהיו A_1, A_2 זוג בעיות הכרעה כך שקיימת רדוקציית *Log-space* בניהן. אזי, קיימת רדוקציית קארפ בניהן.

הוכחה: לפי המשפט שראינו, זמן הריצה של הרדוקציה במקרה זה הוא לכל היותר $2^{O(clogn)} = O(n^c)$ עבור $c > 0$ כלשהו, ולכן זמן הריצה של חישוב הפונקציה f פולינומי וקיימת רדוקציית קארפ בניהן.

הגדרה 176. בעיית הכרעה A נקראת NL-שלמה אם:

1. $A \in NL$
2. לכל $A' \in NL$ קייצת רדוקציית Log-space מA' לA.

הגדרה 177. נגדיר את הבעיה הבאה:

$$ST-CONN = \{(G,s,t) | \text{G is directed graph, s,t} \in V, \exists : s \rightsquigarrow t\}$$

הערה 178. הבעיה מוגדרת עבור גרף מכוון בלבד. עבור גרף לא מכוון - $ST - CONN$ היא לא NL שלמה.

משפט 179. $ST - CONN$ היא NL שלמה.

הוכחה:

1. ראשית נרצה להוכיח כי $ST - CONN \in NL$. לכן נתאר אלגוריתם לא דטרמיניסטי M להכרעת $ST - CONN$:

$M(G, s, t)$:

א. $current = s$

ב. בצע $|V(G)|$ פעמים:

$i.$ בחר קודקוד $next \in \Gamma(current)$ באופן לא דטרמיניסטי.

$ii.$ אם $next = t$ החזר 1.

$iii.$ $current = next$

ג. החזר 0.

קל לראות שאכן המוודא הנ"ל (האלגוריתם) מכריעה את $ST - CONN$ שכן מסלול כנ"ל יכול להיות לכל היותר באורך n. בהנחה כי קיים פתרון, קיימת הרצה של המכונה עבורה יוחזר 1. אם אין מסלול, לכל הרצה שהיא יוחזר 0.

סיבוכיות המקום: נרצה לבחון מה אנחנו שומרים על סרט העבודה. אנו שומרים את $current$ על סדר העבודה ומשנים את ערכו תוך כדי, בדומה שומרים את $next$. וכן שומרים מונה עבור $|V(G)|$. נבחין כי השכנים של $current$ כבר על סרט הקלט. נבחין כי בהינתן n קודקודים עלינו לשמור אינדקס לכל קודקוד (בין 1 לn). לכן, $current, next$ מיוצגים ע"י זוג ערכים ולייצג מס' לכל היותר n נדרשים לכל היותר $O(logn)$ ביטים. כעת, עבור המונה באופן דומה - המונה יגדל בין 1 לn ועל מנת לייצג ערך שהוא לכל היותר n נדרשים לכל היותר $O(logn)$ ביטים. סה"כ סיבוכיות המקום הדרושה היא $3logn = O(logn)$ ביטים.

לכן סה"כ הוכחנו כי $ST - Conn \in NL$.

2. נרצה להראות כי לכל $A \in NL$ קיימת רדוקציית *Log-space*: $A \to ST - CONN$.

תהי $A \in NL$. אזי, קיימת מכונת טיורינג לא דטרמיניסטית שמכריעה את A כך שהמקום שהיא משתמשת בו חסום ב$clogn$ עבור $c > 0$ קבוע. נרצה עבור קלט x לבנות גרף מכוון G וזוג קודקודים s, t כך ש:

$$x \in A \iff (G, s, t) \in ST - CONN$$

נכנה את הקודקודים בG להיות כל הקונפיגורציות האפשריות בריצה של M על x, וכן נגדיר קודקוד חדש t.

הקשתות בG יוגדרו כך: לכל C_1, C_2 קונפיגורציות כך שניתן לעבור מC_1 לC_2 בצעד אחד, נוסיף קשת (C_1, C_2).

נגדיר את הקודקוד s להיות הקונפגירוציה ההתחלתית. כמו כן, נוסיף קשת מכל קונפיגורציה מקבלת (עבורה הגענו למצב מקבל) לt.

לפי הבניה הנ"ל - בבירור שהנכונות טריוואלית שכן: $x \in A \iff$ קיימת ריצה של המכונה עבורה $M(x) = 1 \iff$ קיים מסלול קונפיגורציות מהקונפגפיגורציה ההתחלתית לקונפיגורציה כלשהי מקבלת $\iff$ קיים מסלול בG מs לt.

סיבוכיות המקום של f: נשים לב שבכל שלב נתון מספיק לשמור על סרט העבודה שתי קונפיגורציות בלבד (שכן הפונקציה בסך הכל עוברת זוג זוג של קונפיגורציות ויוצרת קשת ביניהן אם ניתן לעבור ביניהם, שכן היא מנחשת קונפיגורציה C_1, מנחשת C_2. בודקת אם אכן ניתן לעבור $C_1 \to C_2$ ואם כן מוסיפה קשת (C_1, C_2) לסרט הפלט.)

מהו אורך של קונפיגורציה?

1. מיקום על סרט הקלט - על סרט הקלט n מיקומים ולכן נצטרך $O(logn)$ ביטים לשמור.
2. מיקום על סרט העבודה - על סרט העבודה $clogn$ מיקומים (שכן $A \in NL$) ולכן נצטרך $O(log(clogn))$ ביטים לשמור.
3. תוכן סרט העבודה - סרט העבודה באורך $clogn$ ולכן נצטרך $clogn$ ביטים על מנת לשמור אותו.
4. מצב פנימי של המכונה - קבוע כלשהו c'.

סה"כ קונפיגורציה אחת חסומה באורך $logn + clogn + c' + log(clogn) = O(logn)$.

ולכן סה"כ המקום הנדרש הוא $O(logn)$.

לכן סה"כ הראינו כי $ST - CONN \in NLC$.

■

משפט 180. $NL \subseteq P$

הוכחה: תהי $S \in NL$. אזי, כפי שראינו קיימת רדוקציית לוג־ספייס מS ל$ST - CONN$. כיוון שסיבוכיות המקום לוגריתמית, סיבוכיות הזמן של ההפרה פולינומית. כיוון ש$ST - CONN \in P$ (למשל: BFS או DFS) אזי נקבל כי $S \in P$.

■

משפט 181. המחלקה L סגורה לרדוקציית *Log-space* (כלומר, תהי $S \in L$ כך שקיימת רדוקציית לוג־ספייס מS לS': $S \to S'$. אזי, $S' \in NL$)

10.3 תרגול

הגדרה 182. בעבור V קבוצה של מילים (שפה) נגדיר את הכוכב קליני V^* להיות:

$$V^* = \bigcup_{i=0}^{\infty} V^i$$

משפט 183. NL סגורה לכוכב קליני.

הוכחה: תהי $L \in NL$ לכן קיימת מ"ט לא דטרמיניסטית כך ש$M(x)$ מכריעה את L בזיכרון לוגריתמי. נבנה מכונה M_{L^*} שמכריעה את L^*.

$M_{L^*}(x)$:

1. שפור זוג מצביעים i, j (בעלות $O(logn)$).
2. אתחל $i, j = 0$
3. כל עוד $j \neq |x| - 1$:

i. בחר באופן לא דטרמיניסטי ערך $j \in \{i+1, ..., |x|-1\}$

ii. העבר את הראש קורא כותב למיקום הi בסרט הקריאה.

iii. ספלץ את M_L ועקוב אחרי הראש קורא כותב שלו. בכל פעם שM_L ניגשת לביט בקלט שלה ־ אם היא ניגשת לביט בטווח $[0, j-i]$ העבר אליו את הביט הרלוונטי מהקלט. אחרת תעביר אליו "$-$".

iv. אם M_L דחה ־ דחה.

ivv. מחק את הזיכרון שM_L השתמשה בו.

4. אשר.

סיבוכיות מקום: עבור המצביעים i, j נצטרך $O(log(|x|))$ מקום. בנוסף נספלץ את M_L על קלט בגודל לכל היותר $|x|$ לכן נצטרך עוד $clog(|x|)$ מקום. לכן סה"כ המיקום שנשתמש בו לוגריתמי.

נכונות: נוכיח שהמכונה מכריעה את הבעיה.

$x \in L^*$ אזי $x = w_1w_2....w_i$ כך שלכל $1 \leq j \leq i$ מתקיים כי $w_j \in L$. לכן, קיימת ריצה בה נבחר בכל איטרציה את j כך ש$x[i:j] = w_k$ ויתקיים $w_k \in L$. בנוסף, יתקיים $M_L(w_k) = 1$. לכן בריצה זו בכל איטרציה M_L לא תדחה ־ והמכונה M_{L^*} תאשר.

$x \notin L^*$ אזי בכל ריצה, קיימת איטרציה של הלולאה בה $x[i:j] \notin L$ ולכן $M(x[i:j])$ תדחה, ולכן גם M_{L^*} תדחה.

לכן סה"כ M_{L^*} מכריעה את L^* במיקום לוגריתמי. כנדרש.

■

משפט 184. רדוקציית *Log-space* היא טרנזטיבית. כלומר אם f היא פונקציית *log-space* מL_1 לL_2 וg היא גם מL_2 לL_3 אזי קיימת פונקציית *log-space* מL_1 לL_3.

הוכחה: נבחין שאם נריץ את הפונקציות אחת אחרי השניה ־ יתכן שהזיכרון של הפעלת f יהיה ארוך מדי ואז נצטרך יותר מדי מקום. כלומר ־ תוצאת הפונקציה f היא אורך פולינומי ולא נוכל לשמור אותה על סרט העבודה של g (שהוא בגודל לוגריתמי!) ־ גם מהגדרת המודל לא נוכל לכתוב אותה על סרט הקלט/פלט. עם זאת, לא אכפת לנו מזמן הריצה. לכן לא נשמור בשום מקום את הפלט של הרדוקציה וכשנצטרך אותו נחשבו כל פעם מחדש.

$h(x)$:

1. ספלץ את g. בכל פעם שg ניגשת לביט הi של הקלט שלה מריצים את השלבים הבאים:

- הרץ את $f(x)$ וספור כמה ביטים $f(x)$ כותבת לסרט הפלט שלה.
- לכל ביט שמגיע לפני הביט הi מתעלמים מהכתיבה.
- כשf פולטת את הביט הi מעבירים אותו לg ומוחקים את הזיכרון שהשתמשנו בו בעבור f ועבור המונה, וחוזרים לספלץ את g.
- אם f סיימה את הריצה שלו לפני, מעבירים לg את "$-$" וממשיכים בספלוץ.

ניתוח זיכרון: עבור החישובים של f צריך $log(|x|)$ מקום וכך גם עבור המונה. עבור החישובים של g צריך $log(f(|x|))$ מקום. זה לוגריתמי כי $f(|x|)$ באורך פולינומי. עבור המצביע לראש של g צריך גם כן $log(f(|x|))$ מקום. לכן סה"כ נדרש למקום לוגריתמי.

נכונות: המכונה מחשבת את $g(f(x))$ כנדרש, כיוון שבכל פעם שg ניגשת לביט של $f(x)$ היא מקבלת את הביט הנכון. מתקיים כי:

$$x \in L_1 \iff f(x) \in L_2 \iff g(f(x)) \in L_3$$

לכן הראינו כי קיימת רדוקציית *log-space* מL_1 לL_3.

משפט 185. תהי בעיית ההכרעה הבאה:

$Cycle = \{\ G \mid$ גרף לא מכוון המכיל מעגל $G\ \}$

אזי, $Cycle \in NL$.

הוכחה: האלגוריתם מתבסס על כך שבגרף G יש מעגל אמ"מ בגרף $G \backslash \{u, v\}$ יש מסלול מu לv.

$A(G)$:

1. בחר באופן לא דטרמיניסטי קשת (u, v) כך ש $u, v \in V$.
2. ספן $current = v$.
3. בצע $|V(G)|$ פעמים:

i. בחר באופן לא דטרמיניסטי $next \in \Gamma(current)$

ii. אם $\{u, v\} = \{current, next\}$ דחה.

iii. אם $next = u$ אשר.

iv. $current = next$

4. דחה.

ניתוח זיכרון ־ יש לשמור 4 מצביעים: $u, v, current, next$ וכן למונה. כל אחד בגודל $log(|x|)$ מקום ולכן סיבוכיות המקום לוגריתמית.

נכונות: קיים מעגל בG $\iff$ קיימת קשת (u, v) כך שב$G' = G \backslash \{(u, v)\}$ קיים מסלול מu לv $\iff$ קיימת הרצה של $A(G)$ כך ש$\{u, v\}$ נבחר באופן לא דטרמיניסטית כך שניתן להגיע מu לv ללא מעבר בקשת (u, v) $\iff$ $A(G) = 1$.

11 הרצאה 11

משפט 186. המחלקה $DSPACE(O(log^2 n))$ סגורה לרדוקציית *Log-space*.

משפט 187. NL סגורה לרדוקציית *Log-space*.
הוכחה לא פורמלית: תהי $B \in NL$ כך שB $A \leq_{log\text{-}space}$. אזי נסמן את פונקציית הרדוקציה בf. בהינתן קלט $x \in A$ נפעיל את $f(x)$ ולה קיים אלגוריתם בNL כי $f(x) \in B$.

משפט 188. $NTIME(P(n)) \subseteq DTIME(O(2^{p(n)}))$ עבור פולינום p.

11.1 משפט סביץ'

בהקשר של זמן, הדרך היחידה לעבור ממכונת טיורינג לא דטרמיניסטית למכונת טיורינג דטרמיניסטית היא מעבר על כל האפשרויות ־ זה דבר אקספוננציאלי. עם זאת, בהקשר של מקום משפט סביץ' טוען:

משפט 189. (משפט סביץ') מתקיים: $NL \subseteq DSPACE(O(log^2 n))$

משפט סביץ למעשה טוען כי ניתן להפוך כל אלגוריתם לא דטרמיניסטי לאלגוריתם דטרמיניסטי ־ במחיר של ריבוע המקום בלבד.

על מנת להוכיח את הטענה נעזר בסגירות של $DSPACE(O(log^2 n))$. כלומר, אם קיימת רדוקציית Log-space $A \to B$ כך ש$B \in$ $DSPACE(O(log^2 n))$ אזי $A \in DSPACE(O(log^2 n))$. לכן, כל שנצטרך לעשות הוא להוכיח כי בעיה NL־שלמה ב$DSPACE(O(log^2 n))$.
אם נראה זאת, אזי:

$$\forall S \in NL : S < ST - CONN \in DSPACE(O(log^2 n))$$

ומסגירות $DSPACE(O(log^2 n))$ נקבל כי לכל $S \in NL : S \in DSPACE(O(log^2 n))$ כלומר אכן ישנה הכלה. לכן:

משפט 190. $ST - CONN \in DSPACE(O(log^2 n))$
הוכחה: עלינו להראות אלגוריתם **דטרמיניסטי** שמכריע את $ST - CONN$ כאשר המקום שנשתמש בו הוא $O(log^2 n)$. עבור (G, u, v, k) נגדיר פרדיקט:

$$\phi(G, u, v, k) := \begin{cases} 1 & \exists P : u \rightsquigarrow v : |P| \leq k \\ 0 & o.w \end{cases}$$

נשים לב כי $\phi(G, s, t, |V(G)|) = 1 \iff (G, s, t) \in ST - CONN$. לכן אנחנו נחשב את הפרדיקט הנ"ל באופן דטרמיניסטי ב$O(log^2 n)$ מקום.
נגדיר אלגוריתם דטרמיניסטי (רקורסיבי) שמחשב את ϕ.
$M(G, u, v, k)$:
1. אם $k = 1$: אם $(u, v) \in E$ החזר 1, אחרת החזר 0.
2. לכל קודקוד $w \in V(G)$ בצע:
   i. חשב את $M(G, u, w, \lceil \frac{k}{2} \rceil)$ ושמור את התוצאה כσ_1.
   ii. חשב את $M(G, w, v, \lceil \frac{k}{2} \rceil)$ ושמור את התוצאה כσ_2.
   iii. אם $\sigma_1 = \sigma_2 = 1$ השב 1.
3. השב 0.

בבירור האלגוריתם עושה את הדרוש. נבחין כי ישנו טריידאוף אדיר ־ הוא לא שומר שום דבר על הדרך בשביל לא להתעסק עם מקום: אך הוא משלם בזמן. זה אלגוריתם שרץ בזמן ריצה אקספוננציאלי. כעת נרצה להוכיח כי סיבוכיות המקום היא $O(log^2 n)$:
נחלק את החישוב ל2 ־ כמה מקום דורשת קריאה בודדת, וכמה קריאות יש:
קריאה בודדת ־ שמירת u $(O(logn))$ ביטים, שמירת v $(O(logn))$ ביטים, מזהה w $O(logn)$ ביטים, הערך של k $O(logn)$ ביטים, משתנים קבועים σ_1, σ_2 $O(1)$ ביטים, סה"כ קריאה בודדת עולתה $O(logn)$ ביטים במקום.
נבחין כי בזכות תכונת השימוש החוזר במקום: בעת שחישבנו את σ_1 והפונקציה חזרה, המקום שלה מתפנה לחלוטין ומשמש לחישוב σ_2. לכן המקום המקסימלי שנצטרך להחזיק בזיכרון בו זמנית הוא מיקום קריאה בודדת כפול מס' הקריאות שיושבות במחסנית. מס' הקריאות הרי הוא $\frac{k}{2^i} = 1$ כלומר $i = log_2(k)$. נעיר שוב: בזכות העובדה כי כל אחת מהקריאות (ישנן 2 קריאות) יכולות להעיף את המקום שביצעה הקריאה הקודמת ־ נקבל כי באמת כל קריאה בודדת עולתה $O(logn)$ מקום. ולכן סה"כ סיבוכיות המקום:

$$log(n) \cdot log(k) = O(log^2 n)$$

ולכן, $ST - CONN \in DSPACE(O(log^2 n))$.

■

11.2 משפט אימרמן (1987)

משפט 191. (משפט אימרמן) $NL = coNL$

כלומר, משפט אימרמן טוען כי כל בעיה S שניתנת לפתרון באמצעות מ"ט לא דטרמיניסטית בסיבוכיות מקום לוגריתמית, גם הבעיה המשלימה שלה $\overline{S}$ ניתנת לפתרון באמצעות מ"ט לא דטרמיניסטית בסיבוכיות מקום לוגריתמית. לשם כך: אנחנו נראה מס' טענות עזר.

משפט 192. תהי $S \in coNL$ כך ש$S \in NLH$ (NL קשה) אזי $NL = coNL$

הוכחה: ראשית נוכיח כי $NL = coNL \Longrightarrow NL = coNL$.$coNL \subseteq NL$. כלומר נניח הכלה בכיוון ראשון, וכעת נראה הכלה בכיוון שני. אם כן:

$$S \in NL \Longrightarrow \overline{S} \in coNL \subseteq NL \Longrightarrow \overline{S} \in NL \Longrightarrow S \in coNL$$

כעת תהי $S \in coNL \cap NLH$. אם כן, תהי $A \in coNL$. אזי, $\overline{A} \in NL$. אנו יודעים כי $S \in NLH$ ולכן: $\overline{A} \leq_{Log\text{-}space} S$. באופן שקול $\overline{S} \leq_{Log\text{-}space} A$. אנו יודעים כי $\overline{S} \in NL$ (שכן $S \in coNL$) ומסגירות NL לרדוקציית $Log\text{-}space$ נקבל כי $A \in NL$. כלומר סה"כ הראינו $coNL \subseteq NL$ ולכן $coNL = NL$. ■

לכן, על מנת להוכיח את משפט אימרמן, נוכיח כי $ST-CONN \in coNL$ (שכן היא NL קשה) ואז סיימנו. באופן שקול, נרצה להוכיח כי $\overline{ST-CONN} \in NL$ ואם נראה זאת סיימנו את ההוכחה.

נגדיר את הפונקציה הבאה:

$$Total-Reach(G,s) = |\{u | \exists s \rightsquigarrow u\}|$$

כלומר, מס' הקודקודים שניתן להגיע מs אליהם בG. נרצה להראות אלגוריתם לא דטרמיניסטי שמחשב את הפונקציה הנ"ל. לפני כן ־ עלינו להסביר מהו חישוב לא דטרמיניסטי של פונקציה, הרי עד היום אלגוריתם לא דטרמיניסטי עבורנו הכריע בעיות הכרעה. לכן:

הגדרה 193. נאמר כי מכונת טיורינג לא דטרמיניסטית מחשבת פונקציה f אם לכל x:

1. קיימת ריצה עבורה $M(x) = f(x)$
2. בכל ריצה מתקיים $M(x) \in \{f(x), \perp\}$

לכן, נניח שאכן הצלחנו לחשב פונקציה זו באופן לא דטרמיניסטי במקום לוגריתמי. בהמשך נוכיח זאת. נרצה אבל להראות שזה יפתור לנו את הבעיה:

משפט 194. אם קיימת מ"ט לא דטרמיניסטית שמחשבת את $Total-Reach(G,s)$ במקום לוגריתמי, אז $\overline{ST-CONN} \in NL$.

הוכחה: נתאר מ"ט לא דטרמיניסטית שמכריעה את $\overline{ST-CONN}$ במקום לוגריתמי.

:$M(G,s,t)$

1. חשב את $TR(G,s) \to c_1$. אם $c_1 = \perp$ השב 0.
2. חשב את $TR(G \backslash \{t\}, s) \to c_2$. אם $c_2 = \perp$ השב 0.
3. החזר 1 אמ"מ $c_1 = c_2$

שהרי, אם לא קיים מסלול מs לt, זה אמ"מ מס' הקודקודים שניתן להגיע מs בגרף ללא t שווה למס' הקודקודים שניתן להגיע מs בגרף עם t.

כיוון שTR חשיב במיקום לוגריתמי, והרי שמרנו c_1, c_2 ב$O(logn)$ ביטים, ולכן סה"כ המיקום לוגריתמי ונקבל $\overline{ST-CONN} \in NL$. ■

כעת כל שנותר בהוכחה הוא להראות שניתן לחשב את $TR(G,s)$ במיקום לוגריתמי.

עבור גרף G, קודקוד s ומס' i נסמן בR_i את קבוצת הקודקודים שניתן להגיע אליהם מs תוך לכל היותר i צעדים. נבחין כי:

$R_0 = \{s\}$ (הקודקוד s בלבד). לכל $i \geq 1$ נבחין כי:

$$R_i = R_{i-1} \cup \{u | \ \exists w \in R_{i-1} : (w,u) \in E\}$$

נבחין כי $TR(G,s) = |R_{|V|}|$. נחשב זאת ב$|V|$ איטרציות כאשר באיטרציה הi נניח כי אנחנו יודעים את $|R_{i-1}|$ ונחשב את $|R_i|$.

אלגוריתם לא דטרמיניסטי לחישוב $|R_i|$ בהינתן הגודל של $|R_{i-1}|$:

1. בחר באופן לא דטרמיניסטי $1 \leq g \leq |V|$.
2. בדוק אם $|R_i| \geq g$. אם הבדיקה נכשלה החזר $\perp$.
3. בדוק אם $|R_i| \leq g$. אם הבדיקה נכשלה החזר $\perp$.
4. החזר $|R_i| = g$ (מצאת את g).

נבחין שאם הגענו לשלב 4 אזי $g \leq |R_i| \leq g$ ולכן $|R_i| = g$. מה שנותר לנו הוא להסביר כיצד אנחנו בודקים אם $g \leq \geq |R_i|$.

נבחין ששלב 2 הוא שאלה מהצורה "האם קיים" ־ כלומר האם קיימים לפחות g קודקודים שיש מסלול מs אליהם לאחר i צעדים.

שלב 2:

1. $counter = 0$
2. לכל קודקוד $u \in G$:
   - i. בדוק אם יש מסלול מs לu באורך לכל היותר i צעדים. אם כן $counter = counter + 1$.
3. החזר 1 אמ"מ $counter \geq g$.

מקום: עלינו לשמור $counter$ $O(logn)$, וכן לשמור קודקוד u וכן בדיקה כזו לוקחת $O(logn)$ מקום [ניחוש קודקוד בכל שלב]. סה"כ $O(logn)$ סיבוכיות המקום של שלב 2.

כעת נרצה להבחין כי שלב 1 גם כן לוקח $O(logn)$ מקום וגם 4. אם נוכיח כי שלב 3 עלותו גם כן $O(logn)$ מקום סיימנו את ההוכחה.

שלב 3: אנחנו צריכים לבדוק האם $|R_i| \leq g$. באופן שקול, נרצה לבדוק האם $|\overline{R}_i| \geq |V| - g$. נבחין כי $\overline{R}_i$ הוא מס' הקודקודים שלא ניתן להגיע מs אליהם בתוך לכל היותר i צעדים.

א. $counter = 1$ (מודד את מס' הקודקודים שהשתכנענו שלא שייכים לR_i)

ב. לכל קודקוד $u \in G$ בצע:

i. $counter2 = 1$ (מודד את מס' הקודקודים שאנו השתכנענו שבR_{i-1} ואינם u ואין מהם קשת לu)

ii. לכל קודקוד $w \in G$ שאינו u:

1. בדוק אם $w \in R_{i-1}$ (קיים מסלול מs לw באורך לכל היותר $i-1$ צעדים).
2. אם מצאנו מסלול וגם אין קשת (w, u) אזי $counter2++$ (הקודקוד u בוודאות לא בR_i!).

iii. אם $counter2 = |R_{i-1}|$ אזי $counter1++$.

ג. אם $counter1 \geq |V| - g$ אזי החזר 1, אחרת החזר 0.

סיבוכיות מקום: כל ההבאים במקום לוגריתמי - $counter1, counter2, u, w$. וכן הבדיקה $w \in R_{i-1}$ ניתנת כמו שהסברנו קודם במקום לוגריתמי. סה"כ סיבוכיות המקום לוגריתמית.

סה"כ - האלגוריתם (**אלגוריתם לא דטרמיניסטי לחישוב** $|R_i|$ **בהינתן הגודל של** $|R_{i-1}|$) ניתן לחישוב פר איטרציה במקום לוגריתמי. לכן, התהליך של החישוב $|R_{|V|}|$ מתבצע במקום לוגריתמי בתהליך איטרטיבי. בסיום, מתקיים $TR(G,s) = |R_{|V|}|$, ובאמצעות $TR(G,s)$ אנו יכולים להכריע את $\overline{ST-CONN}$ בזמן לוגריתמי כפי שראינו. לכן סה"כ $ST-CONN \in coNL$ (שכן המשלימה שלה בNL) ולכן כיוון שהיא בעיה NL-קשה נקבל כי $coNL = NL$, כנדרש!

■

11.3 תרגול

משפט 195. (משפט סביץ' המוכלל) תהי $s(n) \geq log(n)$ שניתנת לחישוב ב$s(n)$ זיכרון מתקיים:

$$NSPACE(O(s(n))) \subseteq DSPACE(O((s(n))^2)$$

הוכחה: תהי $A \in NPSACE(O(s(n))$. נגדיר שפה חדשה A' באופן הבא:

$$A' = \{x \circ 1 \circ 0^{2^{s(|x|)}} | x \in A\}$$

נבחין כאשר y היא מילה כלשהי בA', אזי $|x|$ מהווה חלק לוגריתמי של y. כדי להכריע האם $x \in A$ נדרש זיכרון (לפי הגדרת A) של $s(|x|) = log(2^{s(|x|)}) \leq log(|y|)$ ולכן $|s(|x|)|$ הוא לוגריתמי בy.

ראשית, נשים לב כי $A' \in NSPACE(O(logn))$. שכן: בהינתן קלט y לA':

1. יש לבדוק שy בפורמט החוקי. ניתן לממש זאת ע"י מונה שסופר עד $|y|$ ולכן ניתן לממש בזיכרון של $log(|y|)$ [צריך לבדוק שיש $2^{s(|x|)}$ אפסים בסוף, ניתן לזכור את $|x|$ באופן לוגריתמי ולספור עם מונה. וכן יש לבדוק שלאחר x מקבלים 1].
2. יש לבדוק כי $x \in A$ וזאת ניתן לעשות בזיכרון $s(|x|)$ כפי שראינו, שזה הרי לוגריתמי בy כפי שאמרנו.

לכן סה"כ $A' \in NSPACE(O(logn))$. לכן, לפי משפט סביץ' מתקיים כי $A' \in DSPACE(O(log(n))^2)$.

כעת נוכיח כי $A \in DSPACE(O(s(n))^2)$. להלן אלגו' שמכריע את A עם כמות מקום רצויה - $M_A(x)$:

א. הגדר $y = x \circ 1 \circ 0^{2^{s(|x|)}}$

ב. הכרע האם $y \in A'$ והחזר את התשובה.

נראה כי שלב ב' ניתן להכריע בזיכרון דטרמיניסטי $DSPACE(O(log(n)^2)$ וכן:

$$s(|x|) \leq log(|y|) \leq 2s(|x|)$$

ולכן את שלב 2 ניתן לממש ב$O(log(n))^2 \leq O((2s(|x|)^2) = O(s(|x|)^2)$.
לכן סה"כ המכונה $M_A(x)$ מכריעה את x ב$DSPACE(O(s(n)^2))$.

השאלה שננסה לענות עליה - היא אם מוסיפים עוד מקום לאלגוריתם (מקום משמעותי): האם זה מוסיף לו כוח חישובי?

משפט 196. (משפט הירכיית המקום) לכל שתי פונקציות $S_1, S_2 : \mathbb{N} \to \mathbb{N}$ כך ש:
1. $S_1(n) \geq log(n)$
2. $S_2(n) = \omega(S_1(n))$ (למעשה משמעות הדבר כי S_2 גדולה ממש מS_1 ברמת החסם. למשל אם $S_1 = O(n)$ אזי S_2 לא יכולה להיות $O(n)$ אלא $O(nlogn), O(n^2)$ וכו'..)

אזי, $DSPACE(O(S_1(n)) \neq DSPACE(O(S_2(n))$ (כלומר, ההכלה $S_1 \subseteq S_2$ טריוואלית. משמעות הדבר כי ההכלה השנייה לא נכונה. כלומר - יש בעיות שניתן להכריע באמצעות $S_2(n)$ מקום ולא ניתן עם $S_1(n)$ מקום).

הוכחה:

על מנת להוכיח את הטענה, נגדיר בעיה ונוכיח שהיא נמצאת ב$DSPACE(O(S_2(n)))$ ולא נמצאת ב$DSPACE(O(S_1(n)))$.
למעשה, אנחנו נבצע הוכחה דומה להוכחה שראינו במודלים חישוביים על ATM. רק שנראה בעיה שהיא כן כריעה בניגוד לATM. נגדיר את הבעיה הבאה:

$$A = \{(M,w)| \text{ M is turing Machine that aceepts w in at most } 2^{S_2(n)} \text{ steps and uses at most } S_2(n) \text{ work tape cells s.t: } n = |M,w|\}$$

1. נוכיח כי $A \in DSPACE(O(S_2(n))$ - נגדיר אלגוריתם שבהינתן קלט (M,w) מקצה $s_2(n)$ תאים על סרט העבודה לצורך חישובים של $M(w)$ ובנוסף שומר מונה של מס' הצעדים שM מבצעת ומריץ את $M(w)$. אם M חרגה מגבולות המקום או הזמן (לפי המונה) אזי נחזיר 0. אחרת - נשיב 1 (אם אכן $M(w) = 1$). נבחין כי מס' המקום שאנו נדרשים להשתמש: ראשית $s_2(n)$ מקום שהקצנו ועוד מקום עבור המונה, ספירה עד $s_2(n)$ כלומר סה"כ $log(s_2(n))$ מקום נוסף עבור המונה, וסה"כ המקום הוא $s_2(n) + log(s_2(n))$ והרי $s_2(n) > log(s_2(n))$ ולכן סה"כ המקום הוא $O(S_2(n))$ וקיבלנו כי $A \in DSPACE(O(S_2(n))$.
2. כעת נוכיח כי $A \notin DSPACE(O(S_1(n))$ - נב"ש כי $A \in DSPACE(O(S_1(n))$. אזי קיימת מכונת טיורינג M_A שמכריעה את A תוך שימוש ב$O(S_1(n))$ תאים בסרט העבודה. כלומר קיימים קבועים $c, c' > 0$ כך שM_A משתמשת בלכל היותר $cS_1(n)$ תאים ומבצעת לכל היותר $2^{c' \times S_1(n)}$ צעדים (מממשפט חסם על הזמן). אנו יודעים כי $S_2(n) \in \omega(S_1(n))$ ולכן קיים n_0 כך שלכל $n \geq n_0$ מתקיים $s_2(n) > max\{c, c'\}s_1(n)$.

נגדיר מכונת טיורינג D שמקבלת תיאור של מכונת טיורינג M כלשהי, מריצה את $M_A(M, M)$(כלומר גם המילה w היא תיאור המכונה) ומחזירה תשובה הפוכה. כלומר:

$$D(M) = 1 - M_A(M, M)$$

(ניתן לחשוב על זה כמו לכסון - M_A רצה רק על האלכסון).
מה מחזירה המכונה D כשמריצים אותה על תיאור של עצמה? (והנה השלב שזה מתקשר לATM), כלומר $D(D)$? הרי שהיא בהכרח מריצה מכונה שמכריעה - ולכן היא בהכרח עוצרת. נחלק למקרים:

1. $D(D) = 1$. אזי, $M_A(D, D) = 0$ (כלומר D לא עוצרת על המילה D) ולכן $D(D) = 0$

נעיר כי החלק שבו $S_1(n)$ נכנסת להוכחה הוא החלק שבו אנחנו מריצים את $D(D)$. עבור קלט גדול מספיק, המשאבים שM_A באמת צריכה כדי לסיים את חישובה על (D, D) הם קטנים בהרבה מהמגבלות שבשפה A. (ואם לא מתקיים $|D| \geq n_0$ אזי נוסיף מצבים פיקטיביים למכונה (מצבים ללא חצים אליהם)).
כמה מקום וצעדים עושה המכונה D שמריצים אותה על עצמה? היא מריצה $M_A(D, D)$. נסמנן $n = |(D, D)|$. נשים לב כי D מבצעת לכל היותר $2^{s_2(n)}$ צעדים ומשתמשת בלכל היותר ב$S_2(n)$ תאים בסרט העבודה. כלומר $D(D)$ עומדת בגבולות הזמן והמקום של A אך $(D, D) \notin A$ [לכן $D(D) = 0$ אבל $D(D) = 1$ בסתירה].
הכיוון השני זהה ופחות מפורט:

2. $D(D) = 0$, אזי $M_A(D, D) = 1$ כלומר $(D, D) \in A$ ולכן $D(D) = 1$ בסתירה.

לכן, לא קיימת M_A כנ"ל - ולכן $A \notin DSPACE(O(S_1(n))$.
כנדרש.

■

12 הרצאה 12

12.1 אלגוריתמים הסתברותיים

הגדרה 197. מכונת טיורינג הסתברותית הינה מכונת טיורינג לא דטרמיניסטית כאשר אם יש כמה אפשרויות בצעד מסויים, כל אפשרות נבחרת בהסתברות שווה.

אם נרצה אז ניתן להניח כי ישנן בדיוק 2 אפשרויות (תמיד ניתן לפצל את הבחירה ל־2 קבוצות וכך הלאה).

הפלט של מכונת טיורינג הסתברותית הינו משתנה מקרי שמקבל ערך 1 או 0 בהסתברות מסוימת.

דוגמה 198. נתבונן בדוגמה לבעיה הבאה ־ בהינתן שני פולינומים P,Q על n משתנים בייצוג של נוסחה עם כפל, חיבור וסוגריים (עם ()() ולא לאחר שנפתחו). יש להכריע האם $P=Q$.
לדוגמה, קלט אפשרי לבעיה הינו:

$$P(x,y,z)=(x+2y)(3y-z)$$

$$Q(x,y,z)=y(3x-6y-2z)-xz$$

קל לראות כי $P=Q$. נבחין כי בדרך הנאיבית לפתור את זה ־ ניתן לפתוח את הסוגריים, ולהשוות מקדמים. אם כל המקדמים זהים ־ קיבלנו אותו פולינום. נבחין כי חסם תחתון לזמן הריצה גם אם מתעלמים מזמן פתיחת הסוגריים ־ הוא מס' המונומים ($ax^iy^jz^k$) האפשריים (שניתן להשוות ולבדוק בסוף). נסמן בd את הדרגה המקסימלית של מונום $d=i+j+k$ אזי זמן הריצה חסום מלמטה ע"י מס' המונומים האפשריים ושקול לפתרון המשוואה $x_1+x_2+.....+x_n\le d$ שחסום לפי בדידה ע"י $\binom{n+d-1}{d}$. על מנת להפוך זאת לשוויון נרצה להוסיף משתנה דמה ונקבל $\sum_{i=1}^{n+1}x_i=d$ שכעת חסום ע"י $\binom{n+d}{d}$. זה כמובן לא פולינומי ובפרט גדול מ$O(n^d)$. לכן באופן דטרמיניסטי לא ידוע על אלגוריתם פולינומי שמסוגל לעשות זאת.
באמצעות הסתברות ניתן להכריע באופן הבא: נבחר באופן אקראי ערכים למשתנים $x_1,....,x_n$ מתוך הקבוצה $\{1,...,2d\}$. נציב ונבדוק אם $P(x_1,...,x_n)=Q(x_1,...,x_n)$ ונחזיר 1 אמ"מ קיבלנו אותו ערך. זמן הריצה כאן כעת פולינומי, ומהי ההסתברות שנהיה צודקים?
אם הפולינומים זהים, ההסתברות שנצדוק תהיה 1. ואם הם שונים, מהי ההסתברות שנבחר ערכים $\{x_1,...,x_n\}$ כך ש־ $P(\{x_i\}_{i=1}^n)=Q(\{x_i\}_{i=1}^n)$? נגדיר את הפולינום $K(\{x_i\}_{i=1}^n)=P(\{x_i\}_{i=1}^n)-Q(\{x_i\}_{i=1}^n)$. נבחין שהאלגוריתם יטעה אמ"מ יבחר שורש של הפולינום K.
נטען את הטענה הבאה (למת שוורץ זיפל):

$$Pr[R(x_1,....,x_n)=0]\le\frac{d}{|S|}$$

כאשר המשתנים $x_1,....,x_n\in S$. לכן אצלנו נקבל כי:

$$Pr[K(x_1,....,x_n)=0]\le\frac{d}{2d}=\frac{1}{2}$$

לכן ההסתברות שנהיה צודקים ־ היא לפחות חצי, והאלגוריתם פולינומי. אם נבחר ערכים מתוך הקבוצה kd נראה כי ההסתברות תהיה $\frac{1}{k}$. ניתן גם לבצע אמפליפיקציה.

משפט 199. תהי S בעיית הכרעה ותהי M מ"ט הסתברותית פולינומית שמכריעה את S ולא טועה אף פעם (כלומר לכל x: $Pr[M(x)=\text{correct answer}]=1$), אזי קיימת מכונת טיורינג דטרמיניסטית פולינומית שמכריעה את S.
הוכחה: נגדיר מכונת טיורינג דטרמיניסטית M': M' תריץ את M ובכל פעם שיש לה בחירה לא דטרמיניסטית היא תבחר את האפשרות הראשונה. קל לראות שהמכונה דטרמיניסטית, פולינומית, ומכריעה את S גם כן.

הגדרה 200. (המחלקה RP). נגדיר את המחלקה RP (אלגו' הסתברותיים עם זמן ריצה פולינומי) באופן הבא ־ נאמר כי $S\in RP$ בעיית הכרעה אם קיים אלגוריתם הסתברותי פולינומי A כך ש:
1. $x\in S\Longleftarrow Pr[A(x)=1]\ge\frac{1}{2}$
2. $x\notin S\Longleftarrow Pr[A(x)=0]=1$

(כלומר, עבור קלטים שלא כחלק מהבעיה האלגוריתם תמיד צודק. עבור קלטים שחלק מהבעיה, האלגוריתם צודק בהסתברות לפחות חצי).

משפט 201. $RP \subseteq NP$
הוכחה לא פורמלית: הרי שאם $S \in RP$, קיים אלגוריתם הסתברותי ־ מכונת טיורינג לא דטרמיניסטית, כך שאם $x \in S$ אזי קיימת הרצה (שכן ההסתברות לפחות חצי) עבורה $A(x) = 1$ ואם $x \notin S$ אזי לכל הרצה (לכל y) $A(x) = 0$.

משפט 202. שאלה פתוחה: לא ידוע האם $NP \subseteq RP$.

הגדרה 203. (המחלקה BPP ־ טעות חד צדדית): תהי S בעיית הכרעה. נאמר כי $S \in BPP$ אם קיים אלגוריתם הסתברותי פולינומי A כך ש:
1. $x \in S \Longleftarrow Pr[A(x) = 1] \geq \frac{2}{3}$
2. $x \notin S \Longleftarrow Pr[A(x) = 0] \geq \frac{2}{3}$

(מאפשרים טעות בשני הצדדים, אך בשני המקרים רוצים שההסתברות שנצדוק תהיה לפחות $\frac{2}{3}$).

הגדרה 204. נגדיר את המחלקות הבאות: RP_1, RP_2.
1. נאמר כי $S \in RP_1$ אם קיים אלגוריתם הסתברותי פולינומי A ופולינום p כך ש:
   - $x \in S \Longrightarrow Pr[A(x) = 1] \geq \frac{1}{p(|x|)}$ (ההסתברות שיחזיר תשובה נכונה די נמוכה)
   - $x \notin S \Longrightarrow Pr[A(x) = 0] = 1$ (תמיד יחזיר תשובה נכונה)
2. נאמר כי $S \in RP_2$ אם קיים אלגוריתם הסתברותי פולינומי A ופולינום p כך ש:
   - $x \in S \Longrightarrow Pr[A(x) = 1] \geq 1 - \frac{1}{2^{p(|x|)}}$ (לרוב יחזיר תשובה נכונה)
   - $x \notin S \Longrightarrow Pr[A(x) = 0] = 1$ (תמיד יחזיר תשובה נכונה)

למעשה מחלקות אלו תמיד מחזירות תשובה נכונה עבור קלט שלא שייך לבעיה. הן שונות (לכאורה) זו מזו בקלט שכן שייך לבעיה. בפועל מסתבר שהן לא שונות ־

משפט 205. מתקיים $RP_1 = RP_2$
הוכחה: כיוון אחד ברור, $RP_2 \subseteq RP_1$. לכן נרצה להראות $RP_1 \subseteq RP_2$.
תהי $S \in RP_1$ ויהי A האלגו' הפולינומי הסתברותית וp הפולינום מהגדרת RP_1. הרעיון יהיה לבנות אלגו' שמריץ את A מס' מסוים של פעמים על מנת להוכיח שכעת ההסתברות של $Pr[A(x) = 1]$ תגדל. נגדיר אלגוריתם A' שמריץ את A $k(x)$ פעמים (בהמשך נגדיר את $k(x)$) ויחזיר 1 אמ"מ A החזיר 1 בלפחות אחת מן ההרצות.
נבחין כי אם $x \notin S$ עדיין יתקיים כי לפי הגדרת A $Pr[A'(x) = 0] = Pr[\bigwedge_{i=1}^{k} A'_i(x) = 0] = 0$ (שכן בכל הרצה A יחזיר 0).
כעת, נניח כי $x \in S$.

$$Pr[A'(x) = 1] = 1 - Pr[A'(x) = 0] = 1 - Pr[A(x) = 0 \text{ in all k times}] = 1 - Pr[A(x) = 0]^k \geq 1 - (1 - \frac{1}{p(|x|)})^k$$

שכן, אם $x \in S$ אזי $Pr[A(x) = 1] \geq \frac{1}{p(|x|)}$ ובאופן שקול $Pr[A(x) = 0] = 1 - Pr[A(x) = 1] \leq 1 - \frac{1}{p(|x|)}$. נמשיך עם הפיתוח. נראה כי מאי השוויון $(1 - \frac{1}{n})^n \leq \frac{1}{e}$, ונגדיר $k(x) = p(|x|)^2$ ונקבל:

$$1 - (1 - \frac{1}{p(|x|)})^k = 1 - (1 - \frac{1}{p(|x|)})^{p(|x|)^2} \geq 1 - (\frac{1}{e})^{p(|x|)} = 1 - \frac{1}{e^{p(|x|)}} \geq_{e>2} 1 - \frac{1}{2^{p(|x|)}}$$

ולכן סה"כ עבור $k(x) = p(|x|)^2$ קיבלנו כי $Pr[A'(x) = 1] \geq 1 - \frac{1}{2^{p(|x|)}}$.
נראה כי A' פולינומי שכן מריץ $p(|x|)^2$ פעמים אלגו' פולינומי. לכן סה"כ קיבלנו כי $S \in RP_2$.
מכאן קיבלנו $RP_1 = RP_2$, כנדרש.

■

משפט 206. $RP \subseteq BPP$

משפט 207. (חסם צ'רנוף) יהיו $w_1, ..., w_n$ משתנים מקריים שווי התפלגות שמקבלים ערכים ב$\{0, 1\}$ בלתי תלויים. עבור כל $\varepsilon > 0$ מתקיים:

$$Pr[\frac{\sum_{i=1}^{n} w_i}{n} \geq \mathbb{E}[\frac{\sum_{i=1}^{n} w_i}{n}] + \varepsilon] \leq e^{-2\varepsilon^2 k}$$

משפט 208. תהי $S \in BPP$. אזי לכל פולינום p קיים אלגו' פולינומי הסתברותי פולינומי A כך שלכל x:

$$Pr[A(x) = \text{Correct answer}] \geq 1 - \frac{1}{2^{p(|x|)}}$$

הוכחה:
תהי M המכונה שמכריעה את S לפי הדרישות של BPP. נגדיר מ"ט M' שמריצה את M $k(x)$ פעמים ומחזירה את תשובת הרוב. ($k(x)$ יקבע בהמשך).

יהי $w_i \in \{1,0\}$ משתנה אינדיקטור לכך שהמכונה M טועה בהרצה הi כלומר,

$$w_i = \begin{cases} 1 & \text{M return wrong answer} \\ 0 & \text{o.w} \end{cases}$$

מתי M' טועה? כאשר מס' הטעויות גדול מחצי (שהרי, M' מחזירה את תשובת הרוב. אם היא טעתה בתוצאה הסופית ־ משמעות הדבר שהיא טעתה בלפחות חצי מההרצות). כלומר:

$$\frac{\sum_{i=1}^{k} w_i}{k} > \frac{1}{2}$$

נרצה לחסום את ההסתברות שהמאורע הנ"ל קרה. לכן נחשב את התוחלת עבור כל w_i.

$$\mathbb{E}[w_i] = 0 \cdot Pr[w_i = 0] + 1 \cdot Pr[w_i = 1] = Pr[w_i = 1] = Pr[\text{M return wrong answer}] \leq \frac{1}{3}$$

$$\Longrightarrow \mathbb{E}[\frac{\sum_{i=1}^{k} w_i}{k}] = \frac{1}{k}\sum_{i=1}^{k}\mathbb{E}[w_i] \leq \frac{1}{k} \cdot k \cdot \frac{1}{3} = \frac{1}{3}$$

כלומר ־ התוחלת שנטעה היא בשליש מההרצות לערך. אם כן, חישבנו את התוחלת. אנו נרצה לחשב את ההסתברות לטעות של M'. כעת נעזר בחסם צ'רנוף.

$$Pr[\text{M'(x) wrong}] = Pr[\frac{\sum_{i=1}^{k} w_i}{k} > \frac{1}{2}] = Pr[\frac{\sum_{i=1}^{k} w_i}{k} > \frac{1}{3} + \frac{1}{6}]$$

$$\leq Pr[\frac{\sum_{i=1}^{k} w_i}{k} > \mathbb{E}[\frac{\sum_{i=1}^{k} w_i}{k}] + \frac{1}{6}] \leq_{\varepsilon=\frac{1}{6}} e^{-2\cdot\frac{1}{36}k} = e^{\frac{-k}{18}} = \frac{1}{e^{\frac{k}{18}}}$$

לכן נגדיר $k(|x|) = 18p(|x|)$ ונקבל:

$$= \frac{1}{e^{\frac{18p(|x|)}{18}}} = \frac{1}{e^{p(|x|)}} > \frac{1}{2^{p(|x|)}}$$

וסה"כ הסיכוי של M' לטעות כפי שרצינו, לכן $Pr[A(x) = \text{Correct answer}] \geq 1 - \frac{1}{2^{p(|x|)}}$. ■

הגדרה 209. עבור מ"ט הסתברותית M עם זמן ריצה t ניתן להגדיר מ"ט דטרמיניסטית M' שמקבלת סדרה של בחירות $r \in \{0,1\}^{t(n)}$ (מס' ביטים כמס' הצעדים שעושה המכונה) ומריצה את $M(|x|)$ כאשר הבחירות שלה נקבעות מראש לפי המחרוזת r.
לכל x מתקיים:

$$\Pr_{r\in\{0,1\}^{t(n)}}[M'(x,r) = 1] = \Pr[M(x) = 1]$$

שכן, ההסתברות לבחירה בסדרה כלשהי r היא בדיוק ההסתברות לריצה של הבחירות הללו במכונה ההסתברותית M.

משפט 210. שאלה פתוחה ־ האם $BPP \subseteq P$ (הכיוון השני ברור).

משפט 211. $BPP \subseteq P/Poly$
הוכחה: תהי $S \in BPP$, לכן קיימת מ"ט הסתברותית פולינומית M כך שלכל x (לאחר אמפליפיקציה שראינו שניתן לבצע)

$$Pr[M(x) = \text{Correct answer}] \geq 1 - \frac{1}{2^{|x|+1}}$$

(הוכחנו זאת עבור כל פולינום $p(|x|)$. בפרט עבור הפולינום $p(n) = n+1$)

נסמן ב־t את הפולינום שחוסם את זמן הריצה של M, אזי לפי ההגדרה השקולה קיימת מ"ט דטרמיניסטית בעלת זמן ריצה t כך שלכל x:

$$\Pr_{r\in\{0,1\}^t}[M'(x,r)=\text{Correct answer}]=Pr[M(x)=\text{Correct answer}]\geq 1-\frac{1}{2^{|x|+1}}$$

בשלב הראשון בהוכחה, נאמר כי סדרת בחירות $r_n\in\{0,1\}^{t(n)}$ היא סדרת בחירות טובה עבור קלטים באורך n אם לכל $|x|=n$ מתקיים כי $M'(x,r)=\text{Correct answer}$. נרצה להוכיח כי לכל n קיימת סדרת בחירות טובה. כדי להוכיח זאת אנחנו נשתמש בשיטה ההסתברותית שראינו במבנים בדידים (נתבונן על מאורע, אם נוכיח שההסתברות שיקרה היא חיובית ממש, אזי קיימת בפרט דגימה בתוך מרחב המדגם ־ סדרת הבחירות, שהיא סדרת בחירות טובה). כעת נחשב את ההסתברות עבור סדרת בחירות טובה. נגדיר A להיות מאורע כי r היא סדרת בחירות טובה עבור קלטים באורך n.

$$\Pr_{r\in\{0,1\}^{t(n)}}[A]=\Pr_{r\in\{0,1\}^{t(n)}}[\forall x\in\{0,1\}^n:M'(x,r)=\text{Correct answer}]=1-\Pr_{r\in\{0,1\}^{t(n)}}[\exists x\in\{0,1\}^n:M'(x,r)=\text{Wrong answer}]$$

$$1-Pr_{r\in\{0,1\}^{t(n)}}[\bigcup_{x\in\{0,1\}^n}M'(x,r)=\text{Wrong answer}]$$

$$\geq_{\text{Union Bound}}1-\sum_{x\in\{0,1\}^n}Pr[M'(x,r)=\text{Wrong answer}]\geq 1-\sum_{x\in\{0,1\}^n}\frac{1}{2^{n+1}}=1-|\{0,1\}^n|\times\frac{1}{2^{n+1}}=1-\frac{1}{2}=\frac{1}{2}>0$$

לכן, ישנה בהכרח סדרת בחירות כנ"ל. כלומר, לכל n קיימת סדרת בחירות טובה.
בשלב השני בהוכחה, נקח את המכונה M' ונגדיר את סדרת העצות $\{a_n\}_{n=1}^{\infty}$ כך שלכל $n\in\mathbb{N}$ העצה a_n תהיה סדרת הבחירות הטובה עבור קלטים באורך n (שראינו בשלב הראשון של ההוכחה שקיימת). מתקיים כי M' פולינומית וכן: לכל n $|a_n|\leq t(n)$. וכן לכל x:

$$x\in S\iff M'(a_{|x|},x)=1$$

ולכן $S\in P/Poly$. נבחין שהרעיון בהוכחה היה להראות באמצעות השיטה ההסתברותית ־ שתמיד קיימת סדרת בחירות שתניב בהסתברות ממש 1 תשובה נכונה תמיד. לכן עבור סדרת הבחירות ששווה לעצה ממש, נקבל כי $S\in P/Poly$.

■

הערה 212. היחס בין BPP ל־NP גם כן לא ידוע. עם זאת, ידוע כי $BPP\subseteq\Sigma_2$.

משפט 213. $BPP\subseteq\Sigma_2$
הוכחה: לא למבחן.

משפט 214. $\overline{A}\in BPP\iff A\in BPP$. כלומר, $BPP=\overline{BPP}$.

משפט 215. $BPP\subseteq\Pi_2$

משפט 216. שאלה פתוחה: האם $BPP\subseteq NP$.

12.2 תרגול

הגדרה 217. נגדיר את המחלקה:

$$coRP=\{\overline{S}|S\in RP\}$$

בהגדרה שקולה: תהי S בעיית הכרעה. נאמר כי $S\in coRP$ אם קיים אלגו' הסתברותי פולינומי A כך ש:

$$x\in S\Longrightarrow\Pr[A(x)=1]=1$$

$$x\notin S\Longrightarrow\Pr[A(x)=0]\geq\frac{1}{2}$$

משפט 218. $RP \subseteq NP$
הוכחה: תהי $S \in RP$. אזי, קיימת מ"ט הסתברותית פולינומית M_S כך ש:

$$x \in S \Longrightarrow \Pr[M_S(x) = 1] \geq \frac{1}{2}$$

$$x \notin S \Longrightarrow \Pr[M_S(x) = 0] = 1$$

תהי M'_S מ"ט שמסמלצת את M_S ובכל פעם שM_S בוחרת בין מס' אפשרויות, M'_S מבצעת בחירה לא"ד בין אותן אופציות.
אם $x \in S$ אזי:

$$\Pr[M_S(x) = 1] = \frac{|number\ of\ computation\ paths\ on\ which\ M\ accepts|}{|number\ of\ the\ machine's\ computations\ path|} \geq \frac{1}{2} > 0$$

לכן, מס' מסלולי הריצה בהן המכונה M_S מקבלת גדול ממש מ0, לכן קיימת בהכרח ריצה של M'_S שבה היא מבצעת סדרת בחירות לא"ד שתואמות למה שהמכונה M_S הגרילה בריצה שקיבלה.
אם $x \notin S$, אזי מהנתון $Pr[M_S(x) = 0] = 1$, לכן לכל סדרת בחירות שתעשה M_S ותסמלץ ע"י בחירות לא דטרמיניסטיות M'_S יוחזר 0.
לכן סה"כ $S \in NP$, כנדרש.

■

משפט 219. RP סגורה לאיחוד. כלומר, יהיו $S_1, S_2 \in RP$ אזי $S_1 \cup S_2 \in RP$
הוכחה: תהיינה $S_1, S_2 \in RP$. לכן קיימות מ"ט הסתברותיות פולינומיות A_1, A_2 כך שלכל $i \in \{1,2\}$ מתקיים:

$$x \in S_i \Longrightarrow \Pr[A_i(x) = 1] \geq \frac{1}{2}, x \notin S_i \Longrightarrow \Pr[A_i(x) = 0] = 1$$

נגדיר מכונה $M(x)$ שתריץ את $A_1(x)$ ואת $A_2(x)$ ותחזיר 1 אמ"מ לפחות אחת מהן החזירה 1. קל לראות שהיא פולינומית שכן מריצה זוג מכונות פולינומיות.
כעת, אם $x \in S_1 \cup S_2$. אזי בהכרח $x \in S_1$ או $x \in S_2$ (יתכן בשתיהן). בה"כ $x \in S_1$ אזי:

$$\Pr[M(x) = 1] = \Pr[A_1(x) = 1 \vee A_2(x) = 1] \geq \Pr[A_1(x) = 1] \geq \frac{1}{2}$$

אם $x \notin S_1 \cup S_2$ אזי $x \notin S_1, S_2$. לכן:

$$\Pr[M(x) = 1] = Pr[A_1(x) = 1 \vee A_2(x) = 1] \leq Pr[A_1(x) = 1] + Pr[A_2(x) = 1] = 0 + 0 = 0$$

לכן $\Pr[M(x) = 1] = 0$ (הסתברות אי שלילית). לכן $S_1 \cup S_2 \in RP$.

■

משפט 220. לכל פונקציה $\alpha : \mathbb{N} \to [0,1]$ נסמן בRP_α מחלקה דומה לRP עם החלפת הקבוע $\frac{1}{2}$ בα. אזי, לכל פולינום p מתקיים:

$$RP_{\frac{1}{p(n)}} = RP = RP_{1-\frac{1}{2^{p(n)}}}$$

משפט 221. המחלקה RP סגורה לחיתוך. כלומר, יהיו $S_1, S_2 \in RP$. אזי, $S_1 \cap S_2 \in RP$.
הוכחה: תהיינה $S_1, S_2 \in RP$. לכן, לפי טענת העזר קיימות מ"ט הסתברותיות פולינומיות A_1, A_2 כך ש:

$$x \in S_i \Longrightarrow \Pr[A_i(x) = 1] \geq 0.724601$$

$$x \notin S_i \Longrightarrow \Pr[A_i(x) = 0] = 1$$

נבנה מכונה $M(x)$ שתריץ את $A_1(x)$ ואת $A_2(x)$ ותחזיר 1 אמ"מ שתיהן החזירו 1. ברור כי M פולינומית שכן מריצה זוג מכונות פולינומיות. כמו כן:
אם $x \in S_1 \cap S_2$ אזי $x \in S_1, x \in S_2$ ולכן:

$$\Pr[M(x) = 1] = \Pr[A_1(x) = 1 \wedge A_2(x) = 1] = \Pr[A_1(x) = 1] \cdot \Pr[A_2(x) = 1] = 0.724601^2 > \frac{1}{2}$$

אם $x \notin S_1 \cap S_2$. אזי, יתכנו שני מקרים. בה"כ $x \notin S_1$. אזי,

$$Pr[A_1(x) = 1] = 0 \Longrightarrow Pr[A_1(x) = 1 \wedge A_2(x) = 1] = 0 \Longrightarrow Pr[M(x) = 0] = 1$$

לכן סה"כ $S_1 \cap S_2 \in RP$.

הגדרה 222. נגדיר עבור בעיית הכרעה S:

$$\chi_S(x) = \begin{cases} 1 & x \in S \\ 0 & x \notin S \end{cases}$$

משפט 223. $RP \subseteq BPP$.
הוכחה: תהי $S \in RP$. אזי לפי טענת עזר קיימת מ"ט הסתברותית פולינומית A כך ש:

$$x \in S \Longrightarrow Pr[A(x) = 1] \geq \frac{2}{3}, x \notin S \Longrightarrow Pr[A(x) = 0] = 1$$

אותה מכונה מקיימת כי $\frac{2}{3} \leq 1 = [Pr[A(x) = 0] \Longleftarrow x \notin S$ ולכן $S \in BPP$

משפט 224. לכל פונקציה $\alpha : \mathbb{N} \to [0, 1]$ נסמן בBPP_α את המחלקה BPP עם החלפת הקבוע $\frac{2}{3}$ בα. אזי לכל פולינום p מתקיים:

$$BPP_{\frac{1}{2}+\frac{1}{p(n)}} = BPP = BPP_{1-\frac{1}{2^{p(n)}}}$$

משפט 225. BPP סגורה לאיחוד. כלומר, תהיינה $S_1, S_2 \in BPP$. אזי, $S_1 \cup S_2 \in BPP$.
הוכחה: תהיינה $S_1, S_2 \in BPP$. לכן לפי טענת עזר, קיימות מ"ט הסתברותיות פולינומיות A_1, A_2 המקיימות:

$$\Pr[A_i(x) = \chi_{S_i}] \geq \frac{5}{6}$$

נגדיר אלגו' הסתברותי $M(x)$: הרץ את $A_1(x), A_2(x)$ והחזר 1 אמ"מ לפחות אחת מהן החזירה 1. בבירור שהוא פולינומי.
- אם $x \in S_1 \cup S_2$ אזי בה"כ $x \in S_1$ ונקבל:

$$\Pr[M(x) = 1] = Pr[A_1(x) = 1 \vee A_2(x) = 1] \geq Pr[A_1(x) = 1] \geq \frac{5}{6} > \frac{2}{3}$$

- אם $x \notin S_1 \cup S_2$ אזי $x \notin S_1$ וגם $x \notin S_2$. לכן,

$$\Pr[M(x) = 0] = Pr[A_1(x) = 0 \wedge A_2(x) = 0] = Pr[A_1(x) = 0] \cdot Pr[A_2(x) = 0] = \frac{5}{6}^2 = \frac{25}{36} > \frac{2}{3}$$

לכן $S_1 \cup S_2 \in BPP$, כנדרש.

13 סיכום

13.1 רדוקציות

1. $IS \leq_m^P SAT$
2. $DS \leq_m^P VC$
3. $SAT \leq_m^P CSAT$
4. $Clique \leq_m^P IS$
5. $VC \leq_m^P IS$
6. $DHC \leq_m^P SAT$
7. $HC \leq_m^P DHC$
8. $AC \leq_m^P Clique$
9. $3COL \leq_m^P SAT$
10. $SSS \leq_m^P VC$
11. $Partition \leq_m^P SSS$

13.2 בעיות NPC

1. IS
2. $Clique$
3. SAT
4. DS
5. VC
6. $3SAT$
7. DHC
8. DS
9. HC (Hamilton cycle)
10. AC (Almost clique)
11. $3COL$
12. $Subset - Sum = SSS$
13. $Partition$
14. ADS

13.3 שאלות פתוחות

1. $NP =^? P$
2. $NP =^? coNP$
3. לכל $k \in \mathbb{N}$: $\Sigma_k =^? \Pi_k$
4. $L =^? P$
5. $NL =^? L$
6. $NP \subseteq RP$
7. $BPP \subseteq P$

13.4 סגירויות וטענות חשובות

1. P סגורה למשלים
2. P סגורה לרדוקציית קארפ
3. $P \subseteq NP$
4. $P \subseteq NP \cap coNP$
5. לכל בעיה S קיימת $S \leq_T \overline{S}$.
6. NP סגורה לרדוקציית קארפ.
7. $A \leq_m^P B \iff \overline{A} \leq_m^P \overline{B}$
8. (משפט קריסת ההירכיה) אם $P = NP$ אזי $PH = P$
9. לכל $k \geq 1$ אם $\Pi_k \subseteq \Sigma_k$ אזי $\Sigma_k = \Sigma_{k+1}$.
10. עבור $k \geq 0$. אם $\Sigma_k = \Sigma_{k+1}$ אזי $PH = \Sigma_k$
11. $NP \subseteq P/Poly \Longrightarrow PH = \Sigma_2$
12. $S \in coNP \wedge S \in NPH \Longrightarrow NP = coNP$
13. $PF \nsubseteq FC$.

14. $P \subseteq NP$
15. $PC \subseteq PF \iff P = NP$
16. רדוקציית קארפ טרנזטיבית.
17. לכל $R \in PC$ בעיית חיפוש, אם $S_R \in NPC$ אזי לR יש רדוקציה עצמית.
18. קיום רדוקציית קארפ $\Longleftarrow$ קיום רדוקציית קוק
19. אם קיימת $S \in NP \cap coNP$ כך שמכל $S' \in NP$ יש רדוקציית קוק לS (שואלים שאלות את S), אזי $NP = coNP$
20. $\Pi_k \subseteq \Sigma_{k+1}$ וכן $\Sigma_k \subseteq \Pi_{k+1}$.
21. משפט אימרמן: $NL = coNL$
22. משפט סביץ': $NL \subseteq DSPACE(O(log^2 n))$
23. משפט הירככית המקום. לכל $S_1 \geq log(n)$ ו$S_2 = \omega(O(S_1))$ מתקיים $DSPACE(O(S_1(n)) \neq DSPACE(O(S_2(n))$

13.5 הגדרות של מחלקות

13.6 בעיות NLC

1. $ST - CONN$

13.7 הסתברות

1. $RP \subseteq NP$
2. $BPP \subseteq P/Poly$
3. $BPP \subseteq \Sigma_2$